NP-Hardness and Additive Inapproximability of Deterministic Communication Complexity

Serge Gaspers* Tao Zixu He† Simon Mackenzie*

Abstract

Given the full truth table of a Boolean function whose input is split between two players, how hard is it to determine the minimum number of bits they must exchange to compute it? We prove that computing deterministic communication complexity is NP-hard. Our exact-hardness result for the standard model was obtained independently and concurrently with Hirahara, Ilango and Loff. This paper includes that result and develops its construction to prove NP-hardness of approximation within every fixed additive error. We also establish exact and fixed-error NP-hardness in Yao's original model, where the players alternate after every bit and only the receiver of the last bit needs to determine the answer. This resolves Yao's 1979 question under his original conventions.

Our standard-model reduction uses interlacing to turn a graph-coloring obstruction into an extra bit of communication. We make this gap repeatable by preserving lower bounds under restrictions of the input sets. Replacing large tuple spaces by short explicit weighted lists then keeps the reduction polynomial in size for every fixed gap. Strict alternation requires a separate encoding.

For the standard model, we also give explicit implications from approximation algorithms to faster algorithms for sparse 3-SAT, without assuming a running-time lower bound for satisfiability. A polynomial-time randomized estimator with error $c \log T / \log\log T$, where T is the number of table entries and $0 \le c < 1/8$, yields a bounded-error 3-SAT algorithm on N variables and $O(N)$ clauses in time $2^{O(N^\alpha)}$ for every fixed $8c < \alpha < 1$. A deterministic polynomial-time estimator with error $c_* \log\log T$, for an effective absolute constant $c_* > 0$, yields a deterministic algorithm in time $2^{O((\log N)^2)}$ on such formulas. The communication protocols being measured remain deterministic.

*University of New South Wales
†University of Massachusetts Amherst

1 Introduction

Alice and Bob know a Boolean matrix M. Alice receives a row index and Bob receives a column index. They must determine the entry at their intersection, communicating as few bits as possible. Their local computation is free, and they must answer correctly for every pair of indices. The deterministic communication complexity $\mathrm{D}(M)$ is the smallest worst-case number of bits used by any such protocol.

We study how hard it is to estimate $\mathrm{D}(M)$ when the algorithm receives the *whole table* as input. Yao posed the exact-computation problem when he introduced communication complexity in 1979 [22].

We prove that computing deterministic communication complexity is NP-hard, and that this remains true when any fixed additive error is allowed. The exact NP-hardness result for the standard model was first presented in earlier versions of this work [6], independently and concurrently with Hirahara, Ilango and Loff [10]. The present paper includes that result and develops the construction into a polynomial-time reduction for every fixed additive gap.

We also prove exact and fixed-error NP-hardness in Yao's original model: the players alternate after every bit, and only the receiver of the last bit needs to determine the answer. The earlier exact-hardness results did not cover these speaking and output conventions. Our alternating construction therefore resolves Yao's question in the model in which it was posed.

The challenge of approximation. The exact-hardness reduction separates a satisfiable-case upper bound from an unsatisfiable-case lower bound by one bit. To tolerate additive error a, we need a gap larger than $2a$. The natural way to enlarge this gap is to combine several such problems. However, a protocol for the combined problem may share communication among them, so their costs need not add, and the lower bound must hold against every such protocol. Combining problems also enlarges the truth table. Since the reduction writes out every entry, the table must have size polynomial in the size of the formula for each fixed gap. Given k, our reductions compute a table and a number B such that the optimal cost is $\leq B$ if the formula is satisfiable and $\geq B + k$ otherwise. We call this an *additive gap* of k bits. We do not establish hardness of approximation within any fixed multiplicative factor above one.

Our tables are total: every entry is specified. Write T for their number of entries. The running time of an estimation algorithm is measured in T. Its output is a number, and it need not produce a protocol. A randomized estimator must meet its error guarantee with probability at least $2/3$ on every table. The protocols whose costs it estimates remain deterministic and must always answer correctly.

Two rules for speaking. In an *unrestricted* protocol, the bits sent so far determine who speaks next, and the same player may send several consecutive bits. In a *strictly alternating* protocol, the players take turns after every bit. Where an unrestricted protocol lets one player send two consecutive bits, an alternating protocol must let the other player speak in between, so the two optimum costs can differ.

The interlacing program. Suppose Alice and Bob must compute $M(x, y)$, where Alice knows x and Bob knows y. Interlacing gives Bob several possible inputs $y_1, \ldots, y_q$, while Alice receives an index i specifying which one matters. Their task is now to compute $M(x, y_i)$. Alice's index acts as a one-step pointer into Bob's tuple, as in pointer chasing [19].

There is a natural protocol: Alice sends i, and the players then compute $M(x, y_i)$ using a protocol for M. But is communicating the index an unavoidable extra cost? Perhaps Bob could

speak first, or the players could exchange information about the index and the answer together, saving bits.

The interlacing framework provides a method for ruling out such savings. It follows the inputs that remain possible after each message and establishes lower bounds for the remaining communication problem. For suitable starting matrices, this analysis forces an optimal protocol to begin with Alice identifying her index [6, 18]. The resulting control over arbitrary protocols is what makes interlacing useful in our hardness reductions.

Interlacing was developed by Mackenzie and Saffidine [18] to refute the exact direct-sum conjecture, and underlies our original NP-hardness reduction [6]. That reduction already had to replace large tuple spaces by polynomial-size input sets. Repeating the construction creates a stronger requirement: each round must preserve the restriction lower bounds needed by the next one. An intermediate version obtained additive inapproximability under ETH using subexponential-time constructions [7].

Polynomial-size gap amplification. Our deterministic construction keeps explicit input sets of polynomial size at every amplification step, so every fixed gap is constructed in deterministic polynomial time. Our randomized construction keeps all inputs during the amplification and represents them implicitly. Only at the end does it sample rows and columns of the final table. A rank-based count shows that the sample preserves the gap with high probability (Theorem 9.4). In both constructions, the lower bounds for unrestricted protocols come from interlacing. Because the two constructions control the table size differently, they give different tradeoffs between the approximation error and the running time of the 3-SAT algorithm that an estimator would imply.

Strict alternation needs a separate construction. The short protocols in our unrestricted construction send multi-bit messages. Making them alternate adds turns, and these extra bits can exceed the gap. We therefore construct a different table, prove its lower bound using the fixed order of speaking, and again keep only a small explicit set of Bob's inputs.

Conventions for the statements. By default, the conversation alone must determine the answer, which we call *public output*. We write $D_A^{\mathrm{alt}}(M)$ for the minimum cost with public output and strict alternation, with Alice starting. Yao's original convention instead allows the receiver of the last bit to use their own input as well as the conversation to determine the answer. We denote its optimum by $Y_{\mathrm{free}}(M)$, where the protocol may choose its starting player independently of the inputs. In both conventions, the bits exchanged must determine when the conversation has ended.

From approximation to satisfiability algorithms. For errors growing with the table size, we state unconditional algorithmic implications: an estimator for communication complexity would yield a 3-SAT algorithm with an explicitly bounded running time. These tradeoffs do not assume ETH or any other running-time lower bound for satisfiability. An inapproximability conclusion follows upon assuming that the resulting 3-SAT algorithm does not exist. The fixed-error NP-hardness results need no such assumption.

1.1 Results

The following table summarizes the fixed-error hardness results and two examples of our general algorithmic implication. The full tradeoff is stated in Theorem 9.7.

Protocol model	Additive error	Consequence for 3-SAT	Estimator type
Unrestricted	Every fixed error	Polynomial time	Deterministic or randomized
Strict alternation	Every fixed error	Polynomial time	Deterministic or randomized
Unrestricted	$c_* \log\log T$	$2^{O((\log N)^2)}$	Deterministic
Unrestricted	$c \log T / \log\log T$, $c < 1/8$	$2^{O(N^\alpha)}$, every $8c < \alpha < 1$	Bounded-error randomized

The last two rows concern formulas with N variables and at most $c_s N$ clauses, for any fixed density bound c_s. Each row describes the algorithm that a polynomial-time estimator would imply; randomized estimators give bounded-error algorithms. The first two rows are NP-hardness results under deterministic reductions.

In each reduction below, Φ is a 3-CNF formula. The YES (satisfiable) case is $\Phi \in \mathsf{SAT}$, and the NO case is $\Phi \notin \mathsf{SAT}$. The reduction computes the threshold B from the formula and the requested gap, without deciding satisfiability.

Theorem 1.1 (Fixed gaps for unrestricted protocols)**.** *For every fixed integer $k \geq 1$, a deterministic polynomial-time algorithm maps a 3-CNF formula Φ to an explicit total Boolean table M and an integer B such that*

$$\Phi \in \mathsf{SAT} \implies \mathrm{D}(M) \leq B, \qquad \Phi \notin \mathsf{SAT} \implies \mathrm{D}(M) \geq B + k.$$

The polynomial may depend on k. The output may be square with power-of-two side length.

A different construction gives the same fixed-gap hardness under strict alternation.

Theorem 1.2 (Fixed gaps under strict alternation)**.** *For every fixed integer $k \geq 1$, a deterministic polynomial-time algorithm maps a 3-CNF formula Φ to an explicit total Boolean table M and an integer B such that*

$$\Phi \in \mathsf{SAT} \implies D_A^{\mathrm{alt}}(M) \leq B, \qquad \Phi \notin \mathsf{SAT} \implies D_A^{\mathrm{alt}}(M) \geq B + k.$$

The polynomial bounds on output size and running time may depend on k.

For additive error a, choose $k > 2a$ and compare the estimate with $B + k/2$. Thus, for every fixed error, estimating either cost in the preceding two theorems in deterministic polynomial time implies $\mathsf{P} = \mathsf{NP}$, and bounded-error randomized estimation implies $\mathsf{NP} \subseteq \mathsf{BPP}$. Changing the starting player costs at most one bit, and so does announcing the answer known to the receiver (Lemma 10.1). By increasing the fixed gap, we therefore also obtain hardness under Yao's output convention.

Corollary 1.3 (Yao's output convention)**.** *For every fixed nonnegative additive error, a deterministic polynomial-time estimator of $Y_{\mathrm{free}}(M)$ on all explicit total Boolean tables implies $\mathsf{P} = \mathsf{NP}$, and a bounded-error randomized one implies $\mathsf{NP} \subseteq \mathsf{BPP}$. The same conclusions hold for estimating the public-output alternating cost with Alice first, Bob first, or a starter chosen independently of the inputs.*

Taking error zero in Corollary 1.3 answers Yao's original question by proving NP-hardness under his speaking and output rules. We do not establish a growing-error bound for the alternating model.

For unrestricted protocols, choosing the gap as a function of the formula size gives an explicit tradeoff between approximation error and the running time of the resulting 3-SAT algorithm

(Theorem 9.7). Here N counts variables in the formula and T counts entries in the communication table. The following two consequences illustrate the randomized and deterministic regimes; the constant $1/8$ uses the sharper size bound for the randomized construction.

Corollary 1.4 (Randomized estimation and 3-SAT running time)**.** *Fix $0 \le c < 1/8$. Suppose a bounded-error randomized polynomial-time estimator approximates $\mathrm{D}(M)$ within additive error $c \log T / \log\log T$ on every sufficiently large explicit total Boolean table with T entries. Then, for every fixed $8c < \alpha < 1$ and every fixed c_s, sparse 3-SAT on N variables and at most $c_s N$ clauses has a bounded-error randomized algorithm with worst-case running time $2^{O(N^\alpha)}$. It suffices for the estimator to work on square tables whose side length is a power of two.*

For the smaller error scale below, the deterministic construction gives a quasipolynomial-time algorithm for sparse 3-SAT.

Corollary 1.5 (Deterministic estimation and 3-SAT running time)**.** *There is an effective absolute constant $c_* > 0$ with the following property. If a deterministic polynomial-time estimator approximates $\mathrm{D}(M)$ within additive error $c_* \log\log T$ on every sufficiently large explicit total Boolean table, then, for every fixed c_s, 3-SAT on N variables and at most $c_s N$ clauses has a deterministic algorithm running in time $2^{O((\log N)^2)}$. It suffices for the estimator to work on square tables whose side length is a power of two.*

These consequences quantify the improvement in satisfiability algorithms that the corresponding estimators would yield. Section 9.4 gives the general tradeoff and explains how to choose the gap for other running-time bounds.

The proofs of Theorem 1.1 and Corollary 1.5 appear in Section 8.2. We prove Theorem 9.7 and Corollary 1.4 in Section 9.4, and Theorem 1.2 and Corollary 1.3 together at the end of Section 10. Section 2 explains the ideas behind these proofs.

1.2 Formal verification and proof dependencies

A companion Lean development provides machine-checked proofs for the constructions and hardness statements covered by its statement correspondence [13]. Its formal statements use explicit interfaces for the cited source reductions and record their hypotheses. The public proof inspector [14] links manuscript statements to the corresponding Lean declarations and exposes their definitions and hypotheses. The repository provides the source, build instructions, and verification records.

Communication lower bounds, explicit constructions, and bounds on output size and running time are distinct proof obligations. The scope of the formalization is determined by the statements proved in the companion development and the external interfaces they assume. The fixed-error NP-hardness results and the implications from estimation to 3-SAT algorithms stated above do not assume ETH. A running-time hypothesis is needed only to turn a growing-error implication into an inapproximability conclusion.

The earlier direct-sum construction of Mackenzie and Saffidine [18] has separately been formalized in Lean [21]; its statements and proofs are available in a public inspector [2].

ChatGPT and Codex were used in proof development and checking; the AI methodology statement describes their roles.

2 Technical overview

Our reduction turns a coloring obstruction into one extra bit of communication. A proper coloring gives a protocol of a known cost $C - 1$. Conversely, interlacing forces every protocol of that cost to commit to a partition of Alice's inputs before Bob reveals which constraint is being tested. In the NO case, the extracted assignment violates a constraint, forcing an additional bit. The difficulty is to prove this against arbitrary interaction, and then to repeat the construction without losing either the lower bound or polynomial output size.

We explain three steps: the one-bit gap, the lower bounds under input deletion that make the gap repeatable, and the explicit short input lists that make every fixed number of rounds a polynomial-time reduction. Strict alternation requires a different encoding, described last.

2.1 Constructing the source matrix

Our source reduction maps a formula to a graph of maximum degree $\Delta = O(1)$ that is three-colorable if the formula is satisfiable (the YES case), and otherwise leaves at least a constant fraction γ of its edges monochromatic under every three-coloring (the NO case, Theorem 5.2). This NO condition survives small deletions. Since $|E| \geq |V|/2$, deleting fewer than $\gamma|V|/(4\Delta)$ vertices destroys fewer than $\gamma|E|/2$ edges, so at least $\gamma|E|/2$ monochromatic edges remain.

The construction takes a base communication matrix R of cost $d = \mathrm{D}(R)$ and adds rows encoding the graph to interlaced copies of R. The first round starts from a simple matrix of known cost (Lemma 5.4). We allow a general base matrix so that the construction can be repeated, with the output of one round as the base matrix of the next. We also need lower bounds that survive deletion of some inputs. We measure the retained rows and columns using fixed positive rational weights, summing to one on each side. These act as fractional counts: they let us control how much each input family contributes, independently of its number of labels. Later, adjusting weights also lets us shorten tuple lists while preserving the counts needed by the proof. The weights do not change communication cost: every protocol must still compute every entry correctly. The deterministic construction never samples from them, and its final output is an ordinary Boolean table with the weights discarded.

A proper three-coloring places the vertices in three groups so that the endpoints of every edge lie in different groups. We call these groups *bins*. The interlacing construction uses a power-of-two number $s \geq 4$ of bins, so the extra bins must be filled without giving the vertices extra colors. We treat each graph vertex as an *item* to be placed in a bin, and add $s - 3$ auxiliary items. At an edge, its two endpoints and the auxiliary items are *active*, and all other items are *inactive*. We require that each bin contain at most one active item at every edge. The auxiliary items are active at every edge, so no two of them share a bin. Since every vertex of our source graph lies on an edge, no vertex can share a bin with an auxiliary item, and the vertices share the remaining three bins, exactly as in three-coloring.

Each item i gives a family of Alice's possible inputs: the pairs (i, u) with u a row of R. A coloring partitions these inputs by the bin of their item. Naming the bin need not identify Alice's item and does not reveal her row of R.

The full construction uses $4s$ labeled groups of items, each containing a copy of the graph's items. For each edge in each group, a binary vector over all items marks the active items of that group with 1 and every other item with 0. We index these vectors by α. In the NO case, every assignment of items to bins has a constant fraction of indices at which one bin contains two active items and a third item that is inactive there, even after deleting a small fraction of the items (Lemma 6.3). We say there is a *collision* at such an index. The inactive third item comes from another group.

There are $4s$ groups but only s bins, and at most s groups can have a bin containing items of no other group. Hence for most groups the bin holding an active pair also holds an item of another group, which is inactive at that index. The reduction never computes an assignment. Instead, the lower-bound proof extracts one from any proposed short protocol.

The matrix has two kinds of Alice inputs. An *item row* consists of an item i and a row u of the base matrix R. The other inputs, called *template rows*, are grouped into s bins and are obtained from R by nested interlaces. Their purpose is to force every protocol of cost at most $C-1$ to begin with $\log s$ consecutive bits from Alice. These bits determine a bin before Bob reveals any index.

Bob's columns have two families. The first supplies a separate index and list of base columns for each bin, together with shared base-column data; it enforces the initial $\log s$ bits. The second uses the same index α and the same list of base columns in every bin. At this index, the template rows use $s-2$ positions of the list, active items use distinct additional positions, and inactive items use one further common position. Thus two active items and one inactive item in the same bin expose $s+1$ positions.

Defining the table without a coloring. A template row carries its bin label p and a tuple z. For each index α, the tuple specifies a pair $z_\alpha = (r, u)$: a position $r \in \{1, \ldots, s-2\}$ and a base row u. Bob's first-family column supplies (α_p, γ_p) for each bin, where γ_p is a list of $2s-1$ base columns. The entry is $R(u, \gamma_{p,r})$, with $(r, u) = z_{\alpha_p}$. Thus each template row uses only its own bin's data. Letting these data vary independently gives the outer interlace that forces Alice's initial messages.

The tuple z is important. If a template row carried just one pair (r, u), its entry after the bin was chosen would not depend on Bob's index α_p. The template subproblem would no longer require that index to be communicated. Instead, z supplies a separate pair for each index: Bob selects which part of Alice's input matters, and then Alice's position r selects which part of Bob's list matters. The players are left with one instance of R. These successive selections are the nested interlaces, with transposition exchanging the players' roles between levels.

The outer bin choice adds another interlace, since Bob has independent data for every bin. We must prove that any sufficiently short protocol starts by resolving that choice; specifying a short protocol that does so would not suffice. Once the prefix is forced, the item rows fall into the same parts as the template rows, and a bin assignment can be extracted from them.

An item row (i, u) has no bin label: the reduction cannot use a coloring to assign it one. Bob therefore also holds a shared list $\xi = (\xi_1, \ldots, \xi_{s+1})$. For each index α, give the active items distinct positions $\pi_\alpha(i)$ in a fixed public order. If all Bob's indices agree on α and item i is active there, the entry is $R(u, \xi_{\pi_\alpha(i)})$; otherwise it is $R(u, \xi_{s+1})$. These rules use only the graph and the two inputs. In the coloring protocol, the bin and index identify an active item, and Bob can check agreement of his indices privately to select the appropriate base column.

The second family sets every bin's index to the same α and every list to the same γ, with $\xi = (\gamma_{s-1}, \ldots, \gamma_{2s-1})$. The template and item rows now read positions of one common list. A single protocol must handle both families, and before Bob speaks, Alice's messages depend only on her row. Consequently, the partition forced by the first family is also the partition tested by the second.

The second family has separate labels and a fixed positive share of the column weight. In the full-product construction, its columns already occur in the first family but may have very small total weight there. The separate weight prevents a small deletion from removing all collision tests. Definition 6.2 gives the full entry rules and weights.

A coloring gives a protocol. Fix the bin assignment supplied by a proper three-coloring. Alice names her bin p, using $\log s$ bits, and Bob sends its index α_p, using $\log q_2$ bits, where the number q_2

of indices is a power of two. Alice then sends one of

$$\underbrace{s-2}_{\text{template positions}} + \underbrace{1}_{\text{active item}} + \underbrace{1}_{\text{inactive item}} = s$$

possible messages. An active item is uniquely determined by its bin and the index, since a proper coloring puts at most one active item in each bin. Bob can therefore identify the selected base column from his input, and the players finish by running a protocol for R.

The two Alice messages each cost $\log s$ bits, giving total cost $C - 1 := d + 2\log s + \log q_2$. For an arbitrary graph, Alice can instead name her active item's position explicitly. This uses at most $2s - 1$ message values and costs one additional bit. Thus the table always has a cost-C protocol, and a coloring saves one bit (Proposition 6.4).

2.2 Why the reduction rules out every short protocol

We prove that the NO-case lower bound survives deletion of a small fraction of the row and column weight, so that it can be reused in later rounds. Suppose a protocol of cost $C-1$, with any pattern of interaction, computes the remaining matrix. From such a protocol we extract a single bin assignment by grouping Alice's rows according to the first $\log s$ bits, which are sent before Bob can reveal any index. For a NO graph, this fixed assignment must contain collisions.

A short protocol fixes a bin assignment. Group the template rows by their bin p. In the first column family, Bob supplies a separate index and list of base columns for every bin, and every combination occurs. A template row in bin p depends only on the data for that bin. On template rows the matrix is therefore an s-fold interlace whose blocks are the bins.

Within one bin, the template rows already require $d + \log q_2 + \log s$ bits, by two applications of the interlacing lower bound (Lemma 6.6). Of the $C - 1$ available bits, only $\log s$ remain to tell the bins apart. Suppose Bob sends a bit while u bins are still possible. Every Alice row possible before his bit remains possible. The interlacing lower bound for u bins still applies to the restriction keeping the larger half of his columns, and one bit fewer now remains, so the remaining bits do not suffice. Similarly, if one of Alice's bits sent more than half of the u bins to one side, that side would need more bits than remain. The separation theorem (Theorem A.3) therefore forces the first $\log s$ bits to be Alice's and to split the bins evenly. Grouping her rows by these $\log s$ bits gives s parts, one for each bin. Each part contains almost all the template-row weight of one bin and very little of the weight of the others.

We now discard the few items that lost more than half of the weight of their rows to the initial deletion, and assign each remaining item to a part containing at least a $1/(2s)$ fraction of the weight of its rows (i, u). Such a part exists by averaging. The rows of one item need not stay in one part, since different base rows u may follow different paths. However, the assignment is fixed before Bob has spoken, so it cannot depend on his edge. For a NO graph, Lemma 6.3 guarantees many indices at which one assigned bin contains two active items and an inactive item. Thus any proposed short protocol determines an assignment with such triples, which a proper coloring would avoid.

A collision gives an extra position. Restrict Bob to the second column family at a collision index α. All bins now use the same index and the same list of base columns. Within the collision's bin, the template rows and the three items read

$$\underbrace{s-2}_{\text{template positions}} + \underbrace{2}_{\text{distinct active positions}} + \underbrace{1}_{\text{inactive position}} = s+1$$

distinct positions. The inactive item is essential: two active items alone would give only $(s-2)+2 = s$ positions. Since s is a power of two, naming one of $s+1$ positions needs one more bit than naming one of s. To obtain the communication lower bound, we must show that interaction cannot bypass this obstruction.

Why collisions force more communication. The extra position explains the gap, but counting positions alone does not prove a lower bound against an interactive protocol. Bob's replies restrict which lists of base columns remain possible, and a lower bound for the original column set need not survive this restriction. We restrict to the second family, in which every bin has the same index α. After Alice's $\log s$ bits, Bob knows α and may send bits that narrow down the possible values of α before Alice speaks again, so that each branch of the protocol covers only some indices. If collisions occurred at only a few indices, Bob could isolate them on a branch with a larger budget. We therefore need collisions at a constant fraction of all indices.

We track the retained list inputs through Bob's messages and stop when Alice next speaks. Each usable index initially contributes one tracked set; a collision contributes an additional mark, recording its extra bit. An unmarked set follows a child retaining at least half its weight. A marked set either retains its mark on one child or splits into two unmarked sets. In the latter case its contribution changes from two to one plus one. Thus the total count is preserved.

A stopping node reached after r Bob bits can accommodate a count of at most $q_2 2^{-r}$ within its remaining communication budget. The stopping nodes are prefix-free, so Kraft's inequality bounds their total capacity by q_2. Initially, however, the count exceeds q_2: the collision indices outnumber the indices lost to the initial input deletions. This contradiction gives the NO lower bound (Lemma 6.8 and Theorem 6.9). Together with the coloring protocol, it yields the one-bit gap against every protocol, whatever its pattern of interaction.

2.3 Making the one-bit gap repeatable

A one-bit gap does not automatically yield a larger gap. Using the output as the next base matrix requires lower bounds on the restrictions that remain after messages, and a hard matrix can have easy restrictions. For example, restricting Bob's tuples in $R(u, y_i)$ to those with all coordinates equal removes the need to learn i.

We therefore maintain a stronger invariant in the NO case. Using the fixed positive rational input weights, for a threshold ρ fixed across amplification rounds, require

retained row weight	retained column weight	communication lower bound
$\geq \rho$	$\geq 3/5$	d
$\geq \rho$	$\geq 3/10$	$d-1$.

The threshold ρ may depend on the source size. We call these the *deletion bounds* (Definition 5.1). The second column threshold is half the first: after Bob sends one bit, the heavier branch retains at least half the column weight, and the allowed lower bound drops by one. The YES upper bounds do not require this invariant.

The graph construction supplies its extra bit only on restrictions retaining almost all the weight on both sides. The deletion bounds needed for another round allow most of the rows and a constant fraction of the columns to disappear. Two further interlaces close this gap.

Preserving the extra bit after input deletion. Transpose the source matrix to N, so its two column families become row families, and form $\mathsf{I}_w(N)$. Alice holds a copy index and a row of that

copy; Bob holds one column for each copy. Consider a restriction retaining almost all row weight but only a small specified fraction of the tuple weight. Suppose it has a protocol of cost $C + \log w - 1$, one bit less than naming a copy and running the cost-C protocol for N.

Pad shorter paths with fixed Alice bits, and stop each path just before Bob's first bit or just after Alice has sent $\log w$ bits, whichever comes first. The first-family rows force almost all their weight in most copies to reach distinct stopping nodes, one assigned to each such copy. These nodes are reached after $\log w$ Alice bits, leaving $C - 1$ bits. Bob has not spoken, so his tuple set is still the initial restriction. Projection estimates show that almost every copy still has almost all its columns available. If its assigned node also kept almost all its rows, the source lower bound would require C further bits. The node must therefore miss a substantial amount of its copy's rows.

Almost all first-family weight is already at the assigned nodes, and little weight was deleted initially. Summing over the assigned copies therefore forces substantial surviving second-family weight to lie elsewhere. Since only Alice has spoken, every surviving row reaches exactly one stopping node. It must reach either another copy's assigned node or a node with no assigned copy.

Neither group can receive enough weight. An assigned node already contains the hard first-family rows of its own copy; receiving too much second-family weight from other copies would force its cost above the remaining budget. The unassigned nodes have only a small total capacity. The combined receiving bound is smaller than the weight that must be displaced, giving a contradiction (Theorem 6.16). Transposing the resulting matrix exchanges the two retained-weight conditions, yielding W with its full lower bound when a small fraction of rows and almost all columns remain.

Recovering the bit in the deletion bounds. The matrix W also satisfies the desired deletion bounds for constant retained column weight, but one bit below its full cost. A final interlace combines this guarantee with the full-cost bound just proved. For full products, a tuple set of retained weight v cannot have small projections at too many coordinates. If its projection onto coordinate i has original weight p_i, containment in the product of its projections gives

$$v \le \prod_i p_i, \qquad \text{hence} \qquad \sum_i \log(1/p_i) \le \log(1/v).$$

Among sufficiently many usable copies, at least one therefore retains almost all its columns, allowing the full-cost bound for W to apply.

Finding one such copy alone would recover only the cost of W; we must also account for the bits needed to identify a copy. An induction over messages does this. At an Alice bit, sufficiently many row blocks remain on one branch. At a Bob bit, keeping the heavier tuple set increases its logarithmic loss by at most one. Choosing enough copies at the outset accommodates these losses. The deletion bounds control the intermediate restrictions, and the full-cost bound closes the remaining one-bit deficit (Theorem 6.17). For the explicit lists, the conditioning argument described next supplies the required projection control instead of the full-product inequality (Theorem 7.2).

Consequently, one complete round has a known overhead h and satisfies the following guarantee. In the YES case,

$$\mathrm{D}(R^+) \le \mathrm{D}(R) + h.$$

In the NO case, if $\mathrm{D}(R) = d \ge 3$ and R satisfies the deletion bounds at value d, then

$$\mathrm{D}(R^+) = d + h + 1,$$

and R^+ satisfies the same deletion bounds at value $d + h + 1$ (Theorem 5.3). This restoration is what makes iteration possible. Starting from a matrix of known cost d_0 satisfying the deletion bounds gives

$$\text{YES: } \mathrm{D}(R_k) \le d_0 + kh, \qquad \text{NO: } \mathrm{D}(R_k) = d_0 + kh + k.$$

The threshold $d_0 + kh$ is computable without deciding whether the graph is colorable.

2.4 Keeping the deterministic construction polynomial

Allowing every tuple of q columns from a set Y creates $|Y|^q$ inputs. We replace each product by an explicit list of polynomially many tuples with positive rational weights. For an absolute constant D, the list has exactly the same joint weights as the full product on every set of at most D coordinates. This is *limited independence*. We build and enumerate the entire list deterministically; the weights do not represent random choices made by the reduction.

The construction extends tuples one coordinate at a time and removes tuples by adjusting their weights. Associate with each tuple a zero–one vector recording the assignments it realizes on every set of at most D coordinates, together with a coordinate recording total weight. The required joint weights specify the weighted sum of these vectors. If the list is longer than the vector dimension, a linear dependence lets us change the weights without changing that sum. Since total weight is fixed, the change has both positive and negative components; we can continue until one tuple has weight zero and remove it, keeping all other weights nonnegative. Compressing after each new coordinate keeps the intermediate lists short, avoiding enumeration of an exponentially large product. Lemma 7.1 proves the support and rational bit-length bounds. The original entry rules still apply to every listed tuple, so the upper-bound protocols are preserved.

The lower bound is subtler: conditioning on earlier messages can destroy independence. For example, retaining only tuples whose first two coordinates agree makes those coordinates dependent. Our conditioning lemma shows that, after retaining any event of weight at least v, there are only $\mathrm{poly}(1/v)$ exceptional coordinates. On every set of at most eight other coordinates, the conditional distribution is close in total variation to the original product distribution (Lemma 6.10). A moment bound proves this: too many disjoint sets of coordinates with substantially changed distributions would contradict the weight of the conditioning event.

We choose the interlacing widths and deletion thresholds so that enough coordinates remain outside the exceptional set. Projecting onto these coordinates gives actual inputs of the retained matrix, with enough weight to apply the deletion bounds. Thus the proof accounts for the dependencies created by messages.

If the base input set has m labels, each list has at most $(1+qm)^D$ tuples, and its rational weights have polynomial bit length (Lemma 7.1). Each round therefore takes time polynomial in the source graph size n and the encoding length of the current matrix, with an exponent independent of the round. After k rounds, size and running time are at most $n^{O(A^k)}$ for an absolute constant $A > 1$. Every fixed gap therefore gives a deterministic polynomial-time reduction (Theorems 1.1 and 8.4).

A suitable choice of $k = \Theta(\log \log N)$ for sparse formulas makes the construction quasipolynomial in N, while $\log \log T = O(k)$. An estimator with a sufficiently small constant times $\log \log T$ error would therefore decide sparse 3-SAT in quasipolynomial time (Corollary 1.5).

2.5 Two further consequences of keeping fewer inputs

Sampling at the end gives a stronger randomized bound. For the randomized result, we keep full products of tuples throughout the amplification. The table is then too large to print, but we can compute single entries from short descriptions of the inputs. After all k rounds, we use the input weights as sampling probabilities: we draw m rows and m columns independently and print the m^2 entries at their intersections. Here the reduction uses randomness; the communication protocols being measured remain deterministic.

In the NO case, a protocol may be chosen after the sample is drawn, so the sample must defeat every protocol of depth one less than the NO cost simultaneously. We call a set of columns *compatible* with a protocol and a set of rows if the protocol is correct on them. Over all protocols of depth t and all row sets, a matrix of real rank r has at most $2^{O(r \log(r+2)\, 2^t)}$ inclusion-maximal compatible column sets (Theorem 9.3). The count depends on rank and protocol depth, not on the original number of input labels. For row sets of weight $\geq 3/5$, the deletion bounds give each compatible column set weight $< 3/5$. A union bound over these sets then shows that, with high probability, $m = \Theta(r \log r \cdot 2^t)$ random columns are not all contained in any one of them. Once such a column sample is fixed, every row set compatible with a short protocol on the sampled columns has weight $< 3/5$, so the same union bound applies to the sampled rows (Theorem 9.4). The sample need only preserve the gap, since no further amplification follows. Each interlace multiplies the rank by at most its width, so the rank after k rounds is at most 2^{d_0+kh} (Proposition 5.5). For a sparse formula on N variables the graph has $N^{1+o(1)}$ vertices and $h = (1 + o(1)) \log N$, so taking $k \approx N^{\alpha}/\log N$ gives $\log T = (4 + o(1))k \log N$ and $k = (\alpha/4 + o(1)) \log T / \log \log T$. The construction then takes time $2^{O(N^{\alpha})}$. An estimate within additive error $c \log T / \log \log T$ distinguishes the two cases whenever $\alpha > 8c$, giving Corollary 1.4.

Strict alternation needs a different encoding. The coloring protocol above uses consecutive bits from the same player. Inserting dummy turns can cost more than the gap, so it does not give hardness under strict alternation.

We instead start from the hypergraph-coloring reduction of Guruswami and Lee [8]. For a power of two s, each edge has s^2 vertices. In the YES case there is an s-coloring in which every edge contains at least $s - 1$ vertices of every color. In the NO case, every coloring with at most 2^s colors has a monochromatic edge (Proposition 10.2).

Alice holds a vertex and a tuple of edge addresses. Bob holds an edge, a coordinate selecting one address, and a Boolean array. The selected address and the vertex's position in Bob's edge identify the array bit to output; the output is zero if the vertex is outside the edge (Definition 10.3). Write $s = 2^t$, and let b bits name an edge. In the YES case, the players exchange the color, address and edge information in alternating turns. The coloring leaves at most $2s - 1$ candidate vertices of Alice's color in Bob's edge, giving total cost $2b + 4t + 2$ (Proposition 10.4).

For the NO lower bound, record Alice's responses to *every* possible sequence of Bob's earlier bits during her first t turns. There are at most $2^{2^t - 1} \leq 2^s$ such response rules. Assign each vertex a rule used by many of its address tuples. The NO promise gives a monochromatic edge, so many rows sharing one early response rule must still identify many different array positions. Counting only the transcript of one interaction would not suffice, because Bob's messages may depend on his entire array.

We construct a polynomial-size explicit list of arrays in which no prescribed pattern on many positions occurs too often. Following a hypothetical short protocol while retaining large sets of rows and arrays leaves more possible arrays than its remaining response patterns can cover (Corollary 10.6, Lemma 10.5, and Proposition 10.7). This gives a NO lower bound of $2b + 6t - 6$, and hence a gap of $2t - 8$. Allowing either starter and receiver-only output loses at most two bits (Lemma 10.1). Choosing t as a sufficiently large constant therefore gives a deterministic polynomial-time reduction for every fixed gap in Yao's convention.

3 Related work

Which optimization problem is hard? For explicitly given total tables, Kushilevitz and Weinreb proved that approximating communication complexity is hard under cryptographic assumptions [16]. Ilango, Loff and Oliveira proved NP-hardness and strong inapproximability for *partial* matrices, where the protocol is required to be correct only on specified input pairs [12]. Our tables are *total*: every pair of a row and a column has a specified bit, and the protocol must be correct on all of them.

Restrictions on the conversation also matter. For total matrices, Hirahara, Ilango and Loff proved additive hardness for the number of bits in a three-message protocol, and hardness of approximating the number of leaves of a protocol tree for every fixed number $k \geq 3$ of messages [9]. A message is a block of consecutive bits from one player, and a leaf is a possible end of the conversation. Neither restricting the number of messages nor counting leaves gives the same optimization problem as minimizing bits under strict alternation or unrestricted speaking order. The exact-hardness results for the unrestricted model [6, 10] left open both hardness with one bit of additive error and Yao's original alternating model, as Hirahara, Ilango and Loff noted [10]. Our fixed-gap theorems resolve both questions.

Ingredients of the constructions. For the unrestricted reductions, we obtain hard graph-coloring instances by combining reductions of Dinur and of Bogdanov, Obata and Trevisan [3, 5]. In these graphs, each vertex meets only a constant number of edges. The alternating reduction uses the hypergraph-coloring promise of Guruswami and Lee [8] described in Section 2.

Both reductions also replace large input sets by small explicit ones that keep some of their properties. For the deterministic interlacing construction, we follow Koller and Megiddo [15, Section 3]: we extend a list of tuples one coordinate at a time and delete tuples without changing the joint weights of the values on every set of a bounded number of coordinates. For strict alternation, the array list is based on the small-bias construction of Alon et al. [1].

Bounds on the number of column sets on which short protocols can be correct, expressed in terms of matrix rank, appear in graph algorithms [4, 20] and in work on hard submatrices [11]. To count such sets, we use the row-sum method of Oum, Sæther and Vatshelle [20]: summing Boolean rows identifies the columns in which all those rows are zero. Mackenzie's hardness condensation [17] also counts the input sets on which short protocols succeed, to shrink a communication problem while keeping its cost high relative to the new input length. In contrast, our sampling theorem (Theorem 9.4) keeps the number of bits above a prescribed bound, and we prove it directly.

Reading guide. Section 4 fixes the communication conventions and the use of input weights. We then state the amplification theorem and its iteration, prove the graph encoding and the two further interlaces, and construct the short input lists used by the deterministic reduction. We then give the final-sampling argument and, separately, the strictly alternating construction. The appendices contain the remaining interlacing estimates, list algorithms, parameter checks and coloring reductions.

4 Preliminaries

4.1 Communication and estimation

An estimator receives the whole table, whereas each player knows only its own input.

Definition 4.1. A table $M : X \times Y \to \{0,1\}$ is total, and its input sets X and Y are finite and nonempty sets of labels. Equal rows or columns may have different indices. Its size is $T = |X||Y|$. A deterministic public-output protocol is a binary tree whose internal nodes specify a speaker and a bit-valued function of that player's input and the preceding transcript. Each leaf has a fixed output, and the protocol must compute every entry correctly. Its cost is its maximum depth, charging every bit, including fixed bits. Local computation is free. Write $\mathrm{D}(M)$ for the least cost with unrestricted speaking order.

An additive-$a(T)$ estimator returns $\widehat{D}(M)$ satisfying $|\widehat{D}(M) - \mathrm{D}(M)| \le a(T)$. Its output must admit polynomial-time comparison with polynomial-length binary rationals. A bounded-error randomized estimator satisfies this guarantee with probability at least $2/3$ on every input. Polynomial time means worst-case time polynomial in T. Estimators for the alternating costs below follow the same conventions.

In the alternating model, the starting player and the output convention can each affect the cost.

Definition 4.2 (Alternating costs)**.** In strict alternation the players speak one bit at a time, in turn. Let D_A^{alt} and D_B^{alt} be the minimum public-output costs when Alice or Bob starts, respectively, and let $D_{\mathrm{free}}^{\mathrm{alt}} = \min\{D_A^{\mathrm{alt}}, D_B^{\mathrm{alt}}\}$. The starter is chosen independently of the inputs. Every bit is charged, and stopping is public: the transcript determines whether communication has ended.

Yao's receiver-output cost Y_{free} permits the same choice of starter, but the receiver of the last bit determines the answer from that player's input and the transcript. At depth zero, the protocol designates one player whose input determines the answer. Stopping is still public and correctness is required on every input.

Our quantitative statements use sparse 3-SAT: the input has N variables and at most $c_s N$ clauses, where c_s is fixed in advance. Running times are worst-case bounds, including all random tapes for randomized algorithms; bounded error means success probability at least $2/3$ on every formula. Constants may depend on c_s and on the estimator. We do not apply a sparsification step with an uncharged exponential cost: all stated running times refer directly to these sparse inputs.

All communication protocols in this paper are deterministic and exact. Only estimators and randomized reductions use random bits. All logarithms are binary unless written ln. Write $[q] = \{1, \dots, q\}$ and $\mathrm{pow}_2(a) = 2^{\lceil \log a \rceil}$ for the least power of two at least $a \ge 1$.

Every $n \times n$ table has $\mathrm{D}(M) \le \lceil \log n \rceil + 1$: Alice sends her row index and Bob sends the entry.

4.2 Relabeling and input weights

We keep the matrices small in two ways, by explicit short lists and by sampling. In both, each new input decodes to an original input, as follows.

If $f : X' \to X$, $g : Y' \to Y$ and $F'(x', y') = F(f(x'), g(y'))$, each player can compute $f(x')$ or $g(y')$ from its own input and run a protocol for F. Hence F' has at most the communication cost of F in every model above, and at most its rank. Both are equal when f and g are onto. Transposition preserves rank and unrestricted cost, and exchanges D_A^{alt} and D_B^{alt}.

In particular, duplicating rows and columns makes a table square with power-of-two side length without changing its cost. We may also require the side length to be at least a prescribed bound b

polynomial in the input size. The side length is then less than $2\max\{b, |X|, |Y|\}$, so the enlarged table can be printed in polynomial time.

For the unrestricted construction, we assign the labels in each input set positive rational weights summing to one. For row weights μ_X and column weights μ_Y, a set's weight is the sum of its labels' weights, for example $\mu_X(A) = \sum_{x \in A} \mu_X(x)$. Uniform weights give ordinary fractions of labels. Unequal weights fix the shares assigned to different input families and preserve the required counts when we shorten tuple lists. The weights are auxiliary data for the construction and its lower-bound proof; communication cost depends only on the table, and every input must be computed correctly. The final deterministic reduction discards the weights. Distinct labels keep separate weights, even when their rows or columns agree.

If X' has weights μ', a decoder $f : X' \to X$ induces weights on X by adding the weights of labels with the same image, and labels outside the image get weight zero. A set $A \subseteq X$ therefore has induced weight $\mu'(f^{-1}(A))$. In particular, a retained set $A' \subseteq X'$ has image of induced weight at least $\mu'(A')$.

Under product weights, a tuple has the product of the weights of its coordinates. Some counting arguments are shorter in probability notation: we regard a label as drawn according to its weights and use the standard terms distribution, marginal, and independence. This is an auxiliary calculation, not a random choice by a communication protocol. An event is a set of labels. Conditioning on a retained set A of positive weight means renormalizing: $(\mu \mid A)(B) = \mu(A \cap B)/\mu(A)$. For each projection below, we specify whether we measure its size with the original weights or with these conditional weights.

Counting surviving blocks. Later proofs need many blocks of a partition to retain enough weight. We derive this from the total retained weight by the following count. Suppose we partition an input set into q blocks, each of weight $1/q$, such as the rows $\{i\} \times X$ of $\mathsf{I}_q(R)$ in Definition 4.3 below. If a restriction retains total weight β, the fractions retained within the blocks sum to $q\beta$. We set aside at most e blocks and suppose that at most r others retain fraction $\geq t$. Bounding these $r + e$ fractions by one and every remaining fraction by t gives

$$q\beta \leq r + e + qt.$$

Thus, if $q\beta > r + e + qt$, more than r blocks not set aside retain fraction $\geq t$. If the blocks instead partition only a part of the input set, of total weight λ, that part retains fraction at least $(\beta - (1 - \lambda))/\lambda$ of its weight, since the rest of the input set contributes at most $1 - \lambda$ to β.

4.3 Interlacing and restrictions

In an interlace, a row is a row of the base matrix together with an index i, a column is a tuple of base columns, and the entry is read from the ith coordinate of the tuple.

Definition 4.3 (Interlacing)**.** For a Boolean matrix $R : X \times Y \to \{0, 1\}$, the full q-fold interlace $\mathsf{I}_q(R)$ has rows $(i, u) \in [q] \times X$, columns $(y_1, \ldots, y_q) \in Y^q$, and entry

$$\mathsf{I}_q(R)((i, u), (y_1, \ldots, y_q)) = R(u, y_i).$$

When R has normalized input weights μ_X, μ_Y, give row (i, u) weight $\mu_X(u)/q$ and column $(y_1, \ldots, y_q)$ weight $\prod_{j=1}^{q} \mu_Y(y_j)$.

Why column deletion matters. Rows of the interlace with different indices i read different coordinates of Bob's tuple, but this distinction may disappear when columns are deleted. For example, consider the 2×2 base matrix

$$R = \begin{array}{c|cc} & 1 & 2 \\ \hline a & 0 & 1 \\ b & 1 & 0 \end{array}.$$

The left-hand table in Figure 1 is its two-fold interlace. We obtain the right-hand table by keeping only columns $(1,1)$ and $(2,2)$. For each fixed $u \in \{a, b\}$, the rows $(1, u)$ and $(2, u)$ then coincide.

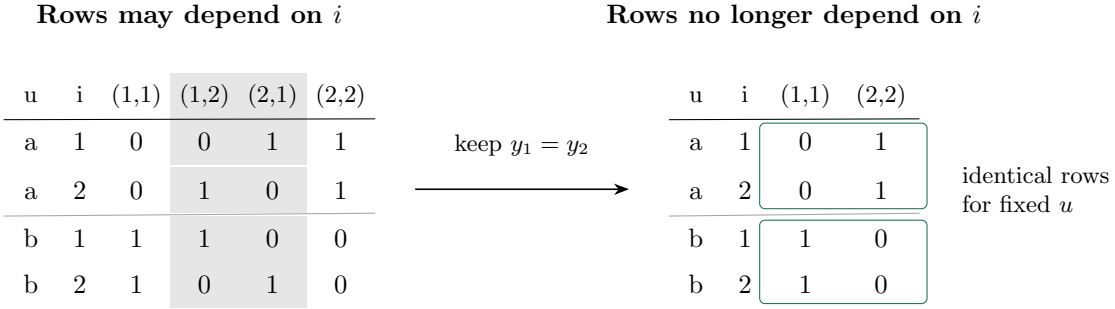


Figure 1: Deleting columns with $y_1 \neq y_2$ removes the dependence on the row index i. For each fixed u, the rows $(1, u)$ and $(2, u)$ agree on every retained column.

The same happens for every base matrix R. On the columns $(y, \ldots, y)$, the entry of $\mathsf{I}_q(R)$ is $R(u, y)$ for every row index i. Hence a lower bound for the full interlace need not survive arbitrary column deletion. For this reason, Definition 5.1 requires lower bounds only for restrictions that retain a prescribed weight of rows and of columns.

Keeping fewer column tuples. Full interlacing creates $|Y|^q$ columns, too many for an explicit polynomial-time reduction. We therefore replace Y^q by a short list with rational weights. For every assignment of values to at most D coordinates, we preserve the total weight of tuples matching that assignment. The definition below calls these exact counting identities D-wise independence. Lemma 7.1 constructs such lists deterministically (Sections 7 and 8).

Definition 4.4 (Weighted lists of tuples)**.** Fix integers $q, D \geq 1$. For a finite alphabet Y with prescribed normalized rational weights μ, an indexed tuple list consists of a finite nonempty label set $\mathcal{L}$, positive normalized rational weights ν on its labels, and a decoder $z : \mathcal{L} \to Y^q$. It is *exactly D-wise independent* if the weights induced on any set $J \subseteq [q]$ of at most D decoded coordinates equal the product weights μ^J. These induced coordinate weights are its marginals. No independence on larger sets is required. A set of labels may contain one label but not another with the same decoded tuple.

For a base matrix $R : X \times Y \to \{0, 1\}$ with column weights μ, the interlace on this list has a column for each label ℓ, with entry $R(u, z_i(\ell))$ at row (i, u). Rows keep the weights of Definition 4.3, and columns get the weights ν. For both kinds of interlace, *block* i is the row set $\{i\} \times X$, and we measure row weight within a block by the base row weights, without division by q.

The full product is the list with labels Y^q, identity decoder and product weights. For a label set K and a set $J \subseteq [q]$, write $\pi_J K = \{z(\ell)_J : \ell \in K\}$ for the projection of its decoded tuples.

To apply a lower bound to a few surviving blocks, we project the retained column tuples onto those blocks. The next observation realizes the projected tuples as actual columns of the restriction.

Observation 4.5 (Restrictions projected to a set of blocks)**.** *Let G be a restriction of an interlace of R, on the full product or on a tuple list, with retained label set K, and let J be a set of blocks. For each $i \in J$, the rows in block i read only coordinate i of the decoded tuple. Choose one label of K for each tuple in $\pi_J K$. The restriction of G to these labels and to the retained rows of the blocks in J is then the full-product interlace on J, restricted to the columns $\pi_J K$ and to the same rows. Every lower bound for this matrix therefore holds for G. Its column weight is $\mu^J(\pi_J K)$, and each block keeps its base row weights. If the marginal on J is μ^J, as for full products and for exactly $|J|$-wise independent lists, then $\mu^J(\pi_J K) \geq \nu(K)$, since $K \subseteq z_J^{-1}(\pi_J K)$. We never use the weights of the chosen representatives, and we assert no independence of the retained labels.*

5 A reusable gap construction

In this section we state the one-round amplifier and then iterate it. In the NO case the amplifier raises the cost by one bit more than in the YES case, and its output again satisfies the lower bounds under input deletion that the next round needs.

5.1 Lower bounds that survive restrictions

The deletion bounds use two column-weight thresholds, the second half the first. The reason is that after Bob sends one bit, the heavier branch retains at least half the column weight. At the lower threshold we allow the bound to drop by one bit.

Definition 5.1 (Deletion bounds)**.** Let $F : X \times Y \to \{0,1\}$ have positive normalized rational input weights. For $0 < \rho \leq 1$ and an integer d, F satisfies the *deletion bounds* $\mathcal{P}(F; \rho, d)$ if, for every $j \in \{0,1\}$ and all $U \subseteq X$, $V \subseteq Y$,

$$\mu_X(U) \geq \rho \text{ and } \mu_Y(V) \geq (3/5)2^{-j} \quad \Longrightarrow \quad \mathrm{D}(F[U,V]) \geq d - j.$$

Here $F[U,V]$ is the restriction of F to $U \times V$. No matching upper bound is required.

Whether the source graph is three-colorable determines on which side of the gap the constructed matrix lies. For an unsatisfiable formula, we need the graph to stay non-three-colorable after a small fraction of its vertices is deleted, since otherwise deleting inputs could erase the lower bound. The reductions of Dinur and of Bogdanov, Obata and Trevisan provide such graphs [3, 5].

Theorem 5.2 (Bounded-degree coloring source)**.** *There are constants $\Delta \geq 1$ and rational $\gamma > 0$, and a deterministic polynomial-time reduction from 3-SAT to nonempty loopless multigraphs $G = (V, E)$ with no isolated vertices and maximum degree at most Δ, such that:*

1. *a satisfiable formula produces a three-colorable graph;*
2. *an unsatisfiable formula produces a graph for which every three-coloring has at least $\gamma|E|$ monochromatic edge occurrences.*

For sparse formulas on $N \geq 1$ variables, the reduction can use a fixed positive integer p such that, for $n = |V|$,

$$N \leq |E|, \qquad n, |E| \leq N(\log(N+2))^p, \qquad n/2 \leq |E| \leq \Delta n/2.$$

The proof in Section G.1 composes these reductions and checks the stated graph-size bounds and conventions.

We call graphs from satisfiable formulas YES and those from unsatisfiable formulas NO. We build the amplifier in two ways, which satisfy the same guarantee. In the *full-product case*, used for the randomized reduction, every interlace uses all column tuples, and Lemma F.2 fixes the parameters. In the *explicit case*, used for the deterministic reduction, the interlaces use the small lists of Definition 8.3, and Lemma F.3 fixes the parameters. In each case the last step restores the deletion bounds at full cost for the next round, by Theorem 6.17 in the full-product case and by Theorem 7.2 in the explicit case.

Theorem 5.3 (Amplification under input deletion)**.** *Let $G = (V, E)$ be a YES or NO graph produced by Theorem 5.2, and put $n = |V|$. In either case there are a public row-weight threshold ρ, a public integer h, and a deterministic, implicitly represented construction $R \mapsto R^+$ on base matrices R with positive normalized rational input weights. In the full-product case, n must exceed an effectively computable constant, and $\rho = 2^{-L}$ with $L = \lceil \log n \rceil$, and $h = L + O_{\Delta,\gamma}(\log L)$. In the explicit case, $\rho = 1/\mathsf{N}$ and $h = 10 \log \mathsf{N}$, with $\mathsf{N} = s^4$ as in Lemma F.3. In both cases:*

(i) On either source promise,

$$\mathrm{D}(R^+) \le \mathrm{D}(R) + h + 1,$$

and, if G is YES, the stronger bound $\mathrm{D}(R^+) \le \mathrm{D}(R) + h$ holds.

(ii) If G is NO and the base matrix R satisfies $\mathrm{D}(R) = d \ge 3$ and $\mathcal{P}(R; \rho, d)$, then

$$\mathrm{D}(R^+) = d + h + 1, \qquad \mathcal{P}(R^+; \rho, d + h + 1).$$

(iii) On either source promise, $\mathrm{rank}(R^+) \le 2^h \mathrm{rank}(R)$.

The proof is assembled at the end of Section 6 (p. 31). It uses the interlacing estimates of Section A and, in the explicit case, the small lists of Section 7. The rank bound follows from Lemma 6.18.

5.2 Iteration

One round adds h bits to the YES upper bound and $h + 1$ bits to the NO lower bound. If we start from the initial matrix below, of known cost d_0, the two bounds therefore differ by i bits after i rounds.

Lemma 5.4 (Initial matrix)**.** *For $L \ge 1$, let $p_0 = \mathrm{pow}_2(64(L+1))$ and $R_0(z, j) = z_j$ on $\{0,1\}^{p_0} \times [p_0]$, with uniform input weights. Then*

$$d_0 := \log p_0 + 1, \qquad \mathrm{D}(R_0) = d_0, \qquad \mathrm{rank}(R_0) = p_0 = 2^{d_0 - 1}, \qquad \mathcal{P}(R_0; 2^{-L}, d_0).$$

For $L = \lceil \log n \rceil$ and $n \ge 4$, the initial matrix has polynomial size in n and can be printed in polynomial time.

The proof is in Section A.4 (p. 54).

Proposition 5.5 (Iterated amplification)**.** *Fix one case of Theorem 5.3, whose threshold is ρ and whose overhead is h, and a source graph to which it applies. Iterate the theorem from the matrix R_0 of Lemma 5.4 with $L = \log(1/\rho)$, using the same parameters in every round, and let R_i be the matrix after $i \ge 0$ rounds. Put $B_i = d_0 + ih$, $H_i = B_i + i$, and $r_i = 2^{B_i - 1}$. A YES graph gives $\mathrm{D}(R_i) \le B_i$. A NO graph gives $\mathrm{D}(R_i) = H_i$ and $\mathcal{P}(R_i; \rho, H_i)$. On either promise, $\mathrm{rank}(R_i) \le r_i$ and $\mathrm{D}(R_i) \le H_i$.*

Proof. Lemma 5.4 gives the case $i = 0$, with $d_0 \geq 3$. By Theorem 5.3, each round raises the YES bound by h, the NO cost and the general upper bound by $h + 1$, and rank by a factor at most 2^h. In the NO case, (ii) restores the deletion bounds at threshold ρ, so the induction continues. □

6 Proof of amplification under input deletion

We now prove Theorem 5.3. The graph gives a gap of one bit. Two further interlaces then make this gap survive input deletion and restore the deletion bounds.

6.1 One choice of widths and tolerances

We need enough coordinates to encode the graph, and enough interlaced copies that the lower bounds still hold after a protocol discards some inputs. We use one choice of parameters for both cases. Its two scales are ρ, the retained fraction of base rows for which hardness must survive between rounds, and s, the number of bins used to group inputs within a round.

Definition 6.1 (Amplification parameters)**.** Let n be the source graph size. Choose powers of two s, ρ^{-1} with $n \leq \rho^{-1}$, $\rho \leq s^{-4}$, and $s \geq s_0$, the effective constant of Lemma F.1. Set

$$q_1 = s - 2, \qquad q_2 = s^2/\rho, \qquad w = s^8, \qquad Q = s^{24},$$
$$x = \rho^8, \qquad \eta = s^{-2}, \qquad \theta = (512Q)^{-1}, \qquad \delta_U = \eta/(16w).$$

The source construction uses q_1, q_2 coordinates, and the two further interlaces have widths w, Q. The deletion bounds of Lemma 6.6 for the matrices of the source construction use row-weight threshold x, and the lower bound for the source matrix N survives deleting an η fraction of each input set's weight (Theorem 6.9). For the transpose W of the first further interlace (Definition 6.2), θ is the minimum retained row weight and δ_U is the allowed fraction of deleted column weight.

In the full-product case, we take $L = \lceil \log n \rceil$, $\rho = 2^{-L}$, and $s = \mathrm{pow}_2(2^{16}(L + 1))$. Lemma F.2 shows that these choices meet the constraints above once n exceeds an effective constant. In the explicit case, we take $\rho = s^{-4}$ for a sufficiently large dyadic s with $s^4 \geq n$. Section F verifies the numerical requirements shared by both cases, namely the bounds (10), and the additional conditions of each case. All these choices are fixed across rounds.

6.2 Encoding the graph

We construct the source matrix of Section 2 from interlaced copies of the base matrix R and rows encoding the graph. In a NO instance, every assignment of items to bins has a *collision*: a coordinate at which one bin contains two active items and an inactive item. We show in the rest of the proof that a collision forces the extra bit of the NO case. Because of the template rows, defined below, every protocol cheaper than the NO-case cost must begin with $\log s$ bits from Alice, and these bits distinguish the s bins (Theorem A.3). The item rows encode the graph.

Definition 6.2 (Source construction)**.** Let $R : X \times Y \to \{0, 1\}$ have positive normalized rational input weights, and let $s \geq 4$ be dyadic, meaning a power of two. Coordinates ranging over integer sets such as $[q_2]$ are uniform below, and product sets carry the product of the specified factor weights.

Active items. We repeat the graph in several groups so that the proof can discard groups that lose too many items. An item is a vertex or auxiliary label in one of $4s$ groups: $\mathcal{I} = [4s] \times (V \sqcup [s - 3])$.

In each group, each edge occurrence has a coordinate activating its two endpoints and all $s-3$ auxiliary items. All other items are inactive. Pad these $4s|E|$ coordinates with inactive coordinates to length $q_0 = \mathrm{pow}_2(4s|E|)$, then repeat each coordinate q_2/q_0 times, where q_2 is a dyadic multiple of q_0. Write $v_i(\alpha)$ for the bit marking item i at index $\alpha \in [q_2]$, and π_α for the injection of the at most $s-1$ active items into $[s]$ that preserves a fixed order on $\mathcal{I}$. These bits do not depend on any assignment of items to bins.

Template interlaces. Put $q_1 = s-2$, leaving two message values for active and inactive items. Define

$$\Gamma = Y^{2s-1}, \qquad J = [q_1] \times X, \qquad S = J^{q_2}, \qquad J_2 = [q_2] \times \Gamma.$$

The template rows use the first q_1 positions of a list $\gamma \in \Gamma$, and we reserve its last $s+1$ positions for the item rows. The two successive interlace matrices are

$$A(\gamma, (r,u)) = R(u, \gamma_r), \qquad R_{\mathrm{temp}}(z, (\alpha, \gamma)) = A(\gamma, z_\alpha).$$

Thus A has rows indexed by Γ and columns by J, and R_{temp} has rows indexed by S and columns by J_2. Moreover, A^{T} is an interlace of R with repeated column labels, and $R_{\mathrm{temp}}^{\mathsf{T}}$ is an interlace of A. A template label $z \in S$ is a tuple of pairs $z_\alpha = (r,u)$, one for each index. The pair specifies position r and a base row u, giving the matrix entry $R(u, \gamma_r)$.

Matrix entries. The matrix B_{raw} has template rows (p,z), indexed by a bin $p \in [s]$ and a template label, and item rows (i,u), indexed by an item and a base row:

$$\mathcal{K} = ([s] \times S) \sqcup (\mathcal{I} \times X).$$

We give the template rows most of the weight, so that deleting an η fraction of the row weight leaves most of their weight intact. Set $\lambda = 255/256$ and $\mu = 1/256$. Template rows have total weight λ, and vertex rows and auxiliary rows each have total weight $\mu/2 = 1/512$. Within each of these three types, bin or item labels are uniform and the other labels follow their specified weights. For a column $Z = (\xi, j_1, \ldots, j_s)$, where $\xi \in Y^{s+1}$ and $j_p = (\alpha_p, \gamma_p) \in J_2$, set

$$B_{\mathrm{raw}}((p,z), Z) = R_{\mathrm{temp}}(z, j_p),$$

$$B_{\mathrm{raw}}((i,u), Z) = \begin{cases} R(u, \xi_{\pi_\alpha(i)}), & \alpha_1 = \cdots = \alpha_s = \alpha,\ v_i(\alpha) = 1, \\ R(u, \xi_{s+1}), & \text{otherwise.} \end{cases}$$

Thus an item row uses its assigned position when the indices $\alpha_1, \ldots, \alpha_s$ agree and the item is active there, and otherwise it uses the last entry of the shared list.

Two column families. The first column family $\Omega = Y^{s+1} \times J_2^s$ has product weights. The second column family is

$$\mathcal{D} = \{(\mathrm{tail}(\gamma), j, \ldots, j) : j = (\alpha, \gamma) \in J_2\}, \qquad \mathrm{tail}(\gamma) = (\gamma_{q_1+1}, \ldots, \gamma_{2s-1}),$$

with the weights induced from J_2. We give the two families the weights

Family	Weight	Role
Ω	λ	Force Alice's initial messages
$\mathcal{D}$	μ	Read the three positions of a collision.

Thus the first column family has weight λ, like the template rows. Keep the two family tags distinct even when their raw columns coincide. At every bin p, the label j_p has the weights prescribed on J_2

in either family. We call the columns of the two families Ω-*inputs* and $\mathcal{D}$-*inputs*. After transposition they become rows, and we then also call them Ω-*rows* and $\mathcal{D}$-*rows*.

The two interlaces. Transpose the matrix and restrict its rows to the first column family:

$$N = B_{\text{raw}}^{\mathsf{T}}, \qquad N_{\text{ref}} = N[\Omega, \mathcal{K}],$$

where the weights on Ω are normalized. The matrix N_{ref} is used only in the lower-bound analysis. Finally set

$$U = \mathsf{I}_w(N), \qquad W = U^{\mathsf{T}}, \qquad R^+ = (\mathsf{I}_Q(W))^{\mathsf{T}}.$$

As in Section 2, the Ω-inputs force Alice's first $\log s$ bits to name a bin, and this fixes an assignment of the items to bins. Against a $\mathcal{D}$-input with label (α, γ), the rows of two items active at α and of one inactive item read three distinct positions of the same list γ. A collision puts all three items in one bin. The interlace U extends the lower bound for N to restrictions that keep only a small fraction of one input set. This costs one bit in the deletion bounds (Theorem 6.16), and the final interlace restores that bit.

Throughout, we write $d = \mathrm{D}(R)$. With the parameters of Definition 6.1, we define the costs and the threshold used in the lower-bound analysis (in the explicit case, we drop the subscript g that Section 8 attaches to its parameters):

$$H_1 := d + \log s, \qquad H_2 := H_1 + \log q_2, \qquad C := H_2 + \log s + 1, \qquad x_{\text{cap}} := 2s\rho. \tag{1}$$

We will show that A, R_{temp} and N have NO-case lower bounds H_1, H_2 and C. The collision bound of Lemma 6.6 requires each position involved to retain base row weight at least x_{cap} within its block.

The following $s = 4$ example compares two bin assignments with the same active items. The endpoints u, v and auxiliary item a are active, while item h, from another group, is inactive. At the depicted coordinate α, let $\pi_\alpha(u) = 1$ and $\pi_\alpha(v) = 2$. The rows of u, v use γ_3, γ_4 on columns in the second family, while h uses the last entry $\xi_{s+1} = \gamma_7$.

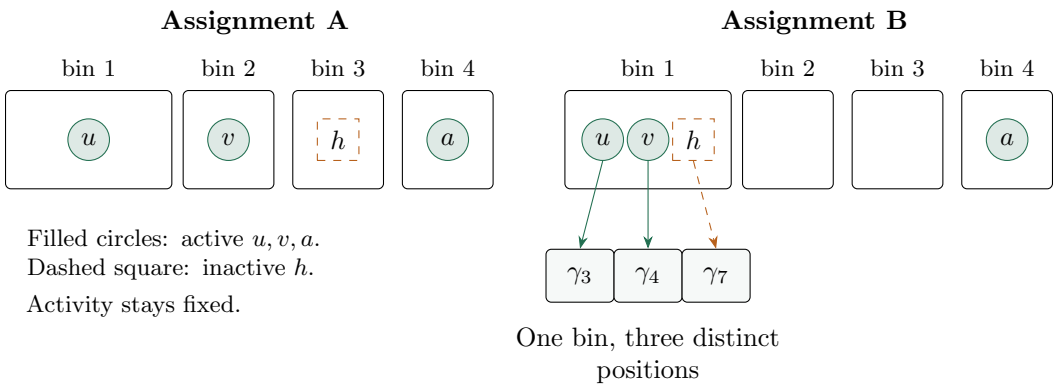


Figure 2: The same active and inactive items can have different bin assignments. In assignment B, one bin contains the active endpoints u, v and the inactive item h. Against a $\mathcal{D}$-input, their rows use three distinct positions of the same list. These are two illustrative assignments, not steps of a protocol.

The next lemma shows that, for a NO graph, collisions as in Figure 2 survive deletion. After we delete a few items, every assignment still has collisions at a constant fraction ζ of the coordinates.

Lemma 6.3. *A YES graph admits an assignment of $\mathcal{I}$ to $[s]$ with at most one active item in each bin at every coordinate. For a NO graph, delete fewer than $(s/2)\delta_v n$ vertex items and fewer than $s/2$ auxiliary items, where*

$$\delta_v = \frac{\gamma|E|}{2\Delta n}, \qquad \zeta = \frac{\gamma}{24}.$$

Every assignment of the remaining items to the s bins then has at least ζq_2 coordinates at which one bin contains two active items and an inactive item.

We prove the collision guarantee in Section 6.3. First, the YES assignment gives a short protocol.

Proposition 6.4. *If $\mathrm{D}(R) \le y$, then $\mathrm{D}(N) \le y + 2\log s + \log q_2 + 1$. For a YES graph, the final $+1$ can be omitted. Both bounds remain valid after restricting either input set.*

Proof. For a YES graph, fix a public bin assignment χ from Lemma 6.3. We describe a protocol for B_{raw} and exchange the players at the end. Alice holds a template row (p, z) or an item row (i, u), and Bob holds $Z = (\xi, j_1, \ldots, j_s)$ with $j_p = (\alpha_p, \gamma_p)$. They exchange:

Alice	bin p (use $p = \chi(i)$ for an item row)	$\log s$ bits,
Bob	α_p	$\log q_2$ bits,
Alice	one of q_1 template positions, active, inactive	$\log s$ bits.

The third message has $q_1 + 2 = s$ possibilities. For a template row, write $z_{\alpha_p} = (r, u)$ and send code r, and the answer is $R(u, (\gamma_p)_r)$. For an item row, send active precisely when $v_i(\alpha_p) = 1$. If all α_j agree and the code is active, Bob identifies the unique active item in bin p and uses $\xi_{\pi_{\alpha_p}(i)}$. Otherwise he uses ξ_{s+1}. Bob can test agreement from his own input. For both row types, the players then finish by running a cost-y protocol for R once.

For an arbitrary graph, we use any public assignment and replace the active code by the s possible values of $\pi_{\alpha_p}(i)$. There are now $q_1 + s + 1 = 2s - 1$ positions, so the third message costs one additional bit. The same rules work for every column tag. Exchanging the players gives the protocol for $N = B_{\mathrm{raw}}^{\mathsf{T}}$, and restriction preserves its cost. □

6.3 A collision survives the conversation

The lower bound uses the NO promise only through Lemma 6.3. We need its collision count for every assignment of the retained items, because Alice's first messages fix the assignment before any index is revealed.

Proof of Lemma 6.3. For a YES graph, let $c : V \to [3]$ be a proper three-coloring given by Theorem 5.2. Assign each vertex v to bin $c(v)$, and assign the $s - 3$ auxiliary labels to the remaining bins. Every edge coordinate activates that edge's two endpoints and those auxiliaries in one group, so all its active items occupy distinct bins. Padded coordinates activate no items.

Now suppose that G is NO. By the deletion hypotheses in Lemma 6.3, fewer than $s/2$ groups lose an auxiliary item and fewer than $s/2$ groups lose more than $\delta_v n$ vertices. Hence more than $3s$ groups retain every auxiliary item and all but at most $\delta_v n$ vertices.

Fix such a group. If two auxiliaries share a bin, they are active together at every edge coordinate. Otherwise the auxiliaries occupy $s - 3$ distinct bins, leaving three free bins. Color the retained vertices in those free bins by their bins, and extend the coloring arbitrarily to all other vertices. The count below holds for every extension. At least $\gamma|E|$ edge occurrences are monochromatic by the NO promise, and at most $\Delta\delta_v n = \gamma|E|/2$ meet a deleted vertex. Every remaining monochromatic edge gives two active items in one bin: an endpoint in an auxiliary bin shares that bin with its

auxiliary, and if neither endpoint is in an auxiliary bin, the two endpoints share a free bin. Thus this group has at least $\gamma|E|/2$ edge coordinates with two active items in one bin.

It remains to find an inactive item in the same bin. For each counted coordinate, we choose one bin containing the two active items. We discard a group from the count if one of its chosen bins contains retained items from that group alone. Every discarded group has an exclusive chosen bin, and two groups cannot have the same exclusive bin, so at most s groups are discarded. In each of the more than $2s$ remaining groups, every chosen bin also contains a retained item from another group. That item is inactive at the chosen coordinate. We have therefore found more than $\gamma s|E|$ coordinates with a collision. Since $q_0 < 8s|E|$, their fraction after padding exceeds $\gamma/8$, and hence the stated $\zeta = \gamma/24$. Uniform repetition to width q_2 preserves that fraction. □

The next estimate is used to prove the template-layer deletion bounds below for full-product interlaces. For explicit lists we instead use Lemma 6.11. Both applications are in the appendix; the collision argument uses the resulting template-layer bounds.

Lemma 6.5. *Suppose $\mathcal{P}(M; x, H)$ holds for a rationally weighted matrix M, where $H \geq 3$. Let $u \geq 16$ be dyadic with $ux \leq 1$. Every full-product interlace restriction retaining at least $9u/16$ blocks, each of base row weight at least ux, and a column set of weight at least $2^{-u/1024}$, has cost at least $H + \log u$.*

The proof is in Section A.3, on p. 53.

For the collision argument, we need deletion bounds for A, R_{temp} and N_{ref}, and a *collision bound*, which says that the template positions together with the three positions of a collision cost one bit more than H_1.

Lemma 6.6 (Template-layer deletion bounds)**.** *Consider either case of Theorem 5.3: the full-product construction of Definition 6.2 with the parameters of Lemma F.2, where $\rho = 2^{-L}$, or the explicit construction of Definition 8.3 with the parameters of Lemma F.3, where $\rho = \rho_{\mathrm{g}} = 1/\mathsf{N}$ and the subscript g is suppressed. Suppose $\mathrm{D}(R) = d \geq 3$ and $\mathcal{P}(R; \rho, d)$, let $x = \rho^8$ be the block threshold, and define $H_1, H_2, C, x_{\mathrm{cap}}$ by (1). Then*

$$\mathcal{P}(A; x, H_1), \qquad \mathcal{P}(R_{\mathrm{temp}}; x, H_2), \qquad \mathcal{P}(N_{\mathrm{ref}}; x, C-1).$$

In the interlace of R on Γ (on Γ_{g} in the explicit case), the first q_1 positions together with any three distinct tail positions require at least $H_1 + 1$ bits whenever every position retains base row weight at least x_{cap} within its block and the retained set of list labels has weight greater than $1/2$.

The proof is in Section A.4, after the initial-matrix proof (p. 54).

Three proofs below follow retained input sets along a protocol prefix and count them where the prefix stops. A mark lets a set count twice until it splits into two sets. In the collision application, the mark records the extra lower bound supplied by a collision.

Lemma 6.7 (Kraft count)**.** *Let q be a power of two, and let V be a set of nodes of a binary protocol tree, none an ancestor of another, all at depth at most $\log q$. For $v \in V$ at depth r_v put $\kappa_v = q2^{-r_v}$. Suppose each node $v \in V$ holds h_v objects, $z_v \leq h_v$ of them marked, and $\sum_{v \in V}(h_v + z_v) > q$. Then some v has $h_v + z_v > \kappa_v$. For this v, if $\kappa_v \geq 2$ then $h_v \geq \kappa_v/2 + 1$, and if $\kappa_v = 1$ then $h_v \geq 2$ or $h_v = z_v = 1$.*

Proof. Kraft's inequality gives $\sum_v \kappa_v \leq q$, so some v has $h_v + z_v > \kappa_v$, and $z_v \leq h_v$ gives $2h_v > \kappa_v$. If $\kappa_v \geq 2$, then κ_v is even and $h_v \geq \kappa_v/2 + 1$. If $\kappa_v = 1$, then $h_v + z_v \geq 2$, so $h_v \geq 2$ or $h_v = z_v = 1$. □

In the applications, the objects are retained input sets. We follow them through the messages of one player, starting from sets of weight at least t, by the following *tracking rule*, which keeps every tracked set at weight at least $t2^{-r}$ after r such messages. An unmarked set follows a child retaining at least half its weight. A marked set splits into two unmarked sets if both children meet the next threshold, and otherwise it follows a child meeting that threshold and keeps its mark. Thus every mark splits at most once. Before it splits, a marked set loses less than $\sum_{r\ge 0} t2^{-r-1} = t$ of its weight. A split replaces the contribution 2 of a marked set to $\sum_v (h_v + z_v)$ by $1 + 1$. Hence $\sum_v (h_v + z_v)$ equals the number of initial sets plus the number of initial marks.

Lemma 6.8 (Following collision witnesses)**.** *Let $q \ge 2$ be dyadic, let $0 \le b < q$ be an integer, and let $x > 0$ satisfy $2qx \le 1/8$. Suppose a protocol with at most $H + \log q$ bits left has $q - b$ indices, each with a retained set of list inputs of weight at least $3/4$. Each set consists of inputs with that fixed index. Mark $w_0 > b$ of these sets. Assume the following lower bounds, where all set weights are computed from the original input weights.*

(i) Until Alice speaks, a set of retained lists at one index with weight at least x, against the unchanged set of Alice's inputs, costs at least $H > 0$.

(ii) At the first Alice bit, one child retains a set of Alice's inputs on which, for every dyadic $u \ge 2$ with $ux \le 1$, retained sets at any $u/2 + 1$ distinct indices, each of weight at least ux, cost at least $H + \log u$.

(iii) A marked set retaining weight greater than $1/2$ costs at least $H + 1$ before Alice speaks.

Then no such protocol exists.

Proof. We track the retained list sets of each index through Bob bits by the tracking rule with $t = 2qx$, and we stop at the first Alice bit. A marked set loses less than $2qx \le 1/8$ before it splits, so an unsplit mark retains weight greater than $1/2$. By (i), a tracked set cannot end at a leaf, nor reach Bob depth $\log q + 1$: it would still have weight at least x but only $H - 1$ bits left. The stopping nodes therefore satisfy the hypotheses of Lemma 6.7, where v holds h_v tracked sets and z_v unsplit marks, and $\sum_v (h_v + z_v) = q - b + w_0 > q$. Let v be the node given by that lemma, with $\kappa = \kappa_v$. Its sets have distinct indices, because descendants of a split never rejoin, and each has weight at least $2\kappa x$.

If $\kappa \ge 2$, then (ii) with $u = \kappa$ requires $H + \log \kappa$ bits after Alice's bit, but only $H + \log \kappa - 1$ remain. If $\kappa = 1$ and $h_v \ge 2$, use (ii) with $u = 2$ and the available weights $2x$. If $h_v = z_v = 1$, then by (iii) the unsplit marked set needs $H + 1$ bits at v, where only H remain. □

Theorem 6.9 (Source hardness after deletion)**.** *Use either the full-product construction of Definition 6.2 or the explicit-list construction of Definition 8.3, with a NO graph and $\mathrm{D}(R) = d \ge 3$, and define $H_1, H_2, C, x_{\mathrm{cap}}$ by (1). Suppose the deletion bounds and the collision bound stated in Lemma 6.6 hold. Assume the numerical inequalities (10) of Lemma F.1, that the weights on S give exact eight-wise independence with the prescribed weights on J as marginal, and that the first column family's bin tuple has exactly eight-wise independent weights with the prescribed J_2 marginal. Then $\mathrm{D}(N) = C$, and every restriction retaining at least $1 - \eta$ of each input set's total weight has cost at least C. In either case of Theorem 5.3 these hypotheses follow from Lemma 6.6, Lemma F.1, and the exact marginals of the construction.*

Proof. We first outline the proof. Alice's first $\log s$ bits assign the items to bins. The collisions of this assignment give the marked sets of Lemma 6.8, and Lemma 6.6 gives its three lower bounds. That lemma then rules out depth $C - 1$.

Suppose a restriction of N retaining at least $1-\eta$ of each input set's weight has a protocol of depth at most $C-1$. We transpose it to a protocol for B_{raw}. Each template bin loses at most $\eta s/\lambda < 1/1000$ of its total S weight. The first column family Ω retains normalized weight at least $1-\eta/\lambda > 99/100$. Template entries ignore the tail ξ. We therefore project the retained Ω-inputs onto their bin tuple $(j_1,\dots,j_s)$ (in the explicit case, onto their $\mathcal{J}_{\mathrm{g}}$ label) and choose one retained completion for each image label. These representatives form a restriction of an interlace of R_{temp} on the template rows, with column weight at least $99/100$ (Observation 4.5).

We apply Theorem A.3 to the protocol inherited on these representatives, without pruning its tree. We take $M = R_{\mathrm{temp}}$, $d = H_2$, $u = s$, x in place of its ρ, and row threshold $999/1000$ in place of its ξ. Its hypotheses follow from the two deletion bounds $\mathcal{P}(R_{\mathrm{temp}}; x, H_2)$, the depth $C-1 = H_2 + \log s$, and the numerical inequalities $999/1000 > 2sx$ and $999/1000 - 2(s-1)x > 998/1000$. By that theorem, every binary history of length below $\log s$ is reached by template inputs, and every such node is Alice's. They are actual nodes of the original protocol, so Alice sends its first $\log s$ bits on all retained inputs, including item rows and $\mathcal{D}$-inputs. At the resulting s nodes, the respective template bins retain sets $A_1,\dots,A_s \subseteq S$, each of weight greater than $998/1000$. We needed the representatives only to establish this prefix. From now on we use all originally retained columns, with $C-1-\log s = H_2$ bits left.

We next fix the item assignment before choosing an index. We set aside items whose rows retain less than half of their base row weight in the restriction. By the row weights in Definition 6.2, each item type has total weight $1/512$, so we set aside fewer than $4096\eta sn$ vertex items and fewer than $4096\eta s(s-3)$ auxiliary items. These numbers are below the deletion thresholds of Lemma 6.3, because (10) gives $4096\eta sn \le \delta_v n/16$ and $4096\eta s(s-3) < s/2$, as checked in the proof of Lemma F.1. By averaging over these s nodes, we can assign every remaining item to a node that receives at least $1/(2s)$ of its base weight. The parameter choices give $4s^2\rho \le 1$, so

$$\frac{1}{2s} \ge 2s\rho = x_{\mathrm{cap}}.$$

Thus the assigned part retains at least x_{cap} of the item's base row weight. This assignment is fixed before the index is chosen. By Lemma 6.3, at least ζq_2 coordinates contain two active items and an inactive item in one bin. We assign each such coordinate to one node $j \in [s]$ at which its collision occurs and write $U_j \subseteq [q_2]$ for the resulting disjoint sets.

For each j, we discard two kinds of indices α: those at which the template labels in A_j project onto too little base row weight, and those at which too little $\mathcal{D}$-weight is retained. First, we discard α for A_j if one of the q_1 blocks of the J-projection of A_j at α retains base row weight less than x_{cap}. Since $x_{\mathrm{cap}} \le 1/8$ by (10) and each block has weight $1/q_1$ in J, the J-projection then misses more than $7/(8q_1)$ of the weight of J. If there were at least $8q_1$ such indices, the number Z of $8q_1$ of them at which z_α lies outside this projection, for $z \in S$ drawn from its weights, would satisfy $\mathbb{E}Z > 7$ and, by pairwise independence, $\operatorname{Var} Z \le \mathbb{E}Z$. Every label in A_j has $Z = 0$, so Chebyshev's inequality would give $\Pr(A_j) \le \Pr(Z=0) < 1/7$, contrary to $\Pr(A_j) > 998/1000$. Thus fewer than $8q_1$ indices are discarded for A_j. Second, we discard α if the $\mathcal{D}$-inputs with index α retain less than $3/4$ of their weight. The $\mathcal{D}$-inputs have total weight μ, and their index is uniform on $[q_2]$. Each such α therefore loses more than $\mu/(4q_2)$ of the column weight. Since the restriction deletes at most η of the column weight, we discard fewer than $4\eta q_2/\mu$ such indices. Let B_j be the union of these two sets of discarded indices and put $b_j := |B_j|$. The inequalities in (10) give

$$\sum_{j=1}^{s} b_j < 8sq_1 + 4s\eta q_2/\mu < \zeta q_2/4.$$

Since $\sum_j |U_j| \geq \zeta q_2$, some j satisfies $|U_j| > 4b_j$. Fix this j, write $b := b_j$, and define $w_0 := |U_j \setminus B_j|$. Then $w_0 \geq |U_j| - b > 3b$.

Fix a collision index $\alpha \in U_j \setminus B_j$. Let i_1, i_2 be two active items and i_3 an inactive item in one bin at α, as given by Lemma 6.3. We choose retained template rows for the first q_1 projected blocks and retained rows of i_1, i_2, i_3. By the entry rule in Definition 6.2, against a $\mathcal{D}$-input with label (α, γ) these rows read the following entries, where u is the base row of the respective row:

$$R(u, \gamma_\ell) \quad (\ell \in [q_1]), \qquad R(u, \gamma_{q_1+\pi_\alpha(i_1)}), \qquad R(u, \gamma_{q_1+\pi_\alpha(i_2)}), \qquad R(u, \gamma_{q_1+s+1}).$$

The three tail positions are distinct by the collision definition and the tail convention in Definition 6.2. Each selected row group has base weight at least x_{cap}, because $\alpha \notin B_j$ and the item assignment above was fixed with that threshold. The selected rows depend only on the projected labels and not on γ, so they form a restriction of the interlace of R on Γ in Lemma 6.6. The collision bound of Lemma 6.6 therefore requires $H_1 + 1$ bits whenever the retained $\mathcal{D}$-inputs at index α have weight greater than $1/2$ in Γ.

We now apply Lemma 6.8 with $q = q_2$ and $H = H_1$. Its $q_2 - b$ indices are those outside B_j, and its $w_0 > 3b$ marked sets are those at the indices in $U_j \setminus B_j$. The numerical bounds (10) give $q_2 x \leq 10^{-4}$, hence $2q_2 x \leq 1/8$. We now verify its hypotheses (i), (ii) and (iii).

Until Alice speaks, the set A_j of template labels is unchanged. Its projection onto each J-coordinate has weight at least $\Pr(A_j) > 998/1000 > 3/5$, so (i) follows from the deletion bounds of A (Observation 4.5). At the first Alice bit, one child retains weight greater than $499/1000 > 9/25$. After transposition, the entries of that child at indices with retained list sets are $A(\gamma, z_\alpha)$, so they form the interlace of A with those sets of lists as row blocks and tuple list S. Since S is exactly eight-wise independent and this child retains template-label weight greater than $9/25$, Lemma 6.11 with $M = A$, $d = H_1$ and $2^a = u$ gives cost at least $H_1 + \log u$ for any $u/2 + 1$ retained index sets of weight at least ux. This gives (ii), with representatives retained at that child. The collision bound established above gives (iii), which applies before that Alice bit. Hence Lemma 6.8 excludes depth $C - 1$. The upper-bound protocol of Proposition 6.4 has depth C, which proves equality. □

6.4 Lower bounds after restricting tuple inputs

Restricting the tuple inputs can destroy the independence among their coordinates (Section 2), so we use independence only in the original tuple weights. Let $0 < v \leq 1$ be a lower bound on the weight of an event, and let $0 < \tau \leq 1$ bound the allowed error. The next lemma shows that, after conditioning on the event, every joint marginal on at most eight coordinates is within total variation distance $\varepsilon(v, \tau)$ of the product weights, provided these coordinates avoid a set of at most $e(v, \tau)$ exceptional coordinates. Set

$$\varepsilon(v, \tau) := \min\{\tau, 2^{-30} v^{1/256}\},$$
$$e(v, \tau) := 8 \left\lceil 2^{20} \varepsilon(v, \tau)^{-2} v^{-1/512} \right\rceil, \qquad e(v) := e(v, 1).$$

We use independence order $8192 = 8 \cdot 1024$ so that any 1024 events on pairwise disjoint sets of at most eight coordinates are independent under the original weights, as a 1024th-moment bound requires.

Lemma 6.10 (Conditioning outside exceptional coordinates)**.** *Let ν be the normalized weights of an indexed exactly 8192-wise independent tuple list (Definition 4.4) on Z^q with marginal μ, and let $0 < v, \tau \leq 1$. For every label event Y with $\nu(Y) \geq v$, there is a set $E \subseteq [q]$, with $|E| \leq e(v, \tau)$, such that*

$$d_{\mathrm{TV}}\big((\nu \mid Y)_J, \mu^J\big) \leq \varepsilon(v, \tau) \qquad (J \subseteq [q] \setminus E,\ |J| \leq 8).$$

Here total variation is the maximum absolute difference between the two distributions' probabilities of an event. The reduction need not compute the event or the exceptional set.

The proof is in Section C.1.

Combined with Observation 4.5, this bounds the column weight of projected restrictions after conditioning. If $K \subseteq Y$ and J consists of at most eight coordinates outside E, then the inverse image of $\pi_J K$ contains K, so

$$\mu^J(\pi_J K) \geq (\nu \mid Y)_J(\pi_J K) - \varepsilon(v,\tau) \geq (\nu \mid Y)(K) - \varepsilon(v,\tau).$$

Thus the projected restriction of Observation 4.5 has column weight at least $(\nu \mid Y)(K) - \varepsilon(v,\tau)$, and we need no product assumption on $\nu \mid Y$.

The next two results apply to full products and to explicit lists. They share a proof in Section C.3 (p. 63).

Lemma 6.11 (Surviving blocks in a tuple list)**.** *Suppose $d \geq 2$ and $\mathcal{P}(M; x, d)$. Let the tuple list for an interlace be exactly eight-wise independent with the base-column marginal. If an integer $a \geq 1$ satisfies $2^a x \leq 1$, and a restriction retains at least $2^{a-1}+1$ blocks, each of base row weight at least $2^a x$, together with a tuple-label set of total weight at least $9/25$, then the restricted interlace has complexity at least $d + a$.*

After a protocol prefix, the retained tuple labels may weigh less than the $9/25$ of Lemma 6.11. We then condition on the event Y selected by the prefix and discard its exceptional coordinates, as follows.

Theorem 6.12 (Surviving blocks after conditioning)**.** *Suppose $d \geq 2$, $\mathcal{P}(M; x, d)$, and an exactly 8192-wise independent tuple list has weights ν and the base-column weights as marginal. Let $\nu(Y) \geq v > 0$, choose the exceptional set from Lemma 6.10 with any error bound $0 < \tau \leq 1$, and let an integer $a \geq 1$ satisfy $2^a x \leq 1$. If a restriction retains at least $2^{a-1}+1$ blocks indexed outside that exceptional set, each of base row weight at least $2^a x$, and a column set $K \subseteq Y$ with $(\nu \mid Y)(K) \geq 1/2$, then its complexity is at least $d + a$.*

6.5 Where displaced inputs can go

We now use w copies of N to obtain a lower bound when only a small fraction of one input set remains, following the copies through a hypothetical short protocol as in Section 2. The argument applies to both constructions. In $U = \mathsf{I}_w(N)$ each copy has row weight $1/w$, but we measure transferred weight in units of one complete copy, and losses among a copy's Ω-rows in weights normalized within Ω. Recall that Ω has weight $\lambda = 255/256$ and $\mathcal{D}$ has weight $\mu = 1/256$.

Common hypotheses. In this section we use only the following hypotheses, where H_1, H_2, C are as in (1) and $H_1, H_2 \geq 2$. To bound arriving weight in Lemma 6.14, we will fix the type/bin/item sequence and the base-label sequence by averaging. The independence in (H3) leaves the template-label weights unchanged until we condition on the retained template inputs.

(H1) The per-copy deletion bounds $\mathcal{P}(A; x, H_1)$, $\mathcal{P}(R_{\mathrm{temp}}; x, H_2)$ and $\mathcal{P}(N_{\mathrm{ref}}; x, C-1)$.

(H2) The matrix N has exact cost C, including on every restriction retaining $1-\eta$ of each input set's total weight.

(H3) The weights on S give exact 8192-wise independence across its J-valued coordinates, and the bin tuple in Ω has the same independence order and the prescribed J_2 weights as marginal. The tuple weights ν in U have the original weights on $\mathcal{K}$ at each coordinate and a representation by three independent sequences: type/bin/item labels, base labels with the X weights, and template labels with the S weights. Each sequence has a value at every copy and is exactly 8192-wise independent across copies. The decoder uses (p, z_i) at a template position and (a, u_i) at an item position. Thus the decoded labels in $\mathcal{K}$ at any at most 8192 copies have the product of the original $\mathcal{K}$ weights. The tuple coordinates $z_{i,\alpha}$, indexed by copy i and coordinate α, are also exactly 8192-wise independent: group at most 8192 coordinates by copy, then use independence between copies and among the tuple coordinates within each template input. By independence, fixing some of the three complete sequences leaves the remaining ones with their original weights.

(H4) The widths and deletion thresholds satisfy the numerical bounds in (10): each quantity in the first column is below the corresponding bound in the last column. These bounds provide enough coordinates to encode the graph and ensure that, after deleting inputs and setting aside exceptional coordinates, enough inputs remain for the lower-bound arguments.

In both cases, (H1) follows from Lemma 6.6 and (H2) from Theorem 6.9. Lemma F.1 verifies (H4) for the parameter choices of Definition 6.1. For (H3) with full products, we take all complete sequences, including values ignored by the decoder, with product weights across the three sequences. Decoding induces the original product weights on $\mathcal{K}^w$, so pulling back preserves the weights of retained sets and the entries and depth of protocols. We never need to enumerate this auxiliary representation. In the explicit case the complete sequences are the actual indexed input labels (Definition 8.3 and Lemma 8.2).

By Lemma 6.10 with $v = \theta$ and $\tau = \eta/2$, conditioning on a tuple event of weight at least θ makes at most $e(\theta, \eta/2)$ copies exceptional. Write $E_{\mathrm{exc}} = 2e(\theta, \eta/2)$. The reduction need not identify the exceptional copies.

We stop each path just before Bob's first bit or just after Alice's $(\log w)$-th bit, whichever comes first. Up to these *stopping nodes* only Alice has spoken, so each row follows the same path for every remaining column. By following the Ω-rows, we assign distinct stopping nodes to all but at most E_{exc} copies (Lemma 6.13).

Lemma 6.13. *Under the common hypotheses above, suppose a restriction of U retains row weight at least $1 - \delta_U$ and a tuple event Y' of original weight at least θ, with a protocol of depth at most $C + \log w - 1$. Apply Lemma 6.10 to this one event with error bound $\eta/2$, removing at most $E_{\mathrm{exc}}/2$ copies. At an Alice node of depth r, put $\kappa = w2^{-r}$ and follow each remaining set of Ω-rows from one copy, of normalized weight at least $2\kappa x$, into every child retaining weight at least κx. We count these row sets as follows. An unsplit row set contributes one. Once a row set first splits into two children meeting the threshold, each descendant contributes one-half. We add these contributions at each node. They count row sets and are not input weight.*

Stop at the first node where Bob speaks or at depth $\log w$, whichever comes first. The latter stopping condition applies regardless of the next speaker. Complete paths without row sets by fixed Alice bits. At a node where Bob first speaks with $\kappa \geq 2$, the total contribution is at most $\kappa/2$. A node at depth $\log w$ has at most one row set. At least $w - E_{\mathrm{exc}}$ nodes at that depth are reached by unsplit row sets from distinct copies. Assign each such node to the copy of its row set. Over all other stopping nodes, the sum of κ is at most E_{exc}.

Proof. Since $\delta_U = \eta/(16w)$, initial deletion removes at most $w\delta_U = \eta/16$ complete copies in total, hence at most $\eta/16$ of any single copy, so every copy starts with normalized weight on Ω at least

$1 - \eta/(16\lambda) > 2wx$, by (10). A row set always has a child retaining at least half its input weight. As with the marks in the tracking rule after Lemma 6.7, the first split of a row set preserves the total count, since $1 = \frac{1}{2} + \frac{1}{2}$, and further splits can only increase it. Descendants of a copy never rejoin. Each of the at least $w - E_{\mathrm{exc}}/2$ copies outside the exceptional set starts with an unsplit row set, so the total count at the stopping nodes is at least $w - E_{\mathrm{exc}}/2$.

Until Bob speaks, the tuple set is the fixed Y'. For every copy whose coordinate lies outside the exceptional set, its projection has original $\mathcal{K}$ weight at least $1 - \eta/2 > 3/5$ (the display after Lemma 6.10 with $K = Y = Y'$) and gives a restriction of N_{ref} (Observation 4.5). Since a row set has row weight $\geq 2x$, the bound $\mathcal{P}(N_{\mathrm{ref}}; x, C - 1)$ therefore requires at least $C - 1 > 0$ further bits, and no row set reaches a leaf. After we pad paths without row sets, every root-to-leaf path has one stopping node, so the sum of κ over these nodes is w.

At a node where Bob first speaks with $\kappa \geq 2$, suppose there were at least $\kappa/2 + 1$ row sets. They come from distinct copies whose coordinates lie outside the exceptional set for the same event Y'. The Bob child retaining more column weight has conditional weight at least one-half. We apply Theorem 6.12 to N_{ref} and this child with $a = \log \kappa$. Here a is an integer, $\kappa x \leq wx \leq 1$, and each row set has weight $2\kappa x \geq \kappa x$. The child requires $C - 1 + \log \kappa$ bits, but only $C + \log \kappa - 2$ remain. Consequently there are at most $\kappa/2$ row sets, and their total contribution is at most $\kappa/2$.

At a node at depth $\log w$, two row sets of weight $2x$ would require C bits by the same theorem with $a = 1$ and all of Y', but only $C - 1$ bits remain. Thus a node at depth $\log w$ has at most one row set. If the set is unsplit, we assign the node to the copy supplying it. Otherwise, the set's contribution is at most one-half. If c is the number of assigned nodes, the count at every other node is at most half its value of κ, whence

$$w - E_{\mathrm{exc}}/2 \leq c + \tfrac{1}{2}(w - c), \qquad w - c \leq E_{\mathrm{exc}}.$$

Each assigned node has $\kappa = 1$. An unsplit row set follows only one path, so the assigned nodes correspond to distinct copies. The sum of κ over all other stopping nodes is bounded as claimed. □

Rows displaced from their copy's assigned node must reach either an unassigned node or a node assigned to another copy. We use the first bound for unassigned nodes and the second for nodes assigned to another copy. Their proofs are in Sections B.2 and B.3, respectively.

Lemma 6.14. *For the protocol and stopping nodes of Lemma 6.13, the total weight from $\mathcal{D}$ across all copies reaching a node where Bob first speaks before depth $\log w$ is less than $2s\kappa$. At a node at depth $\log w$ it is less than $2s$. Weight is measured in units of one complete copy of N.*

Lemma 6.15. *In the same protocol, for each copy i with an assigned node, let ε_i be the normalized weight of copy i's Ω-rows missing at that node. Every other copy that has an assigned node sends at most $\mu\delta_i$ weight from $\mathcal{D}$ to the node assigned to copy i, where*

$$\delta_i = 2q_2 x + 4\varepsilon_i.$$

These bounds lead to the contradiction shown in Figure 3, in which P_i is the $\mathcal{D}$-weight of copy i surviving outside its assigned node and a_i is its initially deleted $\mathcal{D}$-weight.

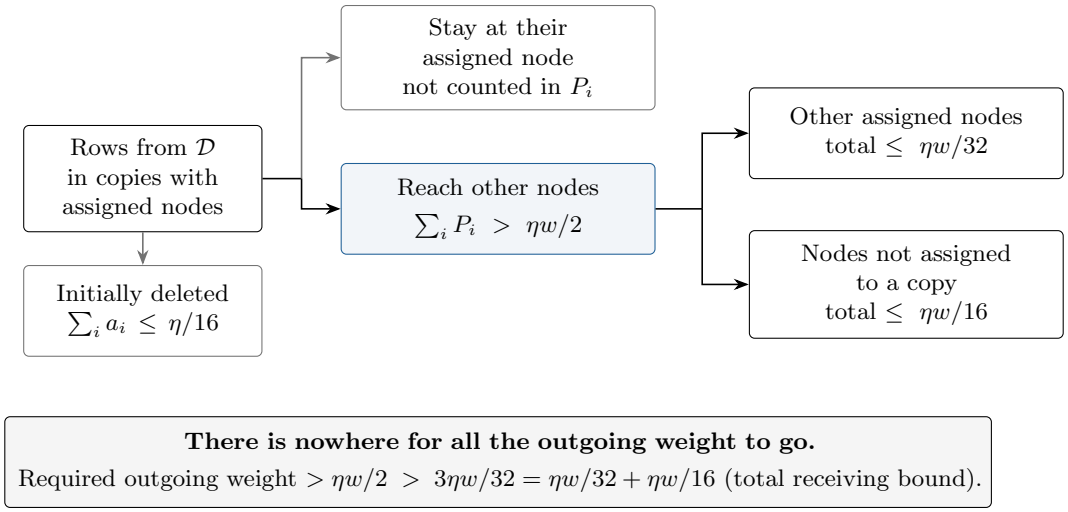


Figure 3: Where the displaced $\mathcal{D}$-weight would have to go. Every surviving $\mathcal{D}$-row outside the node assigned to its copy must reach one of the two groups of stopping nodes on the right, whose combined bound on arriving weight is too small. The second group contains nodes not assigned to copies, either where Bob first speaks before depth $\log w$ or at that depth. Rows deleted initially and rows staying at the node assigned to their copy are counted separately. The proof of Theorem 6.16 derives these bounds. All weights are in complete-copy units, and arrow widths are not to scale.

For W, the deletion bounds we obtain are one bit below the full cost. The next subsection recovers that bit.

Theorem 6.16 (Lower bounds for W)**.** *Under the common hypotheses of Section 6.5,*

$$\mathrm{D}(W) = C + \log w, \qquad \mathcal{P}(W; \theta, C + \log w - 1).$$

Every restriction of W retaining row weight at least θ and column weight at least $1 - \delta_U$ has cost at least $C + \log w$.

Proof. Suppose a restriction of W with row weight $\geq \theta$ and column weight $\geq 1 - \delta_U$ has a protocol of depth $\leq C + \log w - 1$, and transpose the restriction and its protocol to U. Its retained tuple event Y' has weight at least θ. By Lemma 6.13 and $E_{\mathrm{exc}} < w/4$ from Lemma F.1, at least $w - E_{\mathrm{exc}} \geq 3w/4$ copies have assigned nodes, each with only $C - 1$ bits left. These copies lie outside the exceptional set, so conditioning on Y' with error bound $\eta/2$ gives each of them a column projection of original weight at least $1 - \eta/2$, which gives a restriction of N (Observation 4.5). By Theorem 6.9, the assigned node must miss more than η of the row weight in its assigned copy.

An unsplit set of Ω-rows discards less than $\sum_{r=0}^{\log w - 1} wx2^{-r} = 2(w-1)x$ on its path, since at depth r the discarded child retains less than $\kappa x = wx2^{-r}$. Initial deletion removes at most $\eta/16$ complete copies. Summing over copies with assigned nodes and using $w^2 x/\eta \leq 1/(32\lambda)$ from (10), we obtain

$$\sum_i \varepsilon_i \leq \frac{\eta}{16\lambda} + 2w(w-1)x \leq \frac{\eta}{8\lambda}.$$

For these copies, the missing Ω-rows, initially deleted $\mathcal{D}$-rows, and $\mathcal{D}$-rows surviving elsewhere partition the part absent from their own assigned node. Thus $P_i + a_i + \lambda\varepsilon_i > \eta$, and

$$\sum_i P_i > \frac{3\eta w}{4} - \frac{\eta}{16} - \frac{\eta}{8} > \frac{\eta w}{2}.$$

We now show that the other stopping nodes cannot receive this much weight. The sum of κ over nodes not assigned to copies is at most E_{exc}, so Lemma 6.14 bounds their arriving weight by $2sE_{\mathrm{exc}} \le \eta w/16$. By Lemma 6.15, other assigned nodes receive at most

$$\mu w \sum_i (2q_2 x + 4\varepsilon_i) \le \eta w \left(\frac{1}{64} + \frac{1}{510}\right) < \frac{\eta w}{32},$$

using $4\mu w q_2 x \le \eta/32$ and the bound on losses within Ω above. Since only Alice has spoken, every surviving row reaches one stopping node. The total arriving weight is therefore less than $3\eta w/32$, contradicting the outgoing bound and proving the claim.

For the deletion bounds $\mathcal{P}(W;\theta, C + \log w - 1)$, we fix $j \in \{0,1\}$ and transpose a restriction of W with row weight at least θ and column weight at least $\beta_j = (3/5)2^{-j}$. In U its tuple event Y has weight at least θ. We discard the $\mathcal{D}$-rows, leaving a restriction of the interlace of $N_{\mathrm{ref}} = N[\Omega, \mathcal{K}]$. The average retained weight on Ω, normalized within Ω, is at least $\beta'_j = (\beta_j - (1-\lambda))/\lambda$. We put $\kappa = w2^{-j}$ and apply Lemma 6.10 to Y with error bound one. If m copies have coordinates outside the exceptional set and retain normalized weight on Ω at least κx, block averaging gives $w\beta'_j \le m + e(\theta) + w\kappa x$ and hence

$$\frac{m}{\kappa} \ge 2^j \beta'_j - wx - \frac{4e(\theta)}{w} \ge \frac{(3/5) - 4(1-\lambda)}{\lambda} - \frac{1}{256} - \frac{1}{32} = \frac{44}{75} - \frac{1}{256} - \frac{1}{32} > \frac{1}{2}.$$

Thus at least $\kappa/2 + 1$ blocks indexed outside the exceptional set remain. We apply Theorem 6.12 to N_{ref} with $a = \log \kappa$, using all of Y, of conditional weight one. The parameter choices give $\kappa \ge 2$ and $\kappa x \le 1$. The lower bound is $C - 1 + \log \kappa = C + \log w - 1 - j$, proving both bounds after transposition. Applied to the whole matrix, the restriction bound proved first gives $\mathrm{D}(W) \ge C + \log w$. Conversely, the player holding the copy index sends it in $\log w$ bits, and the players then run the cost-C protocol for N, giving equality. □

6.6 Restoring the bound for the next round

To repeat the construction, we need the deletion bounds at the exact cost. Theorem 6.17 recovers the missing bit from the bound of Theorem 6.16 for column weight at least $1 - \delta_U$. Its proof follows the many-block estimate in Section A.3 (p. 53).

Theorem 6.17 (Restoring the deletion bounds)**.** *Let W have $m \ge 1$ rows and $n \ge 1$ columns, and suppose its exact cost satisfies $\mathrm{D}(W) \ge 3$. Let $\theta > 0$ and $0 < \delta \le 1/64$. Assume that every restriction retaining row weight at least θ and column weight at least $1 - \delta$ has cost at least $\mathrm{D}(W)$, and that $\mathcal{P}(W;\theta, \mathrm{D}(W) - 1)$ holds. For an integer $b \ge 1$, choose any power of two $Q = 2^r$ satisfying $Q\delta \ge 80b$, and assume $Q\theta \le 1/20$. Then $R^+ = (\mathsf{I}_Q(W))^{\mathsf{T}}$ satisfies*

$$\mathrm{D}(R^+) = \mathrm{D}(W) + r, \qquad \mathcal{P}(R^+; 2^{-b}, \mathrm{D}(W) + r).$$

Independently of the lower-bound hypotheses, any upper bound on $\mathrm{D}(W)$ gives an upper bound larger by r on $\mathrm{D}(R^+)$. The matrix R^+ has exactly n^Q rows and Qm columns.

The final sampling step of Section 9.2 produces a small table only if the rank of R^+ grows slowly. The next lemma bounds this rank in both the full-product and the explicit case.

Lemma 6.18. *In the construction encoding the graph, let $h = 2\log s + \log q_2 + \log w + \log Q$ and suppose $|\mathcal{I}| \le 2sq_2$. Then $\mathrm{rank}(R^+) \le 2^h \mathrm{rank}(R)$. The same bound holds if any tuple set is replaced by an indexed list of tuples from that set, including products of such lists.*

Proof. A width-t interlace stacks t blocks obtained by selecting or repeating columns of its base matrix, and hence multiplies rank by at most t. The template rows form nested interlaces of widths q_1, q_2 and s, so the template part has rank at most $sq_1q_2\,\mathrm{rank}(R)$. For each item i, its block has entries $R(u, \psi_i(Z))$, where the selected column $\psi_i(Z)$ is independent of u, so its rank is at most $\mathrm{rank}(R)$. After the outer interlaces, rank subadditivity gives

$$\mathrm{rank}(R^+) \le wQ(sq_1q_2 + |\mathcal{I}|)\,\mathrm{rank}(R) \le wQsq_2(q_1+2)\,\mathrm{rank}(R) = 2^h\,\mathrm{rank}(R),$$

using $q_1 + 2 = s$. Restricting tuple lists, duplicating indexed labels, and transposing preserve this argument. □

Proof of Theorem 5.3. As in Lemma 6.6, we suppress the subscript g in the explicit case. In both cases $h = 2\log s + \log q_2 + \log w + \log Q$. Beyond its effective cutoff, Lemma F.2 gives $h = L + O_{\Delta,\gamma}(\log L)$ in the full-product case, and Lemma F.3 gives $h = 40\log s = 10\log \mathsf{N}$ in the explicit case.

We first prove (ii), so let the graph be NO, $\mathrm{D}(R) = d \ge 3$ and $\mathcal{P}(R; \rho, d)$. By Lemma 6.6, the per-copy deletion bounds and the collision bound hold. The construction has exact marginals (in the explicit case, those of S_{g} and $\mathcal{J}_{\mathrm{g}}$ in Definition 8.3), and Lemma F.1 gives the numerical bounds (10). Hence these bounds meet the hypotheses of Theorem 6.9, so $\mathrm{D}(N) = C$ even after we delete at most η of each input set's weight. This gives (H1) and (H2) of Section 6.5, and (H4) is Lemma F.1. We verified (H3) for full products right after stating the hypotheses. In the explicit case, Definition 8.3 provides the three independent sequences. Lemma 8.2 shows that decoding the nested template labels gives the exact 8192-wise independence of (H3), and that fixing the other two sequences leaves it unchanged. Thus Theorem 6.16 gives $\mathrm{D}(W) = C + \log w$ and $\mathcal{P}(W; \theta, C + \log w - 1)$, and cost $\ge C + \log w$ on every restriction with row weight $\ge \theta$ and column weight $\ge 1 - \delta_U$. Only the restoration step differs between the cases.

- *Full-product case.* By Lemma F.2, $0 < \delta_U \le 1/64$, $Q\delta_U \ge 80L$, and $Q\theta = 1/512 < 1/20$, so Theorem 6.17 applies with $b = L$ and $\delta = \delta_U$, at threshold $2^{-L} = \rho$.
- *Explicit case.* Theorem 7.2, which restores the deletion bounds at threshold ϕ, applies with $(\phi, \delta, Q) = (\rho, \delta_U, Q)$. Lemma F.3 verifies its numerical hypotheses, and Definition 8.3 gives the outer list the prescribed exact marginals.

In both cases we restore the deletion bounds at threshold ρ and cost

$$C + \log w + \log Q = d + \underbrace{2\log s + \log q_2 + \log w + \log Q}_{h} + 1,$$

so $\mathrm{D}(R^+) = d + h + 1$ and $\mathcal{P}(R^+; \rho, d + h + 1)$.

For (i), on either graph, the players send the copy index at each of the two interlaces, in $\log w + \log Q$ bits, and then run the protocol of Proposition 6.4. This gives $\mathrm{D}(R^+) \le \mathrm{D}(R) + h + 1$. On a YES graph we omit the final $+1$ of Proposition 6.4, giving $\mathrm{D}(R^+) \le \mathrm{D}(R) + h$. In the explicit case every input decodes locally to the corresponding full-product input with the same entry, and the protocol acts only on decoded inputs, so it applies unchanged.

For (iii), the bound $q_2 \ge 4s|E|$ from (10) and $|E| \ge n/2$ imply $|\mathcal{I}| = 4s(n + s - 3) \le 2sq_2$, so Lemma 6.18 gives the rank bound in both cases. □

7 Interlacing on small input lists

7.1 Constructing exact marginals

The deterministic reduction replaces each full product by an explicit tuple list (Definition 4.4) whose induced weights on every set of at most 8192 coordinates equal the product weights.

The construction adds one coordinate at a time. To each tuple we attach the zero–one vector indicating its values on all sets of at most 8192 coordinates. After each extension, the construction moves weight along linear dependences among these vectors. Each move removes at least one tuple, keeps all weights nonnegative and leaves the joint weights on these coordinate sets unchanged. The moves continue until no more tuples remain than there are constraints. This is the sequential construction idea of Koller and Megiddo [15, Section 3]. The proof of the next lemma, including the implementation and bit bound, is in Section D. We store the rational weights explicitly and use them to measure retained input sets.

Lemma 7.1 (Small rational tuple lists)**.** *Fix an integer independence order $\mathsf{D} \geq 1$. Given normalized rational weights μ on an alphabet of size m and an integer $q \geq 1$, a deterministic algorithm constructs an exactly D-wise independent tuple list $\mathsf{Code}_{\mathsf{D}}(q, \mu)$ (Definition 4.4) with a label set of size at most $(1 + qm)^{\mathsf{D}}$. For fixed D, the construction time, total output length and rational-weight bit lengths are polynomial in q, m and the binary encoding length of μ.*

Below we always use Code_{8192}. Keeping the weights as binary-encoded rational numbers avoids expanding them into repeated labels: a denominator with polynomially many bits can represent far too many copies to print in polynomial time.

7.2 Restoring the deletion bounds

The matrix W of Theorem 6.16 satisfies the deletion bounds one bit below its cost, and has full cost on restrictions retaining almost all column weight. One more interlace recovers the missing bit. The proof distinguishes whether Bob has spoken. Before his first message the bound for almost all columns applies. Afterward, Lemma C.3 gives the missing bit.

Theorem 7.2. *Let $0 < \phi \leq 1$, $0 < \delta \leq 1/64$ and $\theta > 0$. Suppose W, with input weights μ_X, μ_Y, satisfies $\mathrm{D}(W) = C \geq 3$, $\mathcal{P}(W; \theta, C - 1)$, and*

$$\mathrm{D}(W[A, B]) \geq C \quad \text{whenever } \mu_X(A) \geq \theta \text{ and } \mu_Y(B) \geq 1 - \delta.$$

Let Q be a power of two satisfying

$$Q \geq 128\big(e(\phi, \delta/2) + 1\big), \qquad Q\theta \leq 1/20,$$

where $e(\cdot, \cdot)$, defined before Lemma 6.10, bounds the number of exceptional coordinates in that lemma. Interlace W Q times on any exactly 8192-wise independent tuple list with the base-column weights, and transpose the result to obtain R^+. Then

$$\mathrm{D}(R^+) = C + \log Q, \qquad \mathcal{P}(R^+; \phi, C + \log Q).$$

Independently of the lower-bound hypotheses, a protocol of depth U for W yields one of depth $U + \log Q$ for R^+.

Proof. To prove the lower bounds, we fix $j \in \{0,1\}$ and a restriction of R^+ retaining row weight $\geq \phi$ and column weight $\geq (3/5)2^{-j}$. After undoing the transpose, the retained columns form a label event Y of weight at least ϕ, and the within-block weights of the retained rows average at least $(3/5)2^{-j}$ over the Q blocks. We apply Lemma 6.10 to Y with error bound $\delta/2$ and put

$$e = e(\phi, \delta/2), \qquad k = Q2^{-j}, \qquad a = \log k.$$

Since Q is a power of two with $Q \geq 128(e+1) > 2$, a is a positive integer. More than $k/2$ blocks have coordinates outside the exceptional set and retained base-row weight at least $k\theta$: otherwise the block-averaging observation gives $(3/5)Q2^{-j} \leq k/2 + e + Qk\theta$, and hence

$$\frac{3}{5} \leq \frac{1}{2} + \frac{2^j e}{Q} + Q\theta \leq \frac{1}{2} + \frac{1}{64} + \frac{1}{20} < \frac{3}{5}.$$

We retain these blocks and all of Y.

Suppose the restriction to these blocks and Y has a protocol of depth at most $C + a - 1$, and let ν denote the tuple-label weights. We follow the protocol for a messages, starting from conditional column weight one, by the weight-halving walk from the proof of Theorem 6.12, which there runs for $a-1$ messages. At each Alice message, we assign every current block to a child retaining at least half its row weight and keep a child assigned at least half the blocks. At each Bob message, we keep a child retaining at least half the current column weight under $\nu \mid Y$. If the protocol ends earlier, we pad with dummy Alice messages. If α messages are Alice's and β are Bob's, then $\alpha + \beta = a$, and the surviving restriction has a column set $K' \subseteq Y$ and

$$\text{more than } 2^{\beta-1} \text{ blocks outside the exceptional set}, \qquad \text{row weight per block at least } 2^\beta\theta,$$
$$(\nu \mid Y)(K') \geq 2^{-\beta}, \qquad \text{at most } C-1 \text{ bits remaining.}$$

If $\beta = 0$, the columns are still all of Y. At least one block indexed outside the exceptional set remains, and its row weight is at least θ. Its projected column set has base weight at least $1 - \varepsilon(\phi, \delta/2) \geq 1 - \delta$, by Lemma 6.10. Choosing retained representatives (Observation 4.5), we obtain a restriction of W that requires C bits by the hypothesis for column weight $\geq 1 - \delta$.

If $\beta \geq 1$, we apply the short-continuation bound Lemma C.3 with $d = C - 1$, $b = \beta - 1$, and $x = \theta$. Its hypotheses hold. The number of blocks is an integer exceeding $2^{\beta-1}$, so it is at least $2^b + 1$. Each block has row weight at least $2^{b+1}x$, and $(\nu \mid Y)(K') \geq 2^{-b-1}$. Also $d \geq 2$ and $2^{b+1}x = 2^\beta\theta \leq k\theta \leq 1$. The lemma therefore requires $d + 1 = C$ bits. In both cases this exceeds the $C - 1$ remaining bits.

Thus each restriction requires at least $C + a = C + \log Q - j$ bits. Transposing back proves both deletion bounds and the lower bound on the whole matrix. For the upper bound, send the block index in $\log Q$ bits, decode that coordinate, and run the depth-U protocol for W. Exchanging the players gives the same cost for R^+. Taking $U = C$ proves equality. □

8 Deterministic gap tables

The shared amplification proof applies to explicit lists once their marginal weights and independence properties are verified. We now construct these lists and bound their encoding length, making each round a deterministic polynomial-time operation. Theorem 8.4 gives the resulting gap and encoding bound after k rounds.

The subscript g marks parameters and matrices of the explicit-list construction. We choose a power of two s_{g} above the fixed cutoff in Lemma F.3, with $\mathsf{N} := s_{\mathrm{g}}^4 \geq |V(G)|$ for the source graph

G. Set $L_{\mathrm{g}} = \log \mathsf{N}$ and $\rho_{\mathrm{g}} = 1/\mathsf{N}$. The choices in Definition 6.1 then fix every other parameter as a function of s_{g}. In particular, since $\rho_{\mathrm{g}} = s_{\mathrm{g}}^{-4}$, $q_{2,\mathrm{g}} = s_{\mathrm{g}}^2/\rho_{\mathrm{g}} = s_{\mathrm{g}}^6$, $w_{\mathrm{g}} = s_{\mathrm{g}}^8$, $Q_{\mathrm{g}} = s_{\mathrm{g}}^{24}$, and the block threshold is $x_{\mathrm{g}} = \rho_{\mathrm{g}}^8$. These values are the same in every round, although the base matrix R changes from round to round.

Corollary 8.1 (Explicit case)**.** *For the construction $R \mapsto R_{\mathrm{g}}^{+}$ of Definition 8.3 with the parameters of Lemma F.3, parts (i) and (ii) of Theorem 5.3 hold with $L = L_{\mathrm{g}}$, so that $2^{-L} = \rho_{\mathrm{g}}$, and with*

$$h = h_{\mathrm{g}} := 2\log s_{\mathrm{g}} + \log q_{2,\mathrm{g}} + \log w_{\mathrm{g}} + \log Q_{\mathrm{g}} = 40 \log s_{\mathrm{g}} = 10 \log \mathsf{N}.$$

The construction is deterministic and polynomial in N and the explicit rational encoding length of R. Every generated input decodes to an input of the corresponding full-product construction with the same entries.

After defining the lists below, we prove this corollary in Section 8.2.

8.1 Replacing products by explicit lists

A coordinate of a tuple in one small list can itself be a label of another small list. The next lemma shows that decoding such nested labels preserves the product weights on every small set of coordinates. It also records why fixing independently weighted labels of other lists does not change these weights. As in Section 4, inputs with the same decoded value keep separate labels and weights.

Lemma 8.2. *Let an indexed list on $Z^{q_{\mathrm{in}}}$ be exactly* 8192*-wise independent with marginal μ_Z. Choose labels of this inner list at q_{out} positions using an exactly* 8192*-wise independent outer list whose marginal equals the inner label weights. Then the decoded coordinates, indexed by outer–inner pairs, are exactly* 8192*-wise independent with marginal μ_Z. If the outer-list label has product weights with the tuple of labels from finitely many other lists, fixing those other labels at any joint value of positive weight leaves its decoded weights unchanged.*

Proof. Given at most 8192 prescribed decoded coordinates, we group them by outer position. Outer independence factors their joint weight over these groups, and inner independence factors each group into its prescribed coordinate marginals. Each factorization involves at most 8192 coordinates, as both independence hypotheses require. Fixing the other labels leaves the outer-label weights unchanged, and hence also leaves their decoded weights unchanged. □

Definition 8.3. Let $R : X \times Y \to \{0,1\}$ be a rationally weighted base matrix and use the parameter choices of Lemma F.3 and construct the items and their values at each coordinate as in Definition 6.2. Replace the product alphabets Γ, S, and J_2^s by the indexed lists Γ_{g}, S_{g}, and $\mathcal{J}_{\mathrm{g}}$ of Lemma 7.1, defined by

$$\Gamma_{\mathrm{g}} = \mathsf{Code}_{8192}(2s_{\mathrm{g}} - 1, \mu_Y), \qquad J_{\mathrm{g}} = [q_{1,\mathrm{g}}] \times X, \qquad S_{\mathrm{g}} = \mathsf{Code}_{8192}(q_{2,\mathrm{g}}, \mu_{J_{\mathrm{g}}}),$$

$$J_{2,\mathrm{g}} = [q_{2,\mathrm{g}}] \times \Gamma_{\mathrm{g}}, \qquad \mathcal{J}_{\mathrm{g}} = \mathsf{Code}_{8192}(s_{\mathrm{g}}, \mu_{J_{2,\mathrm{g}}}).$$

Integer factors are uniform, and all other factors have their base or indexed weights. On decoded labels use

$$A_{\mathrm{g}}(\gamma, (r,u)) = R(u, \gamma_r), \qquad R_{\mathrm{temp,g}}(z, (\alpha, \gamma)) = A_{\mathrm{g}}(\gamma, z_\alpha).$$

Use the raw entry rules, family tags, and type weights of Definition 6.2. For the family corresponding to Ω, the column weights are the product of the weights of a $\mathcal{J}_{\mathrm{g}}$ label and the weights induced on the tail by Γ_{g}. For the family corresponding to $\mathcal{D}$, the weights are induced from $J_{2,\mathrm{g}}$ under the map

defining $\mathcal{D}$: the tail and all repeated labels use the same γ. This defines $B_{\mathrm{raw,g}}$, its transpose N_{g}, and its normalized restriction $N_{\mathrm{ref,g}}$ to inputs with the Ω family tag.

The lower-bound proof fixes the type/bin/item sequence and the base-label sequence before restricting the template labels; the latter must still have their original weights, as required by (H3). To interlace w_{g} copies of N_{g}, we therefore use three tuple lists with product weights between their labels. Each list consists of sequences of length w_{g} and is exactly 8192-wise independent under its label weights. Type/bin/item labels have the weights induced by those labels in the raw rows, base labels have weights μ_X, and template labels have the S_{g} weights. We enumerate the Cartesian product of these three label sets. At each position, the type label in the first sequence determines whether the decoded row is a template row (p, z), from the bin and template labels, or a row (i, u) indexed by an item, from the item and base labels. Since the three sequences are independent and each is exactly 8192-wise independent, their coordinatewise combination gives an exactly 8192-wise independent tuple list with the raw-row marginal. Interlacing N_{g} on this list defines U_{g}. Set $W_{\mathrm{g}} = U_{\mathrm{g}}^{\mathsf{T}}$. We interlace W_{g} at width Q_{g} using Code_{8192} with its column weights and then transpose to obtain R_{g}^{+}. We store all input labels and rational weights explicitly, keeping inputs with equal decoded values separate.

As in Observation 4.5, we obtain projected restrictions from chosen representatives. The per-copy deletion bounds and the collision bound are the explicit case of Lemma 6.6. Their proofs use Lemma 6.10 and Theorem 6.12 to handle retained events of weight as small as x_{g}. With these bounds, the collision and stopping-node arguments of Theorem 6.9 and Section 6.5 apply to the indexed construction.

8.2 Iteration, encoding size and deterministic running time

To prove Corollary 8.1, we need only add the encoding bound to the explicit case of Theorem 5.3.

Proof of Corollary 8.1. We suppress the subscript g. Each width is a fixed polynomial in N. Therefore the fixed number of calls to Lemma 7.1, all of order 8192, and enumeration of the Cartesian products of their label sets take time bounded by a fixed polynomial in N and the encoding length of R. The indexed labels and rational weights needed by the next round have encoding length within the same bound.

Parts (i) and (ii) follow from the proof of Theorem 5.3, with Theorem 7.2 used in place of Theorem 6.17. We apply Theorem 7.2 with $(\phi, \delta, Q) = (\rho, \delta_U, Q)$, and Lemma F.3 verifies its numerical hypotheses. The upper-bound protocol uses only the decoded inputs and not their weights, so it also works for the indexed construction. □

Theorem 8.4. *There are effective absolute constants $\mathsf{A} \geq 2$ and $C_0 > 0$ and a deterministic algorithm with the following guarantee. Given a YES or NO source graph G from Theorem 5.2 and an integer $k \geq 1$, it produces an explicit total Boolean matrix $F_{G,k}$ and a public integer B_k^{g} such that*

$$\textit{YES: } \mathrm{D}(F_{G,k}) \leq B_k^{\mathrm{g}}, \qquad \textit{NO: } \mathrm{D}(F_{G,k}) = B_k^{\mathrm{g}} + k.$$

For every $\mathsf{N} = s_{\mathrm{g}}^4 \geq |V(G)|$ meeting the parameter conditions of Lemma F.3, its running time and full binary output length are at most $\mathsf{N}^{C_0 \mathsf{A}^k}$. The constants depend only on the fixed source reduction and construction. The output may be square with power-of-two side length at least N. The public threshold is

$$B_k^{\mathrm{g}} = d_{0,\mathrm{g}} + k h_{\mathrm{g}},$$

where $d_{0,\mathrm{g}}$ is the cost of the initial matrix in Lemma 5.4 with $L = L_{\mathrm{g}}$:

$$d_{0,\mathrm{g}} = \log \mathrm{pow}_2(64(L_{\mathrm{g}} + 1)) + 1, \qquad L_{\mathrm{g}} = \log \mathsf{N}.$$

Proof. We choose the parameters of Lemma F.3 and let R_0 be the initial matrix of Lemma 5.4 with $L = L_{\mathrm{g}} = \log \mathsf{N}$. We apply Definition 8.3 k times, using each output as the next base matrix and storing each indexed table with its rational input weights. Finally, we forget the weights and duplicate inputs to make a square table with power-of-two side at least N.

By Lemma 5.4, the initial matrix has exact cost $d_{0,\mathrm{g}} \geq 3$ and satisfies the deletion bounds at threshold $\rho_{\mathrm{g}} = 2^{-L_{\mathrm{g}}}$. Proposition 5.5, in the explicit case (Corollary 8.1), therefore gives

$$\mathrm{D}(R_k) \leq d_{0,\mathrm{g}} + k h_{\mathrm{g}} \quad \text{on YES}, \qquad \mathrm{D}(R_k) = d_{0,\mathrm{g}} + k(h_{\mathrm{g}} + 1) \quad \text{on NO}$$

for the matrix R_k after k rounds. Forgetting weights and duplicating rows and columns to make the table square change neither cost. This proves the gap with $B_k^{\mathrm{g}} = d_{0,\mathrm{g}} + k h_{\mathrm{g}}$.

For size and time, let Z_i be $\mathsf{N}+2$ plus the complete encoding length after round i, including labels and normalized rational weights. The polynomial bound for the initial matrix and Corollary 8.1 give effective constants C_1 and a power of two $\mathsf{A} \geq 2$, independent of i and k, such that

$$Z_0 \leq \mathsf{N}^{C_1}, \qquad Z_{i+1}, \text{ time for round } i+1 \leq Z_i^{\mathsf{A}}, \qquad Z_i \leq \mathsf{N}^{C_1 \mathsf{A}^i}.$$

After enlarging the effective constant C_0 if necessary, we bound the sum of the k round times and the final square padding by $\mathsf{N}^{C_0 \mathsf{A}^k}$. Computing the public threshold and the parameters takes time within the same bound. □

We now derive Theorem 1.1 by taking a fixed number k of rounds, and Corollary 1.5 by taking $k = \Theta(\log \log N)$ rounds for a sparse formula on N variables.

Proof of Theorem 1.1. For fixed k, we apply the graph reduction of Theorem 5.2 and then Theorem 8.4. The least value $\mathsf{N} = s_{\mathrm{g}}^4$ meeting the parameter conditions obeys $\mathsf{N} \leq 16 \max\{|V(G)|, \mathsf{N}_0\}$ for an effective constant N_0, by Lemma F.3. Since the graph reduction runs in polynomial time and k is fixed, $\mathsf{N}^{C_0 \mathsf{A}^k}$ is polynomial in the formula length. The resulting public threshold separates cost $\leq B_k^{\mathrm{g}}$ from cost exactly $B_k^{\mathrm{g}} + k$, on a square table with power-of-two side. □

Proof of Corollary 1.5. We choose the number k of rounds so that the gap k exceeds twice the allowed error $c_* \log \log T$. At the same time, k must be small enough that the construction and the estimator run in quasipolynomial time in the size of the sparse formula. Let A be the effective power of two from Theorem 8.4, and set $c_* = 1/(8 \log \mathsf{A})$. Suppose such an estimator exists. For a sparse formula on N variables, Theorem 5.2 and Lemma F.3 give a value $\mathsf{N} = s_{\mathrm{g}}^4 \geq \max\{N, |V(G)|\}$ satisfying the parameter conditions. Since $|V(G)| \leq N(\log(N+2))^p$ for the fixed p of Theorem 5.2, we have $\mathsf{N} = N^{O(1)}$. For sufficiently large N, we choose

$$k = \left\lfloor \frac{\log \log N}{\log \mathsf{A}} \right\rfloor.$$

Then $k = \Theta(\log \log N)$ and $\mathsf{A}^k \leq \log N$. The square output of Theorem 8.4 has threshold B_k^{g} and T entries, where

$$N^2 \leq T \leq \mathsf{N}^{C_0 \mathsf{A}^k}, \qquad \log \log T \leq k \log \mathsf{A} + O(\log \log N).$$

For sufficiently large N, the second bound is at most $3k \log \mathsf{A}$: indeed it is at most $2 \log \log N + O(1)$, whereas $k \log \mathsf{A} = \log \log N + O(1)$. Consequently

$$2 c_* \log \log T \leq 3k/4 < k.$$

Comparing the estimate with the public midpoint $B_k^{\mathrm{g}} + k/2$ therefore decides satisfiability. Both construction and the estimator's fixed polynomial running time in T are bounded by

$$2^{O((\log N)^2)}.$$

The estimator's guarantee holds for all sufficiently large tables, so it applies here because $T \geq N^2$. For the finitely many remaining values of N, including those with $k < 1$, exhaustive search over assignments decides satisfiability. This proves the claimed deterministic running-time implication. □

9 Final sampling and growing error

In the full-product case of Proposition 5.5, the iterated matrix R_k has the desired gap but is too large to print, so we replace it by a sampled submatrix. A single sample must be hard for every protocol of depth below the NO-case bound. A small sample suffices because the rank bound $\operatorname{rank}(R_k) \leq r_k$ limits the number of column sets on which such protocols can be correct (Theorem 9.3).

9.1 Counting column sets on which short protocols are correct

For the union bound in Section 9.2, we count the sets of columns on which a short protocol can be correct on a retained row set. A single count covers all row sets at once. We fix the tree, Alice's messages and the leaf outputs, and leave Bob's messages free to depend on the column.

Definition 9.1 (Columns on which Bob can complete the protocol)**.** Let a depth-t tree P specify a complete binary tree of depth t, the speaker at each internal node, all Alice maps (from her row to her bit at each of her nodes), and all leaf outputs. Bob's maps are unspecified. For a Boolean matrix $F : X \times Y \to \{0, 1\}$ and $U \subseteq X$, let $G_P(U)$ be the set of columns for which some assignment of bits to Bob's nodes makes the tree correct on every row in U. The assignment may depend on the column, but not on Alice's row.

Observation 9.2 (Short protocols as containments)**.** *For retained sets $U \subseteq X$, $V \subseteq Y$ and an integer $t \geq 0$, with correctness on an empty restriction understood vacuously,*

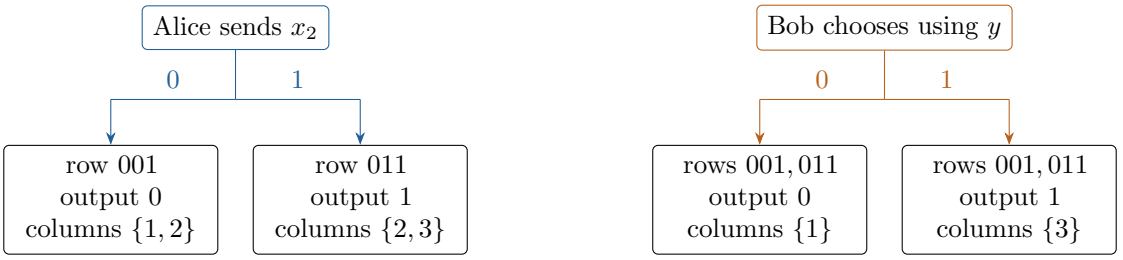
$$\mathrm{D}(F[U, V]) \leq t \quad \Longleftrightarrow \quad V \subseteq G_P(U) \text{ for some depth-}t \text{ protocol tree } P \text{ of this kind.} \tag{2}$$

Proof. Given a protocol of depth at most t, we pad it with fixed dummy messages and extend Alice's maps to all rows. For each column in V, its Bob maps give an assignment as in Definition 9.1, so $V \subseteq G_P(U)$. Conversely, for each retained column we choose such an assignment at Bob's nodes. These assignments define Bob's maps for a correct protocol on $U \times V$. Empty restrictions satisfy both assertions vacuously. □

Figure 4 uses two one-bit trees to show why leaf requirements combine by intersection at an Alice node and by union at a Bob node. We label each leaf by the columns on which its output is correct for the rows reaching it.

The same matrix in both trees: $F(x, y) = x_y$.

	Bob's column y		
Alice's row x	1	2	3
001	0	0	1
011	0	1	1



(a) **Both required: intersection.** $\{1, 2\} \cap \{2, 3\} = \{2\}$.

(b) **Either works: union.** $\{1\} \cup \{3\} = \{1, 3\}$.

Figure 4: Alice's branch depends on the row, so a column on which the tree is correct must satisfy both leaf conditions. Bob's branch depends on the column, so either leaf may work, but his chosen leaf must answer both rows correctly.

The count below uses the row-sum argument of Oum, Sæther and Vatshelle [20, Theorem 4.2]: the columns on which selected Boolean rows all vanish are exactly the zero coordinates of the integer sum of those rows. Counting these sums instead of the row subsets gives a bound that depends only on the rank and not on the number of rows. As a result, the final sample can be small.

Theorem 9.3 (Counting the sets of columns)**.** *Let r, t be nonnegative integers. If a Boolean matrix F has real rank at most r, then, over all depth-at-most-t trees P specified as in Definition 9.1 and all retained row subsets U, there are at most*

$$2^{(r+1)\lceil \log(r+2) \rceil\, 2^{t+1}} \tag{3}$$

distinct sets $G_P(U)$. Repeated row and column labels are allowed.

Proof. We first count, for each leaf, the sets of columns on which its output is correct for every row whose path reaches that leaf. We then count how a tree can combine those sets.

We stack the rows of F and $\mathbf{1} - F$ into one Boolean matrix A. Its row space lies in the span of the rows of F and the all-one vector, so its rank is at most $r + 1$. By the row-sum bound of [20, Theorem 4.2, with $d = 1$], a Boolean matrix of real rank at most $r + 1$ has at most $(r + 2)^{r+1}$ distinct common-zero column sets over all row subsets. To apply that bound, we view the matrix as a bipartite adjacency matrix: its common-zero sets are the complements of unions of neighborhoods. The count includes the empty row subset.

These sets cover both leaf outputs. A leaf with output 0 is correct where the selected rows of F all vanish, and a leaf with output 1 is correct where the selected rows of $\mathbf{1} - F$ all vanish. Write $a = (r + 1)\lceil \log(r + 2) \rceil \geq 1$, so there are at most 2^a leaf conditions. At an Alice node the child conditions intersect, while at a Bob node they unite. The rows reaching a leaf are the retained rows consistent with Alice's messages on its path, so the count includes every retained row set and every choice of Alice's maps. After padding to depth t, the count is at most

$$2^{2^t - 1}(2^a)^{2^t} \leq 2^{a2^{t+1}}.$$

The argument also covers rank zero and repeated row or column labels. □

9.2 Sampling columns and then rows

We sample the same number m of rows and columns, and we take m to be a power of two, so the output has the square power-of-two form of Corollary 1.4. In the proof, we first show that, with high probability, the sampled columns preserve the assumed lower bound H on every row set of original weight $\geq 3/5$. We then analyze the independent row sample.

Theorem 9.4 (Sampling rows and columns)**.** *Let $F : X \times Y \to \{0,1\}$ have real rank at most r, where $r \geq 0$ is an integer, and let μ_X, μ_Y be input distributions on nonempty finite indexed sets. Suppose an integer $H \geq 1$ satisfies, for all $U \subseteq X$ and $V \subseteq Y$,*

$$\mu_X(U) \geq 3/5, \quad \mu_Y(V) \geq 3/5 \quad \Longrightarrow \quad \mathrm{D}(F[U,V]) \geq H.$$

Put

$$m = \mathrm{pow}_2\big(16(r+1)\,\lceil \log(r+2) \rceil\; 2^H\big),$$

where $\mathrm{pow}_2(u)$ is the least power of two at least u. Draw m rows and m columns independently, retaining separate indices for repeated draws. The resulting $m \times m$ matrix has cost at least H except with probability less than $1/100$. Every protocol for F also computes the sampled matrix, so the sampled matrix has cost at most $\mathrm{D}(F)$ on every sample outcome.

Proof. Repeated labels can always be decoded locally. Conversely, a protocol on repeated labels can be restricted to one occurrence of each label.

For every U with $\mu_X(U) \geq 3/5$ and every depth-$(H-1)$ tree P specified as in Definition 9.1, we have $\mu_Y(G_P(U)) < 3/5$: otherwise Observation 9.2 with $V = G_P(U)$ gives $\mathrm{D}(F[U,V]) \leq H-1$, contradicting the restriction hypothesis. As U and P vary, these sets form a family of size at most $2^{m/16}$ by Theorem 9.3. By a union bound, the probability that all sampled columns lie in one of them is at most $2^{m/16}(3/5)^m$. Outside this event, the sampled-column matrix $F_Y(x,j) = F(x,y_j)$ has cost $\geq H$ on every original row set of weight $\geq 3/5$.

We condition on such a column outcome. The transpose F_Y^{T} still has rank at most r. Consider the sets $G_P(U)$ for this transpose, where $U = [m]$ is its full row set and P has depth $H-1$. Each has $\mu_X(G_P(U)) < 3/5$: otherwise Observation 9.2 and transposition give $\mathrm{D}(F_Y[G_P(U),[m]]) \leq H-1$, contradicting the preceding guarantee. There are again at most $2^{m/16}$ such sets. The row draws remain independent with distribution μ_X, so the probability that they all lie in one of these sets is at most $2^{m/16}(3/5)^m$. A protocol of depth at most $H-1$ on the final matrix would give such a containment. Since $m \geq 32$, the total failure probability is at most

$$2 \cdot 2^{m/16}(3/5)^m \leq 2^{1-7m/16} \leq 2^{-13} < 1/100.$$

The universal upper bound follows by decoding sampled indices and running a protocol for F. □

We sample only once, after all amplification rounds on full products. Theorem 9.4 bounds the failure probability but gives no test certifying that a particular sample succeeds.

9.3 Printing the sample and comparing errors

We generate the sampled input labels and evaluate only the corresponding entries of R_k.

We take a source graph on n vertices above the fixed effective cutoff in Lemma F.2, use the parameter choices of that lemma at every round, and write

$$L = \lceil \log n \rceil, \qquad B_k = d_0 + kh, \qquad H_k = B_k + k, \qquad r_k = 2^{B_k - 1}$$

as in Proposition 5.5, which gives $\operatorname{rank}(R_k) \le r_k$. The reduction never computes a rank. A *random tape* is the sequence of random bits used by the algorithm. The next lemma bounds the running time on every tape, not only on likely ones.

Lemma 9.5. *For the level-k iteration from the initial matrix in Proposition 5.5, use r_k, H_k in Theorem 9.4. A joint sampler and entry evaluator use $n^{O(k+1)}$ time and random bits on every tape, with an absolute implied constant independent of k. Every output label is an actual input of R_k, and the sample counts and tape length are fixed. The joint distribution of the generated labels is within total variation less than $1/100$ of the ideal independent sample of Theorem 9.4.*

Here total variation is the largest difference between the probabilities that two distributions assign to an event. The proof in Section E gives the recursive construction with a fixed number of random bits. Sampling is approximate, but every printed entry is exact. Combining the lemma with Theorem 9.4, we obtain the reduction.

Theorem 9.6 (Randomized reduction)**.** *One randomized algorithm, given a source graph on n vertices above the fixed cutoff in Lemma F.2 and an integer $k \ge 1$, outputs a total Boolean matrix M and an integer B. The table is square with power-of-two side length. On every tape the algorithm uses $n^{O(k+1)}$ time and random bits, with an absolute implied constant independent of k. For a YES source, $\mathrm{D}(M) \le B$ on every tape. For a NO source,*

$$\Pr[\mathrm{D}(M) = B + k] \ge 49/50.$$

Here $B = B_k$ and the tape-independent entry count is

$$m = \operatorname{pow}_2\big(16(r_k + 1)\,\lceil \log(r_k + 2) \rceil\; 2^{H_k}\big), \qquad T = m^2.$$

Proof. The algorithm computes the parameters of Lemma F.2 and the initial matrix of Lemma 5.4, then B_k, H_k, r_k, m. It generates m columns and m rows with Lemma 9.5, evaluates all m^2 row–column pairs (keeping repeated labels as distinct indices), and outputs (M, B_k). The time and random-bit bounds follow from Lemma 9.5, and the output is $m \times m$ with m a power of two. On a YES source, every generated label is an input of R_k. Decoding the labels and running a protocol for R_k therefore shows that every sample has cost at most B_k.

On a NO source, Proposition 5.5 gives

$$\mathrm{D}(R_k) = H_k, \qquad \mathcal{P}(R_k; 2^{-L}, H_k), \qquad \operatorname{rank}(R_k) \le r_k.$$

Because $2^{-L} \le 1/2$, the $j = 0$ lower bound applies whenever both retained weights are at least $3/5$. Thus Theorem 9.4 gives ideal sampling failure less than $1/100$. By Lemma 9.5, using the implemented sampler instead of the ideal independent sample adds less than $1/100$ to this failure probability. Therefore, except with probability less than $1/50$, the output has cost $\ge H_k$. Decoding the inputs gives cost $\le H_k$ on every tape, so the cost is then exactly $B_k + k$. □

9.4 From an approximation algorithm to a 3-SAT algorithm

We now combine the two constructions with an estimator, proving the general implication stated in Theorem 9.7. Each construction produces a known threshold B and a gap k. Its size bound determines which approximation errors are small enough to distinguish the two cases.

Theorem 9.7 (From approximation algorithms to 3-SAT algorithms)**.** *There is an effective absolute integer $\mathsf{A} \ge 2$ and, for each fixed clause-density bound c_s, an effective integer $C \ge 1$ with the*

following property. Suppose a deterministic or bounded-error randomized polynomial-time estimator approximates $\mathrm{D}(M)$ *with additive error at most a nondecreasing function* $a(T)$ *on every sufficiently large explicit total Boolean table with* T *entries. Let* $k = k(N) \geq 1$ *be an integer function computable in time polynomial in* $N + k(N)$*. Choose either construction and its size bound:*

Construction	$S(N,k)$
Randomized	$(N+2)^{C(k+1)}$
Deterministic	$(N+2)^{C\mathsf{A}^k}$

If, for all sufficiently large N*,*

$$2a\big(S(N,k(N))\big) < k(N),$$

then 3-SAT on N *variables and at most* $c_s N$ *clauses has an algorithm running in time* $S(N,k(N))^{O(1)}$*. It is deterministic when both the construction and the estimator are deterministic, and bounded-error randomized otherwise. The exponent may depend on the estimator and the algorithm computing* k*. It suffices for the estimator to work on square tables whose side length is a power of two.*

Proof of Theorem 9.7. Apply Theorem 5.2, then Theorem 9.6 in the randomized case or Theorem 8.4 in the deterministic case. Since the source graph and the deterministic size parameter are $N^{O(1)}$, a fixed C bounds both the table size T and construction time by the chosen $S(N,k)$. The constant A is the absolute constant from Theorem 8.4. The tables tend to infinity with N.

Because a is nondecreasing, $a(T) \leq a(S(N,k)) < k/2$. An accurate estimate is therefore below the midpoint $B + k/2$ on a YES output and above it on a successful NO output. If the estimator is randomized, repeat this comparison independently a constant number of times and take the majority answer, reducing its error below $1/100$. The randomized construction adds failure probability at most $1/50$; the deterministic construction adds none. Thus the combined algorithm has error less than $1/3$ whenever randomness is used, and is exact otherwise.

Construction, computing k, estimation, and comparison with the binary rational midpoint all take time $S(N,k)^{O(1)}$: in particular, $S(N,k) \geq N + k$ absorbs the assumed polynomial cost of computing k. Exhaustive search handles the finitely many smaller sizes. □

To read this in terms of a proposed 3-SAT running time $R(N) = 2^{g(N)}$, suppose $g(N)/\log(N+2)$ grows without bound and the chosen number of rounds can be computed within the stated time. A budget $2^{O(g(N))}$ allows

$$k = \Theta\left(\frac{g(N)}{\log(N+2)}\right) \qquad \text{in the randomized construction,}$$
$$k = \Theta\left(\log \frac{g(N)}{\log(N+2)}\right) \quad \text{in the deterministic construction.}$$

The constant in the second choice is chosen so that $\mathsf{A}^k \log(N+2) = O(g(N))$. Accuracy better than half this gap, at the corresponding table sizes, therefore yields a 3-SAT algorithm within that budget. If such a 3-SAT algorithm is excluded by an explicit hypothesis, the same statement gives inapproximability. To quantify a further improvement over a known algorithm, choose a smaller target budget R. The theorem does not treat the best known upper bound as optimal.

For Corollary 1.4, we use the exact sample size rather than the coarser bound $S(N,k)$. This gives the coefficient $1/8$ in the randomized error bound; the algorithmic implication is the same.

Proof of Corollary 1.4. Suppose a bounded-error randomized polynomial-time estimator achieves additive error $a(T) = c \log T / \log\log T$ for some fixed $0 \leq c < 1/8$ on every sufficiently large square

table of power-of-two side length. Fix $8c < \alpha < 1$. We obtain a bounded-error $2^{O(N^\alpha)}$ algorithm for sparse 3-SAT.

For a formula with N variables, Theorem 5.2 produces a source graph with n vertices, where

$$2N/\Delta \le n \le N(\log(N+2))^p$$

for the fixed integer p of that theorem. The lower bound follows from $N \le |E| \le \Delta n/2$, also given there. Thus the graph is above the fixed cutoff for every sufficiently large N, and

$$L = \lceil \log n \rceil = \log N + O(\log \log N).$$

Choose a fixed rational β with $8c < \beta < \alpha$, and put $k = \lfloor N^\beta / \lceil \log N \rceil \rfloor$. This integer is computable in polynomial time, using an integer root for the fixed rational power. By Lemmas 5.4 and F.2, $h = L + O(\log L)$ and $d_0 = O(\log L)$, so

$$B_k = d_0 + kh = (1 + o(1))k \log N.$$

Substituting $r_k = 2^{B_k - 1}$ and $H_k = B_k + k$ into Theorem 9.6 gives $\log m = 2B_k + k + O(\log(B_k + 1))$. Hence the printed size satisfies

$$\log T = 2 \log m = 4B_k + 2k + O(\log(B_k + 1)) = (4 + o(1))k \log N, \tag{4}$$

$$\log \log T = (\beta + o(1)) \log N, \qquad k = \left(\frac{\beta}{4} + o(1)\right) \frac{\log T}{\log \log T}. \tag{5}$$

Consequently $k > 2a(T)$ for all sufficiently large N. The table already has the square power-of-two form required by the assumed estimator.

For each estimator run, we test whether its output is below the rational midpoint $B_k + k/2$. An accurate estimate answers this test correctly on every YES output and every successful NO output. We repeat the run independently a constant number of times and take the majority answer, which reduces the decision error on each such table below $1/100$. This uses only comparison with a binary rational, which the estimator model supports (Definition 4.1). The reduction fails with probability at most $1/50$, so the decision error is less than $3/100 < 1/3$.

Both construction time and random-bit usage are $n^{O(k+1)} = 2^{O(N^\beta)}$ on every tape. The estimator also takes $2^{O(N^\beta)}$ time because $\log T = O(N^\beta)$. For all sufficiently large N, we have $k \ge 1$, the graph is above the fixed cutoff, and T meets the estimator's size threshold. The finitely many smaller values of N are decided by exhaustive search within a fixed bound. Since $\beta < \alpha$, the total running time is $2^{O(N^\alpha)}$, as claimed. □

10 Strictly alternating protocols

We now prove the result that settles Yao's original question. If we simulate the multi-bit messages of the unrestricted construction by inserting the missing turns, the cost can double and the additive gap is lost. We therefore use a separate matrix designed for strict alternation. The end of Section 2 sketches the argument. A balanced hypergraph coloring gives a short alternating protocol (Proposition 10.4). In the NO case, a short protocol induces a coloring of the vertices by Alice's early response rules. A monochromatic edge then gives many array positions on which her remaining messages produce too few answer patterns (Lemma 10.5). With an explicit array list (Corollary 10.6), the table has polynomial size for each fixed gap, and no repeated amplification is needed.

10.1 Starter, output and matrix inputs

Changing who starts or who must know the answer changes the cost by a constant, so the gap transfers between the conventions of Definition 4.2.

Lemma 10.1. *On nonempty input sets, each cost $C \in \{D_A^{\mathrm{alt}}, D_B^{\mathrm{alt}}, D_{\mathrm{free}}^{\mathrm{alt}}, Y_{\mathrm{free}}\}$ satisfies*

$$|C(M) - D_A^{\mathrm{alt}}(M)| \le 2.$$

Proof. Prepending one fixed bit sent by the new starter changes the starter, so $|D_A^{\mathrm{alt}} - D_B^{\mathrm{alt}}| \le 1$ and

$$D_A^{\mathrm{alt}} - 1 \le D_{\mathrm{free}}^{\mathrm{alt}} \le D_A^{\mathrm{alt}}.$$

Public output also gives receiver output. Conversely, at a nonzero-depth terminal transcript the receiver is the next scheduled speaker and can announce the answer in one bit. Public stopping tells both players when to make this announcement. At depth zero, choose the designated output player as the starter and let that player announce the answer. Thus

$$Y_{\mathrm{free}} \le D_{\mathrm{free}}^{\mathrm{alt}} \le Y_{\mathrm{free}} + 1,$$

and the claimed two-bit bound follows. These transformations preserve public stopping and use an input-independent starter. □

A hypergraph coloring is proper if no edge is monochromatic. In the YES case the hypergraph has a balanced coloring with few colors. In the NO case it has no proper coloring even with the larger number of colors that a short protocol would induce. We use the balanced conversion of Guruswami and Lee [8], in a form whose edges have distinct vertices.

Proposition 10.2 (Hypergraph coloring promise)**.** *Fix an integer $t \ge 4$, and set*

$$s_{\mathrm{a}} = 2^t, \qquad K_{\mathrm{a}} = s_{\mathrm{a}}^2, \qquad S_{\mathrm{a}} = 2^{s_{\mathrm{a}}}.$$

There is a deterministic polynomial-time reduction from 3-SAT to K_{a}-uniform hypergraphs with nonempty vertex sets and distinct vertices in every edge such that:

1. *a satisfiable formula has an s_{a}-coloring with at least $s_{\mathrm{a}} - 1$ vertices of every color in every edge;*

2. *an unsatisfiable formula has no proper S_{a}-coloring.*

The polynomial may depend on t.

The proof in Section G.2 derives this promise from the cited result and ensures that every edge has distinct vertices.

Alice holds a vertex and a tuple of addresses. Bob holds an array, a coordinate selecting one of Alice's addresses, and an edge index. If the vertex is outside the selected edge, or the edge index is unused, the answer is zero.

Definition 10.3. Let $\mathcal{H} = (V, E)$ be a source hypergraph with an ordering of the vertices in each edge. Set

$$q_{\mathrm{a}} = 2^{t-1} = s_{\mathrm{a}}/2, \qquad H_{\mathrm{a}} = 2^b,$$

where H_{a} is the least power of two at least $\max\{1, |E|\}$. Index the edges by distinct $\alpha \in [H_{\mathrm{a}}]$, and let $\iota_\alpha(v)$ be the position of v in that edge, or zero if v is absent or the index is unused. Alice's address tuples are all of

$$\mathcal{C} = [H_{\mathrm{a}}]^{q_{\mathrm{a}}},$$

each used once. For any labeled Boolean array list $\mathcal{W} = (w^\ell)$ on

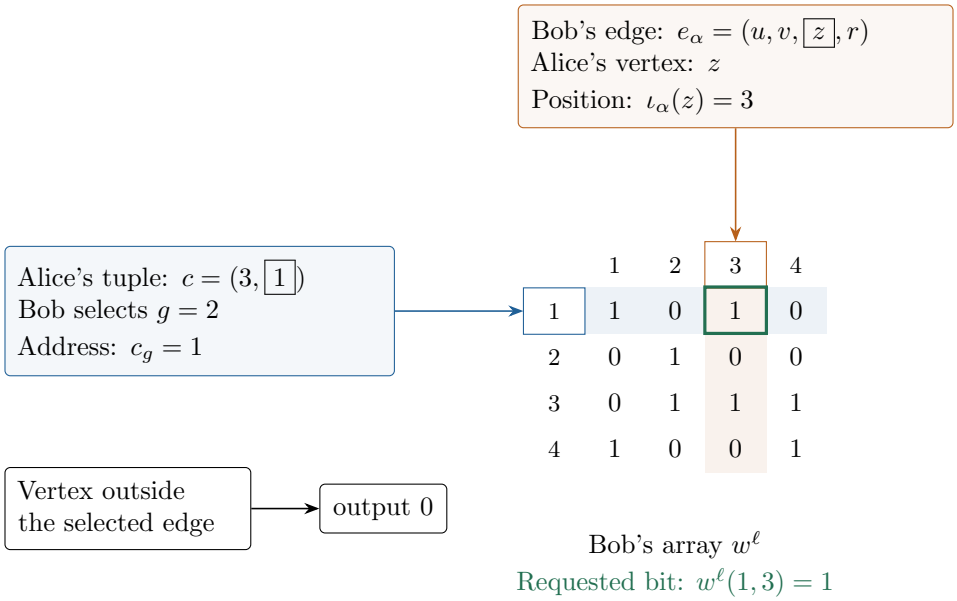
$$\Omega_{\mathrm{a}} = [H_{\mathrm{a}}] \times [K_{\mathrm{a}}],$$

define

$$M_{\mathrm{a}}((v,c),(g,\alpha,\ell)) := \begin{cases} w^\ell(c_g, \iota_\alpha(v)), & \iota_\alpha(v) > 0, \\ 0, & \iota_\alpha(v) = 0. \end{cases}$$

Rows are indexed by $(v,c) \in V \times \mathcal{C}$. Column labels lie in $[q_{\mathrm{a}}] \times [H_{\mathrm{a}}] \times [|\mathcal{W}|]$. Equal arrays retain their separate labels.

In Figure 5, g selects an address from Alice's tuple, which gives the row of Bob's array, and the position of her vertex in Bob's edge gives the column.



	1	2	3	4
1	1	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	1

Figure 5: The address selects a row and the vertex's edge position selects a column, giving $w^\ell(1,3) = 1$. A vertex outside the edge gives zero without reading the array. The indices are illustrative, and the arrows show how the entry is selected, not a protocol's messages.

10.2 The protocol given by a coloring

Once Bob has sent the edge, a balanced coloring lets Alice name her position among the few positions of her color. The schedule in the proof of Proposition 10.4 exchanges color and coordinate, then edge and address, then this position, and costs

$$B_{\mathrm{a}} := (2t-1) + 2b + 2(t+1) + 1 = 2b + 4t + 2.$$

Proposition 10.4. *For the table of Definition 10.3, an s_{a}-coloring with at least $s_{\mathrm{a}} - 1$ vertices of every color in every edge gives*

$$D_A^{\mathrm{alt}}(M_{\mathrm{a}}) \le B_{\mathrm{a}}.$$

Proof. Fix a row (v, c), a column (g, α, ℓ) and a balanced coloring χ. An edge contains at least $s_{\mathrm{a}} - 1$ vertices of each of the other $s_{\mathrm{a}} - 1$ colors. Consequently the vertices whose color is $\chi(v)$ occupy at most

$$K_{\mathrm{a}} - (s_{\mathrm{a}} - 1)^2 = 2s_{\mathrm{a}} - 1$$

positions. We add one message code for "outside the edge." There are then at most $2s_{\mathrm{a}} = 2^{t+1}$ possible message values, encoded in $t + 1$ bits. We fix their ordering publicly for every color and edge index. An unused edge index has only the value for "outside the edge." Alice determines her message value from v, α and $\chi(v)$.

First Alice sends the t bits of $\chi(v)$, while Bob sends the $t - 1$ bits of g between consecutive Alice bits. This uses $2t - 1$ bits and leaves Alice knowing g. Next Bob sends the b bits of α, while Alice sends the b bits of c_g, alternating. This uses $2b$ bits, after which Alice knows α and Bob knows c_g. Finally Bob sends $t + 1$ dummy bits and Alice sends the $t + 1$ bits encoding her message value, after which Bob sends one answer bit. He answers zero for "outside the edge" and otherwise reads the specified array entry. The three stages use B_{a} bits in total. □

10.3 Few response patterns cannot cover many array positions

Bob's early messages may depend on the whole array, so the transcript on one column does not determine how Alice behaves on other columns. To analyze an arbitrary protocol, we therefore also consider Bob histories that do not occur on a given column. For a fixed row, we record Alice's rth bit after every possible history of $r - 1$ Bob bits, including hypothetical histories. Her own preceding bits are then determined. For this count, we pad branches that stop early with fixed bits in the alternating order. For example, her first two turns require three response bits: the initial bit and one response to each possible first Bob bit. In general, her first t turns require $\sum_{r=1}^{t} 2^{r-1} = 2^t - 1$ response bits, giving at most 2^{2^t-1} possible response rules for these turns.

Among all Boolean arrays, the fraction agreeing with prescribed bits on k distinct positions is 2^{-k}. We need only an upper bound of $2^{-k} + \varepsilon$ on this fraction, and a small explicit list already satisfies this weaker bound. Here we count all array labels equally, keeping repeated arrays as separate labels. For a list $\mathcal{W} = (w^\ell)_{\ell \in \mathcal{L}}$ on a finite position set Ω, with $\mathcal{L}$ finite and nonempty, we require

$$\frac{|\{\ell \in \mathcal{L} : w^\ell|_I = z\}|}{|\mathcal{L}|} \le 2^{-|I|} + \varepsilon \qquad (I \subseteq \Omega,\ z \in \{0,1\}^I). \tag{6}$$

Adding these counts, at most a $P(2^{-|I|} + \varepsilon)$ fraction of all labels can match any of P patterns on I. Every fraction below is measured against the original list size $|\mathcal{L}|$, even after a transcript has removed some columns.

Lemma 10.5 rules out a short protocol when many requested positions share Alice's early response rules. The early messages cannot distinguish those positions, so many array labels remain after them. The remaining messages can produce only a few answer patterns, which cannot cover all the surviving labels. In the lemma, m is the total number of Alice turns, and t is the number of initial Alice turns during which her response rules agree across the rows for every possible Bob history.

Lemma 10.5 (Lower bound for alternating protocols)**.** *Let m, t be integers with $m \ge 2t \ge 2$. Suppose the nonempty labeled array list $\mathcal{W}$ satisfies Equation* (6) *with $\varepsilon \le 2^{-4m}$. If $J \subseteq \Omega$ has at least 2^{m-t+1} positions, then the matrix*

$$L_J(j, \ell) := w^\ell(j)$$

has no Alice-first alternating protocol of depth at most $2m - 1$ in which all rows give Alice the same response rules for her first t turns on every possible Bob history.

Proof. Suppose such a protocol exists. We pad every shorter branch to depth $2m-1$ with fixed alternating bits. At the final Alice node, her row and the transcript determine the old output leaf, so we replace her last bit by that output. This uses no extra bit and, since $m > t$, preserves her first t responses.

We stop with about $\log m$ turns per player remaining, so that Bob's remaining response rules require fewer bits to describe than there are surviving array positions. Put $h = \lceil \log m \rceil \le m - t$ and $Q = 2^h$. The inequality follows from $m \le 2(m-t) \le 2^{m-t}$. We follow the first $2(m-h)-1$ messages and keep the Bob child with more column labels and the Alice child with more rows. Alice's first t turns discard no rows because these response rules agree. Each of her remaining $m-h-t$ turns retains at least half the rows, and each of the $m-h-1$ Bob turns retains at least half the columns. The resulting rectangle $X \times Y$, with $Y \subseteq \mathcal{L}$, therefore satisfies

$$|X| \ge 2^{h+1} = 2Q, \qquad \frac{|Y|}{|\mathcal{L}|} \ge 2^{-(m-h-1)}.$$

Exactly h Bob–Alice pairs remain. We record Bob's next bit after every hypothetical history of earlier Alice bits, which gives $1 + 2 + \cdots + 2^{h-1} = Q - 1$ bits. Fixing these bits and Alice's response rules determines her final output on every retained row. On any $2Q$ of those rows, there are therefore at most 2^{Q-1} output patterns.

Since $m \le Q < 2m$, we have $\varepsilon \le 2^{-2Q}$. Every column in Y must match one of these patterns. Applying Equation (6) to each pattern and adding the bounds gives

$$\frac{|Y|}{|\mathcal{L}|} \le 2^{Q-1}(2^{-2Q} + \varepsilon) \le 2^{-Q}.$$

This contradicts the lower bound on the surviving fraction because $Q \ge m > m - h - 1$. □

Since Equation (6) holds for every set of positions, one array list works against every protocol. The bound applies to fractions of the original list; it need not hold within the subset left by a transcript. We use the accuracy $\epsilon_{\mathrm{a}} = 2^{-4(b+3t)} = 2^{-12t} H_{\mathrm{a}}^{-4}$, which is inverse polynomial in H_{a} for each fixed t. At this accuracy, the finite-field construction of Section H gives such a list with polynomially many arrays in H_{a}, and we use it in place of the set of all Boolean arrays in M_{a}.

Corollary 10.6. *For every fixed $t \ge 4$, a nonempty labeled array list on Ω_{a} satisfying Equation* (6) *with*

$$\epsilon_{\mathrm{a}} := 2^{-4(b+3t)}$$

can be generated in deterministic time polynomial in H_{a}.

The construction and proof are in Section H.

10.4 From a hypergraph to an explicit gap

Given a short protocol, we color each vertex by the most common response rules among its address tuples. A monochromatic edge then gives the many distinct array positions that Lemma 10.5 requires.

Proposition 10.7. *Use the complete address list $\mathcal{C}$ and the explicit array list of Corollary 10.6 in M_{a}. A balanced source coloring gives*

$$D_A^{\mathrm{alt}}(M_{\mathrm{a}}) \le B_{\mathrm{a}},$$

whereas the absence of a proper S_{a}-coloring gives

$$D_A^{\mathrm{alt}}(M_{\mathrm{a}}) \geq B_{\mathrm{a}} + 2t - 8.$$

For each fixed t, the full table can be printed in deterministic time polynomial in $|V| + H_{\mathrm{a}}$.

Proof. The upper bound is Proposition 10.4. For the lower bound, we put $m = b + 3t - 3$, so $2m - 1 = B_{\mathrm{a}} + 2t - 9$, and suppose that an Alice-first protocol has depth at most $2m - 1$. We pad it to that depth and, for each vertex v, partition its address tuples by Alice's response rules on her first t turns. By the count of response bits above, there are at most $2^{2^t-1} \leq S_{\mathrm{a}}$ choices of these rules. A largest class C_v therefore has

$$|C_v| \geq H_{\mathrm{a}}^{q_{\mathrm{a}}}/S_{\mathrm{a}}.$$

We color v by this class's response rules. Since the hypergraph has no proper S_{a}-coloring, some edge e_α is monochromatic, that is, all its vertices receive the same response rules.

For $v \in e_\alpha$ and $g \in [q_{\mathrm{a}}]$, let $J_{v,g} = \{c_g : c \in C_v\}$ be the set of addresses occurring in coordinate g. Since C_v is contained in the product of these projections,

$$\prod_{g=1}^{q_{\mathrm{a}}} |J_{v,g}| \geq |C_v| \geq H_{\mathrm{a}}^{q_{\mathrm{a}}}/S_{\mathrm{a}}.$$

A column (g, α, ℓ) reads only coordinate g of Alice's tuple, so we use a single coordinate g for all vertices of the edge. AM–GM, summed over the vertices of e_α, gives a coordinate g with

$$\begin{aligned} N := \sum_{v \in e_\alpha} |J_{v,g}| &\geq K_{\mathrm{a}} H_{\mathrm{a}} S_{\mathrm{a}}^{-1/q_{\mathrm{a}}} \\ &= \frac{K_{\mathrm{a}} H_{\mathrm{a}}}{4} = 2^{m-t+1}, \end{aligned}$$

where $S_{\mathrm{a}} = 4^{q_{\mathrm{a}}}$. For each $v \in e_\alpha$ and $u \in J_{v,g}$, we choose a tuple $c(v,u) \in C_v$ with $c(v,u)_g = u$. We retain the corresponding rows and all columns (g, α, ℓ). Their entries are

$$M_{\mathrm{a}}((v, c(v,u)), (g, \alpha, \ell)) = w^\ell(u, \iota_\alpha(v)).$$

These are N distinct positions, because different vertices occupy different positions in the edge and different addresses give different first coordinates. All retained rows give Alice the same response rules for her first t turns. Since $m = b + 3t - 3 \geq 2t$ and $\epsilon_{\mathrm{a}} \leq 2^{-4m}$, the restriction to these rows and columns contradicts Lemma 10.5. Integer depths give $D_A^{\mathrm{alt}}(M_{\mathrm{a}}) \geq 2m = B_{\mathrm{a}} + 2t - 8$.

There are $|V| H_{\mathrm{a}}^{q_{\mathrm{a}}}$ rows and $q_{\mathrm{a}} H_{\mathrm{a}} |\mathcal{W}|$ columns. For fixed t, the number q_{a} is constant and Corollary 10.6 constructs the arrays in polynomial time. Enumerating all tuples and evaluating the displayed entry rule therefore prints the entire table in polynomial time. □

To finish, we choose t so that the gap $2t - 8$ of Proposition 10.7 is at least the requested gap. We then transfer the resulting hardness to the other starter and output conventions with Lemma 10.1.

Proof of Theorem 1.2 and Corollary 1.3. Given a fixed requested gap $k \geq 1$, we choose $t = \lceil k/2 \rceil + 4$. Then $2t - 8 = 2\lceil k/2 \rceil \geq k$. Composing the reduction of Proposition 10.2 with Proposition 10.7, we obtain, in deterministic polynomial time, a table whose cost D_A^{alt} is at most B_{a} in the YES case and at least $B_{\mathrm{a}} + k$ in the NO case, proving Theorem 1.2.

An additive-u estimate of any cost in Lemma 10.1 is, by the triangle inequality, an additive-$(u+2)$ estimate of D_A^{alt} with the same success probability. We choose any fixed integer $k > 2(u + 2)$ and compare the estimate with $B_{\mathrm{a}} + k/2$. This gives $\mathsf{P} = \mathsf{NP}$ for a deterministic estimator and $\mathsf{NP} \subseteq \mathsf{BPP}$ for a bounded-error randomized estimator, proving Corollary 1.3. □

The address length $q_a = 2^{t-1}$ and the degree of the source reduction may both grow with t. Hence the construction is polynomial only for each fixed gap, and this argument gives no growing-error result for strict alternation.

11 Remaining approximation questions

We obtained fixed-error NP-hardness in both models and algorithmic implications for growing errors in the unrestricted model. The quantitative consequences depend on how fast the explicit table grows with the requested gap. Smaller tables would give faster SAT algorithms from the same approximation guarantee, and stronger inapproximability under assumptions excluding those SAT algorithms.

For unrestricted protocols, our deterministic construction outputs a table of size at most $N^{2^{O(k)}}$ for an N-variable sparse formula and gap k (Theorem 8.4). Can smaller lists preserve the restriction bounds needed for the next round, or can another construction improve the $\log \log T$ error scale?

For strict alternation, a construction whose size and running time depend more mildly on the gap could let the gap grow with the table size, giving analogous approximation-to-SAT implications in Yao's model.

A Interlacing estimates and the initial matrix

We first prove the projection and small-block estimates used in Section C.3. The surviving-block bound proved there then supplies the separation theorem and the column-loss induction below. We use these estimates to prove Lemma 6.5 and Theorem 6.17, and finish with the initial matrix and template-layer bounds (Lemmas 5.4 and 6.6).

A.1 Projecting column tuples and retained weight

After a protocol prefix, a restriction of an interlace may retain many blocks but only a small fraction of its column weight. We select a few blocks whose projected tuples still have enough weight for a lower bound. We measure the loss by the logarithm Δ_p below, which increases by one each time the retained column weight is halved.

Fix a Boolean matrix $M : X \times Y \to \{0, 1\}$ with positive normalized input weights μ and ν. Every weight below uses these original weights or their products. For a nonempty $K \subseteq Y^p$, define $\Delta_p(K) = \log(1/\nu^p(K))$. The projection onto no coordinates has loss $\Delta_0 = 0$, since Y^0 is a single point of weight one.

We show that projection cannot increase the loss, and that projecting onto a suitable set of l of the p coordinates leaves at most an l/p fraction of it.

Lemma A.1. *Let $K \subseteq Y^p$ be nonempty, and write $\pi_J K$ for its projection onto the coordinates in $J \subseteq [p]$. Then*

$$\Delta_{|J|}(\pi_J K) \le \Delta_p(K),$$
$$\Delta_{|J_0|}(\pi_{J_0} K) + \Delta_{|J_1|}(\pi_{J_1} K) \le \Delta_p(K) \qquad (J_0 \sqcup J_1 = [p]).$$

For every integer $1 \le l \le p$, some l-element set J satisfies

$$\Delta_l(\pi_J K) \le \frac{l}{p} \Delta_p(K).$$

Viewing K as a column set of $\mathsf{I}_p(M)$, the interlace on the blocks in J, restricted to the columns $\pi_J K$ and to any retained rows of these blocks, is a restriction of $\mathsf{I}_p(M)$ with columns in K and the same block row weights (Observation 4.5).

Proof. The first two inequalities follow from $K \subseteq \pi_J^{-1}(\pi_J K)$ and $K \subseteq \pi_{J_0} K \times \pi_{J_1} K$, using the product weights.

For the averaged bound, we interpret the fixed weights as a probability distribution and let Z have distribution $Q = \nu^p(\cdot \mid K)$. For distributions P and positive P' on a finite set, their relative entropy is $D_{\mathrm{KL}}(P\|P') = \sum_z P(z)\log(P(z)/P'(z))$, and entropy is $\mathsf{H}(P) = -\sum_z P(z)\log P(z)$, where zero terms are omitted. Conditioning on K gives

$$D_{\mathrm{KL}}(Q\|\nu^p) = \Delta_p(K).$$

Since Q_J is supported on $\pi_J K$, nonnegativity of relative entropy gives

$$\Delta_{|J|}(\pi_J K) \le D_{\mathrm{KL}}(Q_J\|\nu^J).$$

For a uniformly chosen l-subset J, the entropy chain rule gives

$$\mathbb{E}_J \mathsf{H}(Z_J) \ge \mathbb{E}_J \sum_{i\in J} \mathsf{H}(Z_i \mid Z_1,\dots,Z_{i-1}) = \frac{l}{p}\mathsf{H}(Z).$$

Also, since ν^J is a product, each coordinate contributes to $-\mathbb{E}\log\nu^J(Z_J)$ exactly when it lies in J, which happens with probability l/p. Subtracting the entropy inequality therefore yields

$$\mathbb{E}_J \Delta_l(\pi_J K) \le \mathbb{E}_J D_{\mathrm{KL}}(Q_J\|\nu^J) \le \frac{l}{p} D_{\mathrm{KL}}(Q\|\nu^p) = \frac{l}{p}\Delta_p(K).$$

Hence some J satisfies the desired bound. The conditional distribution Q need not be a product, so we compute all projection weights with the original product weights. The last assertion is Observation 4.5. □

A.2 Two or three surviving blocks

Later arguments, such as the proof of Lemma 6.11, reduce a large collection of surviving copies to two or three. We therefore first prove lower bounds for these small collections. Each proof splits into cases according to the first speaker and uses both column-weight thresholds of Definition 5.1.

Lemma A.2 (Small block bounds)**.** *Suppose $\mathcal{P}(M; x, H)$ with $H \ge 2$. Restrictions of a full-product interlace of M with the following numbers of blocks, row weights and product column weights have at least the indicated costs whenever the displayed row threshold is ≤ 1. Here we measure row weight separately in each block.*

blocks	*row weight per block*	*product column weight*	*cost*
2	$2x$	$9/50$	H
2	$2x$	$9/25$	$H+1$
3	$4x$	$9/50$	$H+1$

Proof. For the first row, Lemma A.1 gives a singleton column projection of weight $\ge \sqrt{9/50} > 3/10$. Applying $\mathcal{P}(M; x, H)$ with $j = 1$ to that block gives cost $\ge H - 1 \ge 1$, so the restriction is nonconstant. If Alice speaks first, we follow that block into a child retaining base row weight $\ge x$.

The retained columns are unchanged, so the suffix costs at least $H-1$. If Bob speaks first, his denser child has column weight $\geq 9/100$ and hence a singleton projection of weight $\geq 3/10$. The suffix at that child again costs at least $H-1$. Including the first bit gives cost at least H.

For the second row, the first row already gives nonconstancy. If Alice speaks, some singleton projection has column weight $\geq 3/5$, and a child retaining half the row weight of that block costs at least H. If Bob speaks, his denser child has column weight $\geq 9/50$, so the first row gives suffix cost at least H. Including the first bit gives cost at least $H+1$.

For the third row, projecting onto any two blocks and applying the first row shows that the restriction is not constant. If Alice speaks, we assign each block to a child retaining at least half its row weight. One child receives two blocks of weight $\geq 2x$, and the first row gives suffix cost H. If Bob speaks, his denser child retains column weight $\geq 9/100$. The averaged projection inequality gives two of its three coordinates with product weight at least

$$(9/100)^{2/3} > 9/50.$$

Their row weights are still $\geq 4x \geq 2x$, so the first row again gives suffix cost H. Including the first bit proves the claim. By Observation 4.5, each projection is a restriction of the original matrix. □

The next theorem shows that a short protocol for a width-u interlace must begin with $\log u$ Alice bits, and that each resulting rectangle keeps most of the row weight of one block and little of every other block. Its proof uses Lemma 6.11, whose proof in Section C does not use this theorem.

Theorem A.3. *Suppose $\mathcal{P}(M;\rho,d)$ with $d \geq 2$. Let $u \geq 2$ be dyadic and $1 \geq \xi > 2u\rho$. Interlace M at width u using an exactly eight-wise independent tuple list with the base-column marginal, as in Definition 4.4. Every restriction retaining base row weight at least ξ in every block and tuple-label weight at least $18/25$ has the following property: every protocol of depth at most $d+\log u$ for it begins with $\log u$ Alice bits along every path. The u nodes at depth $\log u$ are in bijection with the blocks: each such node retains more than $\xi - 2(u-1)\rho$ of its assigned block and less than 2ρ of every other block. Full products are a special case, and fixed bits count as transmitted bits.*

Proof. We maintain the following invariant at each protocol-tree node of depth $r < \log u$: there are $\kappa = u2^{-r}$ assigned blocks, each retaining base row weight $\geq \xi 2^{-r} > 2\kappa\rho$, and the retained column set is the original one. The root has this property. By Observation 4.5, the projection onto the coordinate of one assigned block is a restriction of M with column weight $\geq 18/25 > 3/5$, so $\mathcal{P}(M;\rho,d)$ gives cost at least $d \geq 2$ and the node is not a leaf. Since $1 \geq \xi > 2u\rho$, every row threshold $2^a\rho$ with $1 \leq a \leq \log u$ used below is less than one.

If Bob speaks, one child retains column weight $\geq 9/25$ and all κ assigned blocks. By Lemma 6.11 with $a = \log\kappa$, it needs $d+\log\kappa$ bits, but has only $d+\log u - r - 1 = d+\log\kappa - 1$ left. Thus Alice speaks. We assign each block to a child retaining at least half its weight. If a child received more than $\kappa/2$ blocks, it would contain at least $\kappa/2+1$ blocks of weight $\xi 2^{-r-1} > \kappa\rho$, and Lemma 6.11 with $a = \log\kappa$ would again require $d+\log\kappa$ bits, exceeding the $d+\log\kappa-1$ left. Both children therefore receive exactly $\kappa/2$ blocks. This proves the invariant, and hence that Alice sends the first $\log u$ bits. In particular, a fixed Alice bit is impossible, since it would assign every block to one child.

At depth $\log u$, each part has one assigned block of weight greater than 2ρ. If such a node retained two blocks of weight at least 2ρ, they would require $d+1$ bits by Lemma 6.11 with $a=1$, whereas only d remain. Every other block thus has weight less than 2ρ. Each original block is assigned to exactly one part, so its weight elsewhere totals less than $2(u-1)\rho$, proving the final bound. □

A.3 Induction on column loss

For a restriction of $\mathsf{I}_p(M)$ with surviving columns K, Bob's messages increase the loss $\Delta_p(K)$. The following induction gives lower bounds that tolerate this loss, and it uses only lower bounds on the base matrix.

Lemma A.4. *Let $C \geq 3$, $0 < \theta \leq 1/4$, and $0 < \delta \leq 1/64$. Suppose $\mathcal{P}(M; \theta, C-1)$ (Definition 5.1) and every restriction of M retaining row weight at least θ and column weight at least $1-\delta$ has cost at least C. Let $e \in \{0,1\}$, let $a \geq 2e$ be an integer with $2^a\theta \leq 1$, and let $p > 2^{a-1}$ be an integer. If a restriction G of $\mathsf{I}_p(M)$ retains base row weight at least $2^a\theta$ in every block and its nonempty column set K satisfies*

$$\Delta_p(K) \leq (p - 2^{a-1})\delta + \frac{4e}{3},$$

then $\mathrm{D}(G) \geq C + a - e$. Here C is only a lower bound, and we do not assume $\mathrm{D}(M) = C$.

Proof. Say that G is in state (a, e), or (a, e, p), when it satisfies the hypotheses for these values. We prove the lower bound by induction on

$$\mathfrak{m}(a, e) := 2a + 1 - e.$$

An Alice bit halves the row thresholds and lowers a. A Bob bit increases the loss by at most one. We cover this increase either by raising e to one, which adds $4/3$ to the allowed loss, or by projecting onto about half the blocks with Lemma A.1.

With $d = C - 1 \geq 2$ and $x = \theta$, the bound $\mathcal{P}(M; \theta, C-1)$ lets us apply Lemma 6.11 to full products, and $4\theta \leq 1$. At column weight $9/25$, its cases $a = 1, 2$ give cost at least C for two blocks of row weight 2θ, and at least $C + 1$ for three blocks of row weight 4θ. We call these the two-copy and three-copy bounds. Every projection below with loss $\leq 3\delta + 4/3$ has column weight $> 9/25$, because

$$3\delta + \frac{4}{3} \leq \frac{265}{192} < \frac{7}{5} < -\log(9/25).$$

The last inequality follows from $2^7 \cdot 9^5 < 25^5$.

We first verify the base states. For $(a, e) = (0, 0)$, the averaged projection bound in Lemma A.1 gives a singleton $J \subseteq [p]$ with

$$\Delta_1(\pi_J K) \leq \frac{p - 1/2}{p}\delta < \delta.$$

Since $2^{-v} \geq 1 - v$ for $0 \leq v \leq 1$, applied with $v = \Delta_1(\pi_J K)$, the projected column weight is greater than $1 - \delta$. The block in J retains row weight $\geq \theta$, so the hypothesis gives cost at least C.

For $(a, e) = (2, 1)$, the loss bound and Lemma A.1 give a three-coordinate projection with loss at most

$$\frac{3}{p}\left((p-2)\delta + \frac{4}{3}\right) \leq 3\delta + \frac{4}{3} < -\log(9/25).$$

Its column weight exceeds $9/25$, and each block retains weight $\geq 4\theta$. The three-copy bound above gives cost at least $C + 1 = C + 2 - 1$. There is one further base case, $(a, e, p) = (3, 1, 5)$. Its loss is $\leq \delta + 4/3 < -\log(9/25)$, and its rows have weight $\geq 8\theta$. Its column weight therefore exceeds $9/25$, and Lemma 6.11 with $a = 3$ applies to these five blocks, giving $C + 2 = C + 3 - 1$ bits.

We also handle the Bob-first case of $(a, e) = (1, 0)$ now; its Alice-first case is covered by the general step below. A singleton projection has loss $\leq (p-1)\delta/p < \delta$, so its column weight exceeds

$1-\delta$. Its rows retain weight $\geq 2\theta \geq \theta$, so the hypothesis gives cost at least $C \geq 3$ and the restriction is nonconstant. After a Bob bit, the denser child has a two-coordinate projection of loss at most

$$\frac{2}{p}\left((p-1)\delta+1\right) \leq 2\delta+1 < 3\delta+\frac{4}{3},$$

so the two-copy bound gives cost at least C after that bit. Every other valid state has $a \geq 2$, $p \geq 3$, and base row weight $\geq 4\theta$. Some three-coordinate projection has loss $\leq 3\delta + 4/3$, so by the three-copy bound the restriction is not constant.

Consider any remaining case and the first bit of a protocol for it. If Alice speaks, we assign every block to a child retaining at least half its row weight. For child $i \in \{0,1\}$, let p_i be the number of assigned blocks and let L_i be the loss after projecting the retained column set onto those blocks. Then $p_0 + p_1 = p$, and the partition inequality in Lemma A.1 gives $L_0 + L_1 \leq \Delta_p(K)$. Put

$$s_i = \max\{p_i - 2^{a-2}, 0\}.$$

Since $s_0 + s_1 \geq p - 2^{a-1} > 0$, at least one s_i is positive. The loss bounds required of the children with $s_i > 0$ satisfy

$$\begin{aligned}\sum_{i:s_i>0}\left(s_i\delta+\frac{4e}{3}\right) &\geq (p-2^{a-1})\delta+\frac{4e}{3}\\ &\geq \sum_{i:s_i>0} L_i.\end{aligned}$$

At least one eligible child contributes the constant $4e/3$, and the final inequality uses the assumed loss bound on K and $L_0 + L_1 \leq \Delta_p(K)$. Consequently some child with $s_i > 0$ has $L_i \leq s_i\delta + 4e/3$. Each assigned block retains weight $\geq 2^{a-1}\theta$, and there are $p_i > 2^{a-2}$ such blocks. This is state $(a-1, e)$, and the exceptional state $(a, e) = (2, 1)$ was already proved. The induction hypothesis gives cost at least $C + a - 1 - e$ in that child, and Alice's bit gives $C + a - e$. The same argument covers the step from state $(1, 0)$ to state $(0, 0)$.

If Bob speaks, one child has at least half the current column weight, so its loss is $\leq \Delta_p(K) + 1$. When $e = 0$ and $a \geq 2$, this child satisfies state $(a, 1)$, because

$$\Delta_p(K) + 1 \leq (p - 2^{a-1})\delta + \frac{4}{3}.$$

Its induction cost is at least $C + a - 1$, and adding Bob's bit proves the claim.

It remains to treat a Bob bit when $e = 1$ and $a \geq 3$. The extra term $4/3$ has then already been used, so we instead project the denser child onto about half the blocks and show that the projection is in state $(a-1, 1)$. The five-block base case already handles $p = 5$, so $p \geq 6$. We keep $p' = \lceil p/2 \rceil$ blocks. Then

$$\frac{1}{2} \leq \frac{p'}{p} \leq \frac{4}{7}, \qquad \frac{p'}{p}(p - 2^{a-1}) \leq p' - 2^{a-2}, \qquad \frac{4}{7}\left(\frac{4}{3}+1\right) = \frac{4}{3}.$$

By the averaged projection bound in Lemma A.1, some p' coordinates of the denser child therefore have loss at most

$$\frac{p'}{p}\left((p-2^{a-1})\delta+\frac{4}{3}+1\right) \leq (p'-2^{a-2})\delta+\frac{4}{3}.$$

The projected child retains base row weight $\geq 2^{a-1}\theta$ in each of its $p' > 2^{a-2}$ blocks, so it is state $(a-1, 1)$. Its induction cost is at least $C + a - 2$, and adding Bob's bit gives $C + a - 1$, as required. $\square$

We prove Lemma 6.5 by applying Lemma A.4 to $p = 9u/16$ blocks. The induction needs more than $u/2$ blocks, and the extra $u/16$ blocks give slack $(9u/16 - u/2)\delta = u/1024$ at $\delta = 1/64$ in the loss bound. The retained column weight may therefore be as small as $2^{-u/1024}$.

Proof of Lemma 6.5. Let $K \subseteq Y^q$ be the retained column set, where q is the width of the interlace. Since $u \geq 16$ is dyadic, $9u/16$ is an integer. We select a set J of exactly $p = 9u/16$ blocks retaining base row weight $\geq ux$ and project K onto them. The weight bound on K and Lemma A.1 give $\Delta_p(\pi_J K) \leq \Delta_q(K) \leq u/1024$, and the projected interlace is a restriction of the original one. Apply Lemma A.4 with

$$a = \log u, \quad \theta = x, \quad \delta = 1/64, \quad C = H, \quad e = 0.$$

Here $x \leq 1/16$, $p > 2^{a-1} = u/2$, and $2^a x = ux \leq 1$. The hypothesis $\mathcal{P}(M; x, H)$ implies the weaker bounds $\mathcal{P}(M; x, H-1)$ and, since $63/64 \geq 3/5$, cost $\geq H$ at column weight $63/64$. These are the two lower-bound hypotheses of Lemma A.4. Finally,

$$\Delta_p(\pi_J K) \leq u/1024 = (9u/16 - u/2)/64 = (p - 2^{a-1})\delta.$$

The induction gives cost at least $C + a = H + \log u$. □

We use the same induction to restore both deletion bounds at full cost for $R^+ = (\mathsf{I}_Q(W))^{\mathsf{T}}$.

Proof of Theorem 6.17. Put $C = \mathrm{D}(W)$ and $f = \lceil b/\delta \rceil$. The assumptions imply

$$f\delta \geq b, \qquad (f-1)/Q < 1/80, \qquad f < Q/40, \qquad Q \geq 5120.$$

Indeed, $b/\delta \geq 64$, $Q\delta \geq 80b$, and $f \leq b/\delta + 1 < 2b/\delta$. Fix $j \in \{0, 1\}$ and put

$$a_j = r - j, \qquad p_j = 2^{a_j - 1} + f, \qquad t_j = 2^{a_j}\theta.$$

These choices give $a_j \geq 0$, $2^{a_j-1} < p_j < Q$, and $t_j < 1$.

Consider a restriction of R^+ to which $\mathcal{P}(R^+; 2^{-b}, C + r)$ applies with this j, and transpose it to a restriction of $\mathsf{I}_Q(W)$. By Definition 5.1, the latter retains tuple weight $\geq 2^{-b}$ and total block-row weight $\geq (3/5)2^{-j}$. If fewer than p_j blocks had base row weight at least t_j, total row weight would be less than

$$\frac{p_j - 1}{Q} + t_j < 2^{-j-1} + \frac{1}{80} + \frac{1}{20 \cdot 2^j} \leq \frac{3}{5}\,2^{-j},$$

a contradiction, where the last inequality uses $j \leq 1$. We select p_j blocks J meeting the row-weight threshold. For the retained tuple set K, Lemma A.1 gives

$$\Delta_{p_j}(\pi_J K) \leq \Delta_Q(K) \leq b \leq f\delta = (p_j - 2^{a_j - 1})\delta.$$

The projected restriction therefore satisfies the hypotheses of Lemma A.4 with $e = 0$. Its two lower-bound hypotheses are those assumed for W, $\theta < 1/4$ follows from $Q\theta \leq 1/20$, and the row threshold is t_j. The lemma gives cost at least $C + a_j = C + r - j$, so Observation 4.5 and transposition prove $\mathcal{P}(R^+; 2^{-b}, C + r)$.

Sending the copy index and then running a protocol for W costs $r + \mathrm{D}(W)$ bits. This gives equality on the whole matrix, and it turns any upper bound on $\mathrm{D}(W)$ into an upper bound larger by r on $\mathrm{D}(R^+)$. By definition the transpose has n^Q rows and Qm columns. □

A.4 The initial matrix and template layers

For the initial matrix $R_0(z, j) = z_j$ on $\{0,1\}^{p_0} \times [p_0]$, we use rank. Even after column indices are deleted, the restricted matrix has large rank, so every protocol for it has many leaves of each output value.

Proof of Lemma 5.4. Since p_0 is a power of two, $a = \log p_0$ is an integer. Fix $j \in \{0,1\}$, a retained row set $V \subseteq \{0,1\}^{p_0}$ with $|V| \geq 2^{p_0 - L}$, and a retained column set $J \subseteq [p_0]$ with $|J| \geq \lceil (3/5)2^{-j} p_0 \rceil$. Under uniform weights, these are the restrictions tested by $\mathcal{P}(R_0; 2^{-L}, d_0)$. Let v be the dimension of the real span of V. A v-dimensional subspace has v coordinate functionals whose restrictions form a basis, so projection to those v coordinates is injective on the subspace. Since the Boolean cube has at most 2^v points in those coordinates, V has at most 2^v points. The lower bound on $|V|$ therefore implies $v \geq p_0 - L$.

Deleting the $p_0 - |J|$ coordinates outside J can lower the row rank by at most $p_0 - |J|$. Thus the restricted initial matrix has rank r satisfying

$$r \geq |J| - L \geq \frac{3}{5} 2^{-j} p_0 - L > 2^{a-j-1}.$$

The strict inequality follows from

$$\frac{3}{5} 2^{-j} p_0 - 2^{a-j-1} = \frac{p_0}{10 \cdot 2^j} \geq \frac{p_0}{40} > L,$$

where $p_0/40 - L > 0$ follows from $p_0 \geq 64(L+1)$.

Let F denote the restricted initial matrix. Consider a protocol for F. The indicator of each 1-leaf rectangle has rank at most one, and the sum of these indicators is the restricted matrix, so there are at least r one-leaves. Let $\mathbf{1}$ be the all-ones matrix, which has rank one. The complement of the restricted matrix therefore has rank at least $r - 1$, by $\operatorname{rank}(\mathbf{1} - F) \geq \operatorname{rank}(F) - \operatorname{rank}(\mathbf{1})$, so there are at least $r - 1$ zero-leaves. Hence

$$\#\text{leaves} \geq r + (r-1) = 2r - 1 > 2^{a-j}.$$

A depth-t binary protocol has at most 2^t leaves, so the restriction has depth at least $a + 1 - j$. At $j = 0$, this gives $\mathrm{D}(R_0) \geq a + 1$. Conversely, Bob sends the column index in a bits and Alice sends the corresponding bit, so $\mathrm{D}(R_0) \leq a + 1$, and hence $\mathrm{D}(R_0) = a + 1 = d_0$. The unit-vector rows form the identity submatrix, so $\operatorname{rank}(R_0) = p_0 = 2^{d_0 - 1}$.

By minimality of the dyadic choice, $p_0 < 128(L+1)$. If $L = \lceil \log n \rceil$ and $n \geq 4$, then $L + 1 \leq \log n + 2 \leq 2 \log n$, so $p_0 < 256 \log n$ and $2^{p_0} < n^{256}$. Enumerating the 2^{p_0} bit strings and the p_0 column indices therefore prints the table deterministically in polynomial time. □

Proof of Lemma 6.6. We treat both cases together, except for the step that turns many blocks above the threshold into a lower bound. Fix $j \in \{0,1\}$, put $\beta_j = (3/5)2^{-j}$, and let $\lambda = 255/256$ be the total weight of the template rows (Definition 6.2). After transposition, each deletion bound concerns an interlace with the data below. Here q is the interlace width, u is the block parameter to which we apply Lemma 6.5 or Theorem 6.12, and the last column is a lower bound on the average block retention.

target	base	q	u	average block retention
A	R	$q_1 = s - 2$	$s2^{-j}$	β_j
R_{temp}	A	q_2	$q_2 2^{-j}$	β_j
N_{ref}	R_{temp}	s	$s2^{-j}$	$(\beta_j - (1-\lambda))/\lambda$

In each case the retained tuple-label event Y has weight $\geq x$. For N_{ref}, we discard the non-template rows. They have total weight $1-\lambda$, which gives the displayed average retention. We then map each retained Ω label to its bin part, the tuple $(j_1,\dots,j_s)$ in the full-product case and its $\mathcal{J}_{\mathrm{g}}$ label in the explicit case, forgetting the tail ξ, which template entries ignore. This map preserves the original weights, so its image still has weight $\geq x$, and choosing one retained label for each image label gives the template entries (Observation 4.5).

Let m be the number of blocks retaining base row weight at least the threshold $t = u\rho$, ux, and ux, respectively. The block-counting bound $q\beta \leq m + qt$ of Section 4 gives the following lower bounds for m/u:

$$\frac{3}{5}(1-2/s) - q_1\rho, \qquad \frac{3}{5} - q_2 x, \qquad \frac{2^j(\beta_j - (1-\lambda))}{\lambda} - sx.$$

All three exceed $9/16$ by the numerical bounds of Lemma F.1, which both parameter schedules satisfy. They also give the row thresholds $u\rho \leq s\rho \leq 1$ and $ux \leq \max\{q_2 x, sx\} \leq 1$, and $u \geq s/2 \geq 16$ since $s \geq 256$.

Full-product case. Here $x = 2^{-8L} \geq 2^{-u/1024}$ by Lemma F.2, so Lemma 6.5 applies to the more than $9u/16$ blocks above the threshold.

Explicit case. We apply Lemma 6.10 to Y with $v = x$ and $\tau = 1$. Outside at most $e(x)$ exceptional coordinates, the conditional marginals of $\nu \mid Y$ stay close to the product weights, and $e(x) \leq s/128 \leq u/32$ by Lemma F.3. Discarding the blocks at these coordinates leaves more than $9u/16 - u/32 > u/2$ blocks above the threshold. Since u is dyadic, at least $u/2+1$ of them remain. These blocks and $K = Y$, for which $(\nu \mid Y)(K) = 1$, meet the hypotheses of Theorem 6.12 with $a = \log u$, which gives the costs computed below.

In both cases the base bound is $\mathcal{P}(R;\rho,d)$ for the first row and, for the other two rows, the deletion bounds just established for A and R_{temp}. The required costs are therefore

$$d + \log s - j = H_1 - j, \qquad H_1 + \log q_2 - j = H_2 - j, \qquad H_2 + \log s - j = C - 1 - j,$$

which are the three asserted pairs of deletion bounds.

It remains to prove the collision bound for the interlace on Γ. The first q_1 positions and three distinct tail positions give $q_1 + 3 = s + 1 = (2s)/2 + 1$ blocks. By hypothesis, each retains base row weight $\geq x_{\mathrm{cap}} = 2s\rho$, and the retained list labels have weight greater than $1/2 > 9/25$. In either case the list Γ is exactly eight-wise independent with the base-column marginal, so Lemma 6.11 with $a = \log(2s)$ gives $d + \log(2s) = H_1 + 1$ bits. □

B Bounds on transferred inputs

We prove Lemmas 6.14 and 6.15, which bound the $\mathcal{D}$-weight arriving at a stopping node (defined in Lemma 6.13) and the weight sent to the stopping node assigned to another copy. Throughout, we assume the common hypotheses (H1)–(H4) of Section 6.5 and drop the subscript g of the explicit construction. In each source copy, the $\mathcal{D}$-inputs have total weight $\mu = 1/256$, and the template rows and the Ω-inputs each have total weight $\lambda = 255/256$.

B.1 Two template inputs at one stopping node

The next lemma uses the transposed orientation: its rows are template-label pairs, and its two column families come from one copy's Ω-inputs and another copy's $\mathcal{D}$-inputs.

At the node assigned to copy i, an Ω-input of copy i reads the template label at coordinate i, and a $\mathcal{D}$-input of another copy j reads the template label at coordinate j. By (H3), these two labels are independent under the original weights. The next lemma shows that if both families are retained, the protocol needs one bit more than the cost H_2 given by $\mathcal{P}(R_{\mathrm{temp}}; x, H_2)$, even if the two families have the same column labels in J_2.

Lemma B.1 (Two template inputs)**.** *Let $A : \Gamma \times J \to \{0,1\}$ satisfy $\mathcal{P}(A; x, H_1)$ with $H_1 \geq 2$, and let q_2 be dyadic with $2q_2x \leq 1/8$. Take two weighted lists S_o, S_f of words in J^{q_2}, each exactly eight-wise independent with the base-column marginal, and use product weights on $S_o \times S_f$. Put $H_2 = H_1 + \log q_2$ and give $J_2 = [q_2] \times \Gamma$ the product of uniform index weights and the row weights of A on Γ. Rows are pairs of template labels $(z_o, z_f) \in S_o \times S_f$. The columns form two separately tagged copies of J_2, with entries*

$$A(\gamma, (z_o)_\alpha) \quad \textit{and} \quad A(\gamma, (z_f)_\alpha),$$

respectively. The tags remain distinct even when the inner indices coincide. Retain a set of pairs of original product weight at least $49/64$, columns reading z_o of normalized weight at least $1 - \varepsilon$, and columns reading z_f of normalized weight at least δ, each tagged copy of J_2 having total weight one, where

$$\varepsilon \geq 0, \qquad 0 < \delta \leq 1, \qquad \delta > 2q_2x + \varepsilon.$$

Then the retained matrix requires at least $H_2 + 1$ bits.

Proof. Put $t = 2q_2x$. We group columns by their tag and index α. These $2q_2$ groups have total retained Γ-weight at least

$$q_2(1 - \varepsilon + \delta) > q_2(1 + t).$$

If at most q_2 groups had weight at least t, this sum would be at most $q_2 + q_2t$. Thus at least $q_2 + 1$ groups retain weight $\geq t$.

We transpose the matrix. These groups are now distinct row blocks of an interlace of A. Concatenating S_o and S_f with product weights gives an exactly eight-wise independent tuple list on $2q_2$ tagged coordinates, since any set of at most eight coordinates splits between the two independent lists, and each list has the required marginals on its part. We keep the surviving pair labels as tuple labels, whose original weight is $\geq 49/64 > 9/25$, and apply Lemma 6.11 with the integer $a = \log(2q_2)$ and row threshold $2q_2x \leq 1/8$. This gives $H_1 + \log(2q_2) = H_2 + 1$ bits, as required. □

In the application (the proof of Lemma 6.15), we fix a bin at each of the two copies, and the pairs (z_o, z_f) occur only in the bin fixed at copy i, which we call the *distinguished bin*. On the other bins we know nothing about the entries in the other copy's columns, so the next claim extends Lemma B.1 to allow arbitrary entries there.

Claim B.2. *Use the common source weights and deletion bounds of Section 6.5, and put $\lambda_0 = 2q_2x$, $\tau_0 = 2sx$, $C = H_2 + \log s + 1$, and*

$$a_{\min} = \frac{99/100 + \lambda^2 - 1}{\lambda^2}.$$

Retain columns of normalized Ω-weight at least $1 - \varepsilon$, and separately tagged other-copy columns whose weight-preserving map $(\mathrm{tail}(\gamma), j, \ldots, j) \mapsto j$ to J_2 has retained image weight at least δ. Number the bins so that the distinguished bin is bin one. It retains a set $P_* \subseteq S^2$ of pairs (z_o, z_f) of product weight at least $a_{\min}$, and every other bin retains a set of labels of S-weight at least τ_0. Write

coordinates of a column in Ω *as* $Z = (\xi, j_1, \ldots, j_s)$*, where* $\xi \in Y^{s+1}$ *and* $j_p = (\alpha_p, \gamma_p) \in J_2$*. Require entries*

	$Z \in \Omega$	*other-copy image* (α, γ)
$(z_o, z_f) \in P_*$	$A(\gamma_1, (z_o)_{\alpha_1})$	$A(\gamma, (z_f)_\alpha)$
$(p, z),\ p \neq 1$	$A(\gamma_p, z_{\alpha_p})$	*arbitrary.*

If $\varepsilon \geq 0$*,* $0 < \delta \leq 1$*, and* $\delta > \lambda_0 + 4\varepsilon$*, then every such matrix requires at least* C *bits.*

Proof. Suppose this matrix has a protocol of depth at most $C - 1$. In each bin, we follow the retained set of template labels through Alice's bits until Bob first speaks, using the tracking rule stated before Lemma 6.8 with $t = \tau_0$. The only marked set is the set P_* of pairs in the distinguished bin. For the thresholds, we measure its weight by its projection onto z_o, and a split replaces its two parts by their projections onto z_o, which are sets of labels from S. A pair set weighs no more than its projection onto z_o, so an unsplit pair set loses less than τ_0 of its original weight. The common numerical bounds give $\tau_0 \leq 10^{-3}$ and $a_{\min} - \tau_0 > 49/64$. An unsplit set of pairs therefore retains weight $> 49/64$, and every retained set at depth r has original weight $\geq \tau_0 2^{-r}$.

The hypothesis on δ implies $\varepsilon < 1/4$, so the columns from Ω retain normalized weight $> 3/4$, and so does their projection onto any bin. Since $H_2 > 0$, the bounds $\mathcal{P}(R_{\text{temp}}; x, H_2)$ rule out a leaf containing a retained set of weight $\geq x$. A retained set also cannot reach depth $\log s + 1$: it would have weight at least x, but only $H_2 - 1$ bits would remain. Thus every retained set reaches a Bob node by depth $\log s$.

At each stopping node v, let h_v be the number of tracked sets and z_v the number of unsplit pair sets among them. By the tracking rule, $\sum_v (h_v + z_v)$ equals $s + 1$, the s initial sets plus the one initial mark. Thus the stopping nodes satisfy the hypotheses of Lemma 6.7 with $q = s$. Let v, at depth r, be the node given by that lemma, and write $\kappa = \kappa_v$, $h = h_v$, $z = z_v$. The sets at v have distinct bin coordinates, since descendants in different branches never rejoin, and each has original weight $\geq \tau_0 2^{-r} = 2\kappa x$. By Lemma 6.7, $h \geq \kappa/2 + 1$ if $\kappa \geq 2$, while for $\kappa = 1$, either $h \geq 2$ or $h = z = 1$.

We first treat the case $\kappa = 1$, $h = z = 1$, before restricting columns or projecting the pair. Since only Alice has spoken, both tagged column families are still unchanged. We project columns from Ω onto j_1 and other-copy columns onto their J_2 images, choosing one surviving column per image label. These maps preserve the original weights, so the projected weights are at least $1 - \varepsilon$ and δ. The set of pairs has weight $> 49/64$. Thus Lemma B.1 requires $H_2 + 1$ bits, while the stopping node has only $C - 1 - r = H_2$ bits left. The original weights on pairs of template labels are the product of their weights on S, as required by that lemma.

In all other cases, at Bob's bit we choose the child with more weight from Ω, which retains weight $> 3/8 > 9/25$. We replace any set of pairs by its projection onto z_o, which has at least its weight. The entries now form a restriction of an interlace of R_{temp} on the tracked bins. Its original column list is the bin part of Ω, which by (H3) has exactly 8192-wise independent bin coordinates under the original weights in either construction. We forget the tail ξ, which template entries ignore, but keep the whole tuple $(j_1, \ldots, j_s)$ in the full-product case and its whole list index in the explicit case. The retained image has original weight $> 3/8$, and choosing one surviving row or column for each projected label gives the required entries (Observation 4.5). Thus Lemma 6.11 applies in both constructions. For $\kappa \geq 2$ we use width κ. The $\kappa/2 + 1$ blocks have row weight $\geq 2\kappa x \geq \kappa x$, so the lemma requires $H_2 + \log \kappa$ bits, but the child has only $H_2 + \log \kappa - 1$ bits left. For $\kappa = 1$ there are at least two blocks of weight $\geq 2x$, and width two gives the stronger bound $H_2 + 1$. By the parameter choices, all these row thresholds are at most one. This contradiction does not use the other-copy entries outside the distinguished bin, which proves the claim. □

B.2 Total weight arriving at a stopping node

We first bound the total $\mathcal{D}$-weight arriving at one stopping node, including weight from copies without an assigned node. Too much incoming weight leaves many blocks of substantial row weight, and Theorem 6.12, applied after conditioning, then requires more communication than remains.

Proof of Lemma 6.14. Put $\kappa = 1$ at a node at depth $\log w$. Fix any collection of copies, and suppose the weight of their inputs from $\mathcal{D}$ arriving at one stopping node is at least $2s\kappa$. Let $d_i \in [0,1]$ be the fraction of copy i's inputs from $\mathcal{D}$ reaching it, and set $d_i = 0$ outside this collection. Since each such family has weight $\mu = 1/256$,

$$D_0 := \sum_i d_i \geq 512 s\kappa.$$

These fractions are fixed by the Alice-only transcript. At a node where Bob first speaks before depth $\log w$, we choose a child whose column set Y^* has original weight $\geq \theta/2$, which is possible since the tuple event Y' of Lemma 6.13 has weight $\geq \theta$. At a node at depth $\log w$ we put $Y^* = Y'$.

To apply the template lower bounds, we need retained column tuples in which at least half of the incoming weight D_0 comes from copies whose label is a template row. Let T_i indicate that the label at coordinate i of a column tuple is a template row. When the type, bin, and item labels are drawn according to their fixed weights, the centered variables $T_i - \lambda$ are 256-wise independent. The weighted form of Lemma C.1, followed by Markov's inequality, gives

$$\Pr\left\{\sum_i d_i T_i < D_0/2\right\} \leq C_{128} D_0^{-128} \leq C_{128} s^{-128} \leq \theta/4.$$

Here $C_{128} = (2^{14}/(\lambda - 1/2)^2)^{128}$ is the fixed constant of (8), and the last inequality is a row of (10). Thus Y^* intersects the event $\sum_i d_i T_i \geq D_0/2$ in weight $\geq \theta/4$. We average over the type/bin/item sequence and the base-label sequence, and fix a configuration whose retained set Z of template-label sequences has original weight $\geq \theta/4$ and whose template positions have total weight $\sum_i d_i T_i \geq D_0/2$. By (H3), fixing those two sequences leaves the template-label sequence with its original weights and exactly 8192-wise independent J coordinates $z_{i,\alpha}$. We apply Lemma 6.10 to Z with $\tau = 1$, which removes at most $e(\theta/4) \leq sq_2$ coordinates.

Each retained input (α, γ) from $\mathcal{D}$ at a copy i with $T_i = 1$ reads $A(\gamma, z_{i,\alpha})$, regardless of its fixed bin. We group these inputs by (i, α) and measure each group within its fiber, that is, by the fraction of the inputs from $\mathcal{D}$ of copy i with index α that it contains. Since the index α is uniform, the q_2 groups of copy i then have total weight $q_2 d_i$, so the groups with $T_i = 1$ have total base-row weight $\geq q_2 D_0/2$. Set $K_0 = 2s\kappa q_2$ and $t = K_0 x$. From $w^2 q_2 x \leq 1$ and $D_0 \leq w$,

$$t \leq \frac{2s\kappa}{w^2} \leq \frac{1}{256w} < 1.$$

There are at most wq_2 groups, each of weight at most one. If h groups have weight at least t, the total weight of all groups is at most $h + wq_2 t$, so

$$h \geq \frac{q_2 D_0}{2} - \frac{K_0}{w} \geq K_0(128 - 1/w) > K_0 \geq K_0/2 + sq_2.$$

After deleting the exceptional coordinates, more than $K_0/2$ blocks outside that exceptional set remain, each of base row weight $\geq K_0 x$. We retain $K_0/2 + 1$ of them and all labels in Z. Choosing one incoming row for each retained image (i, α, γ) gives a restriction with the required entries, since

distinct images give distinct rows. Row weights use the original Γ weights, and the retained column set has weight one under the fixed conditional weights on Z.

Since K_0 is dyadic, we can apply Theorem 6.12 with base bounds $\mathcal{P}(A; x, H_1)$ and integer $a = \log K_0$. It requires $H_1 + \log K_0 = C + \log \kappa$ bits. Since the protocol has depth at most $C + \log w - 1$, if Bob first speaks before depth $\log w$, the chosen child has only $C + \log \kappa - 2$ bits left, and a node at depth $\log w$ has only $C - 1$. This proves the bound for every collection of copies. □

B.3 Transfer to another copy's assigned node

We next bound what one copy can send to the node assigned to another, which already retains all but the normalized fraction ε_i of the assigned copy's Ω-inputs. A large transfer would yield a restriction of the form in Claim B.2, which needs more bits than remain at that node.

Proof of Lemma 6.15. Let ε_i be the normalized Ω-weight missing at the node assigned to copy i, and put $\delta_i = 2q_2x + 4\varepsilon_i$. If $\delta_i \geq 1$, the conclusion is automatic: the inputs from $\mathcal{D}$ in one copy have weight only $\mu \leq \mu\delta_i$. Otherwise, suppose the transfer exceeds $\mu\delta_i$. Then the node receives a set of inputs labeled by $\mathcal{D}$ from some copy $j \neq i$ of actual weight $\mu\delta$ for some $\delta > \delta_i$.

Both copy indices lie outside the exceptional set for Y', and only Alice has spoken before reaching the assigned node. By Lemma 6.10 with error bound $\eta/2$, the projection of Y' onto coordinates i and j has original product weight $\geq 1 - \eta/2 > 99/100$. We restrict to the event that both coordinates are template rows. This event has weight λ^2, so the retained relative weight is at least

$$a_{\min} = \frac{99/100 + \lambda^2 - 1}{\lambda^2}.$$

We average over the two bin labels and fix a pair of bins attaining this relative weight. Their retained pairs of template labels form a set $P_* \subseteq S^2$ of original product weight $\geq a_{\min}$. The chosen bin at i becomes the distinguished bin. The projection at coordinate i, which lies outside the exceptional set, loses at most $\eta/2$ of its row weight by the same error bound, while each bin has weight λ/s, so every other bin retains weight at least

$$1 - \frac{\eta s}{2\lambda} > 2sx.$$

We choose one surviving completion in Y' for every pair in P_* and for every retained template label in the other bins. The bin label at i distinguishes the representatives of the other bins from those of the pairs, and the projected labels distinguish the representatives within each bin. Thus the chosen completions define a restriction of the original table with the claimed entries.

After transposition, the columns labeled by Ω from copy i have normalized weight $1 - \varepsilon_i$. The columns labeled by $\mathcal{D}$ from j have J_2-image weight $\delta > \delta_i$, since that image map preserves their original weights. A pair (z_o, z_f) reads $A(\gamma_{p_i}, (z_o)_{\alpha_{p_i}})$ on columns labeled by Ω and $A(\gamma, (z_f)_\alpha)$ on columns labeled by $\mathcal{D}$. Other bins have the same template entries on the first family, and their entries on the second family are immaterial. This is the restriction in Claim B.2, after renaming the distinguished bin. The claim requires C bits because $\delta > 2q_2x + 4\varepsilon_i$, whereas the assigned node has only $C - 1$ bits left. This contradiction proves the non-strict transfer bound, including the case $\varepsilon_i = 0$. □

C Conditioning and surviving blocks

We prove Lemma 6.10 by bounding the number of exceptional coordinates, and then show that a short protocol must leave two or three blocks to which Lemma A.2 still applies. For these estimates

only, we view a label drawn according to its fixed weights as a random variable. Expectations then express weighted sums over the fixed tuple list. The 1024th-moment bounds use the original weights ν, under which independence is exact, whereas the fourth-moment bound under $\nu \mid Y$ in Lemma C.2 uses only the approximate marginals outside the exceptional coordinates.

C.1 A moment count and the exceptional coordinates

We use the following moment bound twice: to bound the number of exceptional coordinates, and, in the proof of Lemma 6.14, to bound the probability that too few template positions survive. In the second use the coefficients are the fixed fractions d_i of incoming weight, so the bound applies directly to the incoming template weight $\sum_i d_i T_i$.

Lemma C.1 (Moment bound)**.** *Let $r \geq 1$ be an integer. Suppose $X_1, \dots, X_m$ have mean zero, $|X_i| \leq 1$, and every subfamily of at most $2r$ variables is independent. For fixed coefficients $a_i \in [0,1]$ with $W = \sum_i a_i \geq 1$,*

$$\mathbb{E}\left|\sum_i a_i X_i\right|^{2r} \leq (r^2 W)^r.$$

Unit coefficients give the same bound with $W = m$, and the empty sum is zero. In particular, centered indicators with 1024-wise independence satisfy

$$\mathbb{E}\left|\sum_i X_i\right|^{1024} \leq (2^{18} m)^{512}, \qquad \mathbb{E}\left|\sum_i X_i\right|^{256} \leq (2^{14} m)^{128}.$$

Proof. We expand the even power and group terms by which factor positions have equal indices. A group with an index appearing once has expectation zero. Every remaining pattern has $k \leq r$ index classes, each of size at least two. Labeling the classes in order of first appearance shows that there are at most r^{2r} such patterns. If their sizes are $n_1, \dots, n_k$, the absolute contribution from a fixed pattern is at most

$$\sum_{\substack{i_1,\dots,i_k \\ \text{distinct}}} a_{i_1}^{n_1} \cdots a_{i_k}^{n_k} \leq W^k \leq W^r,$$

because $a_i^{n_j} \leq a_i$. Summing over patterns proves the bound. Taking $r = 512$ and $r = 128$ gives the displayed special cases. □

We take the exceptional coordinates to be the union of a maximal disjoint family of small coordinate sets with large marginal error. The moment bound shows that this family is small.

Proof of Lemma 6.10. Put $\varepsilon = \varepsilon(v, \tau)$. We choose a maximal disjoint family $J_1, \dots, J_m$ of sets of at most eight coordinates whose conditional marginals differ from their product marginals by more than ε. If $m = 0$, we delete nothing. Otherwise, since ν has marginal μ^{J_i} on J_i, the total variation bound gives an event indicator f_i on each J_i with $\mathbb{E}_{\nu|Y} f_i - \mathbb{E}_\nu f_i > \varepsilon$ (take its complement if necessary). Under the original weights ν, any subfamily of at most 1024 such indicators is independent, because its disjoint sets use at most 8192 coordinates.

For $S = \sum_i (f_i - \mathbb{E}_\nu f_i)$, conditional Jensen's inequality and Lemma C.1 give

$$v(m\varepsilon)^{1024} < \nu(Y)\mathbb{E}_{\nu|Y}|S|^{1024} \leq \mathbb{E}_\nu |S|^{1024} \leq (2^{18} m)^{512}.$$

Thus $m < 2^{18} \varepsilon^{-2} v^{-1/512}$. We delete $E = \bigcup_i J_i$, so $|E| \leq 8m \leq e(v, \tau)$. Every remaining set of at most eight coordinates has the desired marginal: otherwise it could be added to the disjoint family, contradicting maximality. □

C.2 Some pair has a large projection

When many blocks survive, we apply Lemma A.2 to a pair of coordinates on which the retained labels project with product weight at least $9/25$. Such a pair exists because, if every one of many disjoint pairs had a smaller projection, the retained set itself would be too small.

Lemma C.2. *Fix disjoint coordinate pairs $J_1, \dots, J_m$ in a tuple list with marginal μ and label weights ν. Suppose every projection of a label set K has product weight $\mu^2(\pi_{J_i}K) < 9/25$. For an exactly eight-wise independent list,*

$$\nu(K) < \frac{2}{m^2} < \frac{9}{50m} \qquad (m \geq 16).$$

For an exactly 8192-wise independent list, let $K \subseteq Y$ with $\nu(Y) \geq v > 0$, and choose the pairs outside the exceptional set from Lemma 6.10, with any error bound $0 < \tau \leq 1$. Then

$$(\nu \mid Y)(K) < \frac{1}{4m} \qquad (m \geq 16).$$

Thus a retained set of conditional weight at least $1/(4m)$ forces one of these pairs to have product projection weight at least $9/25$.

Proof. Write $P_i = \pi_{J_i}K$ and put

$$p_i = \mu^2(P_i) < 9/25, \qquad X_i = \mathbf{1}[z_{J_i} \in P_i] - p_i, \qquad S = \sum_{i=1}^{m} X_i.$$

On K every indicator equals one, so $S > (16/25)m$. Under the product weights, the centered indicators are independent, and $\mathbb{E}X_i^2, \mathbb{E}X_i^4 \leq 1/4$. Only terms with no singleton index survive the fourth-power expansion. Hence

$$\mathbb{E}_{\mathrm{prod}} S^4 = \sum_i \mathbb{E}X_i^4 + 6\sum_{i<j} \mathbb{E}X_i^2 \mathbb{E}X_j^2 \leq \frac{m}{4} + \frac{6}{16}\binom{m}{2} \leq \frac{m^2}{4}.$$

Exact eight-wise independence gives the same fourth moment under ν, since each term uses at most four pairs. Markov's inequality gives $\nu(K) < 2/m^2$, because $1/[4(16/25)^4] < 2$. For $m \geq 16$, $2/m^2 \leq 1/(8m) < 9/(50m)$.

Now suppose the pairs lie outside the exceptional set for the fixed event Y. Every term in S^4 lies in $[-1, 1]$ and depends on at most eight coordinates outside that set. By Lemma 6.10, conditioning on Y changes its expectation by at most 2ε, where $\varepsilon = \varepsilon(v, \tau)$. Summing the m^4 terms and applying Markov therefore gives

$$p := (\nu \mid Y)(K) \leq \frac{2}{m^2} + \frac{2\varepsilon}{(16/25)^4} \leq \frac{2}{m^2} + 16\varepsilon.$$

The error term 16ε does not decrease with the number of pairs, so for large m we also use the original weights ν. Any subfamily of at most 1024 pair indicators involves at most 2048 coordinates and is therefore independent under ν. Since $K \subseteq Y$ and $\nu(Y) \geq v$, Lemma C.1 and Markov's inequality give

$$p = \frac{\nu(K)}{\nu(Y)} \leq \frac{(2^{18}m)^{512}}{v((16/25)m)^{1024}} \leq \frac{(2^{20}/m)^{512}}{v}.$$

Combining the bounds and using $\min\{u, w\} \le u^{511/512} w^{1/512}$ yields

$$\begin{aligned} mp &\le \frac{2}{m} + \min\{16\varepsilon m, 2^{20\cdot 512} v^{-1} m^{-511}\} \\ &\le \frac{2}{m} + (16\varepsilon)^{511/512} 2^{20} v^{-1/512}. \end{aligned}$$

The last term is at most

$$2^{20-26\cdot 511/512} v^{255/131072} < \frac{1}{32},$$

by $\varepsilon \le 2^{-30} v^{1/256}$ and $0 < v \le 1$. For $m \ge 16$, this gives $mp < 1/8 + 1/32 < 1/4$, as required. □

C.3 Small projections and short continuations

Under $\mathcal{P}(M; x, d)$ with $d \ge 2$, Lemma A.2 gives $d+1$ bits for two blocks of row weight $2x$ with product column weight $9/25$, and for three blocks of row weight $4x$ with product column weight $9/50$, whenever the row threshold is at most one. We apply it to projections as in Observation 4.5, whose column weight $\mu^J(\pi_J K)$ is a product weight even when the labels have conditional weights. The protocol walks in the proofs of Lemma 6.11 and Theorem 6.12 both end with the next lemma, which shows that enough surviving blocks and column weight still force $d+1$ bits.

Lemma C.3. *Suppose $d \ge 2$ and $\mathcal{P}(M; x, d)$, and let $b \ge 0$ be an integer with $2^{b+1} x \le 1$. Consider a restriction of an interlace on a tuple list with the base-column marginal weights that retains at least $2^b + 1$ blocks, each of base row weight at least $2^{b+1} x$, and a label set K. Assume one of the following.*

(E) The list is exactly eight-wise independent with weights ν, and $\nu(K) \ge (9/25) 2^{-b}$.

(C) The list is exactly 8192-wise independent with weights ν, $\nu(Y) \ge v > 0$, the exceptional set is chosen from Lemma 6.10 with error bound $0 < \tau \le 1$, the retained blocks have indices outside that set, $K \subseteq Y$, and $(\nu \mid Y)(K) \ge 2^{-b-1}$.

Then the restriction requires at least $d+1$ bits.

Proof. Let J be a set of at most eight retained coordinates. Under (E), exact marginals give $\mu^J(\pi_J K) \ge \nu(K) \ge (9/25) 2^{-b}$. Under (C), where $\varepsilon = \varepsilon(v, \tau) \le 2^{-30}$, Lemma 6.10 gives

$$\mu^J(\pi_J K) \ge (\nu \mid Y)(K) - \varepsilon \ge 2^{-b-1} - \varepsilon.$$

For $b \le 4$ this is again $\ge (9/25) 2^{-b}$, since the smallest margin is $(7/50) 2^{-4} = 7/800 > 2^{-30}$.

For $b \le 4$ we select the blocks directly. If $b = 0$, two blocks meet the first threshold above, and if $b = 1$, three meet the second. If $b = 2$, the coordinates of five surviving blocks have joint product weight $\ge 9/100$, so Lemma A.1 gives a pair of weight $\ge (9/100)^{2/5} > 9/25$. If $b = 3$ or $b = 4$, eight coordinates have weight $\ge 9/400$, so some pair has weight $\ge (9/400)^{1/4} > 9/25$.

For $b \ge 5$, we choose $m = 2^{b-1} \ge 16$ disjoint pairs from the at least $2m + 1$ retained blocks. If every pair projection had product weight less than $9/25$, Lemma C.2 would give $\nu(K) < 9/(50m) = (9/25) 2^{-b}$ under (E), and $(\nu \mid Y)(K) < 1/(4m) = 2^{-b-1}$ under (C), a contradiction.

Each selected block has row weight $\ge 2^{b+1} x$, hence $\ge 2x$, and $\ge 4x$ in the triple at $b = 1$. These thresholds are at most one by $2^{b+1} x \le 1$. Thus Lemma A.2 gives $d+1$ bits in every case. □

Proof of Lemma 6.11 and Theorem 6.12. In Lemma 6.11 we measure column weight under ν, and in Theorem 6.12 under the fixed conditional weights $\nu \mid Y$, where the event Y and its exceptional set stay fixed. Suppose a protocol for the restriction has depth at most $d + a - 1$. We follow it for $a - 1$ messages by a weight-halving walk. At each Alice message, we assign every current block to a child retaining at least half its row weight and keep a child assigned at least half the blocks. At each Bob message, we keep a child retaining at least half the current column weight. If the protocol ends earlier, we pad the leaf with dummy Alice messages, which leave the restriction unchanged. If A messages are Alice's and B are Bob's, then $A + B = a - 1$, at most d bits remain, and the surviving restriction, with label set K', has

$$\begin{aligned} \text{number of blocks} &\geq \left\lceil (2^{a-1} + 1)/2^A \right\rceil \geq 2^B + 1, \\ \text{row weight per block} &\geq 2^{a-A} x = 2^{B+1} x, \\ \text{column weight of } K' &\geq 2^{-B} \cdot \text{initial column weight}. \end{aligned}$$

In Theorem 6.12 the surviving blocks still have indices outside the exceptional set. The initial column weight is $\nu(K) \geq 9/25$ in Lemma 6.11 and $(\nu \mid Y)(K) \geq 1/2$ in Theorem 6.12, so K' satisfies hypothesis (E), respectively (C), of Lemma C.3 with $b = B$. Its row threshold is at most one because $B + 1 \leq a$. It requires $d + 1$ further bits, while only d remain. □

D Constructing rational input lists

The deterministic reduction prints its tuple lists together with their rational weights, so Lemma 7.1 must bound the encoding length and not only the number of tuples. We use the sequential construction of Koller and Megiddo [15, Section 3]: extend each word by every coordinate value with its product weight, then reduce the list while preserving the joint weights on all sets of at most D coordinates. We recall the procedure and bound the bit length of the weights.

Proof of Lemma 7.1. Fix the independence order D and put $P = (1 + qm)^{\mathsf{D}}$, which bounds the number of constraints below. We start with the empty word of weight one. At each step, we append every possible coordinate value y to every retained word and multiply its weight by $\mu(y)$. On the resulting candidate set $\mathcal{U} \subseteq Y^i$, the weights satisfy, by induction on i,

$$\sum_{u \in \mathcal{U}} w_u \mathbf{1}[u_J = a] = \prod_{j \in J} \mu(a_j) \qquad (J \subseteq [i],\ |J| \leq \mathsf{D},\ a \in Y^J). \tag{*}$$

There are at most P equalities in (*), including the normalization equation for $J = \varnothing$. The cited procedure removes linear dependences among the zero–one columns of this system until at most P words remain, and it preserves the equalities and the nonnegativity of the weights. Each stage therefore extends at most P words to at most mP candidates. After q stages the equalities give the required D-wise independence.

Write $a(u)$ for the constraint column of word u. A reduction step takes an integer dependence $\sum_u c_u a(u) = 0$ and sets $w'_u = w_u - t c_u$, where $t = \min_{c_u > 0} w_u / c_u$, which removes at least one word of positive weight. To find the dependence, we take a basis of $r \leq P$ constraint columns, one further column, and r rows on which the basis columns are independent. The determinant minors of this $r \times (r + 1)$ zero–one matrix give an integer relation valid in every row, with coefficients at most $\max\{1, r^{r/2}\}$ by Hadamard's bound, and hence with $O(P \log(P + 1))$ bits. Exact linear algebra finds this relation in polynomial time, and we break all ties by fixed orderings.

Let L_μ be the total binary encoding length of the weights μ. Extending a word multiplies the weights' common denominator by at most the product of the denominators in μ, adding at

most L_μ bits. An elimination multiplies it by the positive coefficient attaining the minimum ratio, adding $O(P\log(P+1))$ bits. Across q extensions and at most qmP eliminations, every denominator therefore has bit length

$$O\big(qL_\mu + qmP^2\log(P+1)\big).$$

Numerators obey the same bound because the weights lie in $[0,1]$. Hence all arithmetic has polynomial cost for fixed D, and so does the output of at most P weighted words of length q. □

E Printing a sample of the implicit table

The randomized reduction uses the implicit table only through sampled rows, columns and their entries, which we generate without printing the table. We first bound the length and generation cost of complete input labels, then implement their elementary draws with fixed blocks of fair bits. Evaluating one entry reads only one base entry per recursion level, but its labels must be generated in full, including the coordinates that the evaluation never reads.

Proof of Lemma 9.5. We fix a source graph on n vertices above the size cutoff in Lemma F.2, put $L = \lceil \log n \rceil$, and use the parameter choices of that lemma at every level. Write $R : X \times Y \to \{0,1\}$ for the current base matrix.

Labels and entries. The two interlaces $U = \mathsf{I}_w(N)$, $W = U^{\mathsf{T}}$ and $R^+ = (\mathsf{I}_Q(W))^{\mathsf{T}}$ give the row and column labels of R^+ (the successor labels)

$$\mathbf{a} = ((i_t, Z_t))_{t\in[Q]}, \qquad \mathbf{b} = (j, (k_i)_{i\in[w]}),$$

where $i_t \in [w]$ and $j \in [Q]$ are uniform, Z_t are tagged columns of B_{raw}, and $k_i \in \mathcal{K}$ are rows of B_{raw}. We draw all these coordinates independently, each according to its weight in Definitions 4.3 and 6.2. The entry rule is

$$R^+(\mathbf{a}, \mathbf{b}) = B_{\mathrm{raw}}(k_{i_j}, Z_j).$$

The entry rules in Definition 6.2 select exactly one entry of R. To generate a row k_i or column Z_t of B_{raw}, we sample according to the row-type and column-family weights from that definition and draw every listed coordinate, including those not visited by the eventual entry evaluation. The family $\mathcal{D}$ copies one tuple and its tail, so it does not require extra draws for the copied positions. We generate only actual input coordinates, not the auxiliary sequences used only in the lower-bound proof. The recursive draw counts are

label	base-row draws	base-column draws
template row	q_2	0
row for an item	1	0
Ω column	0	$(s+1) + s(2s-1) = 2s^2+1$
$\mathcal{D}$ column	0	$2s-1$
successor row	0	$\le Q(2s^2+1)$
successor column	$\le wq_2$	0

Apart from these recursive draws, generating indices and scanning activity lists cost polynomial time in $n + s + q_2 + w + Q + L + 1$, copying a label costs time linear in its length, and no coloring is required. Every width s, q_1, q_2, w, Q is $2^{O(L)}$ by Lemma F.2, and so is every draw count in the table. Hence we can choose common computable integer bounds J_i for the label length, generation time, elementary draw count and entry-evaluation time at level i such that

$$J_0 = O(L), \qquad J_{i+1} \le 2^{c_0 L} J_i, \qquad J_k = 2^{O((k+1)L)},$$

where c_0 is fixed and effective. Here each elementary draw counts as one operation, and we implement it with fair bits below. At the initial matrix R_0 of Lemma 5.4, a row consists of $p_0 = O(L)$ independent uniform bits, a column is a uniform index in $[p_0]$, and an entry can be read in $O(L)$ time. This gives $J_0 = O(L)$.

Since $h = O(L)$ and $d_0 = O(L)$, we have $B_k = O((k+1)L)$. The direct sample size in Theorem 9.4 then gives $m = 2^{O((k+1)L)}$. We generate and store the m row labels and m column labels, then evaluate all m^2 row–column entries. Including the stored tuples and printed entries, this costs

$$O((2m + m^2)J_k) = 2^{O((k+1)L)} = n^{O(k+1)}.$$

All constants are independent of k.

Fixed-bit implementation. Let $N_{\mathrm{draw}} \geq 1$ be a computable bound on the elementary draws for all $2m$ labels, so $N_{\mathrm{draw}} = 2^{O((k+1)L)}$. On every tape, each draw has at most

$$A = \max\{512, 4s(n+s), q_2, w, Q, p_0\} = 2^{O(L)}$$

outcomes, and each rational mixture has a common denominator $\leq A$. This holds for type draws (whose weights in Definition 6.2 have denominators 512 and 256), item indices, bins, activity indices, interlace-copy indices and draws from the initial matrix. Base rows and columns are never drawn directly. We generate them recursively down to the initial matrix. We set $b = \lceil \log(200AN_{\mathrm{draw}}) \rceil$ and reserve N_{draw} blocks of b fair bits. A draw from $[a]$ reads the next block as a uniform integer $U \in \{0, \ldots, 2^b - 1\}$ and returns

$$\mathrm{Draw}(a) = 1 + \left\lfloor aU/2^b \right\rfloor. \tag{7}$$

A rational mixture with common denominator $a \leq A$ maps consecutive intervals of values of $\mathrm{Draw}(a)$, whose lengths are the numerators of its weights, to its types. Unused blocks are ignored.

Each value in $[a]$ has $\lfloor 2^b/a \rfloor$ or $\lceil 2^b/a \rceil$ preimages, so a draw is within total variation $a/2^b \leq 1/(200N_{\mathrm{draw}})$ of the ideal one, and mapping values to types cannot increase this distance. Because every draw reads a fresh block, the bound holds even when the alphabet depends on earlier outcomes. We couple the implemented and ideal draws one at a time. By a union bound over the N_{draw} draws, some pair disagrees with probability at most $N_{\mathrm{draw}}/(200N_{\mathrm{draw}}) = 1/200 < 1/100$.

Every tape thus produces exactly m legal row labels and m legal column labels, and all m^2 entries are evaluated exactly. The tape has fixed length bN_{draw}, where $b = O((k+1)L)$. Arithmetic on $O(b + \log A)$-bit integers adds only polynomial overhead. Hence the running time and the number of random bits are $n^{O(k+1)}$ on every tape. □

F One parameter choice for both constructions

We verify the numerical requirements of the recipe in Definition 6.1: first those shared by both constructions, then, for each choice of tuple weights, the source-matrix deletion bounds and the final interlace.

Recall $n = |V|$, $\lambda = 255/256$, $\mu = 1/256$, and $\zeta = \gamma/24$. The source promise gives

$$n/2 \leq |E| \leq \Delta n/2, \qquad \delta_v = \frac{\gamma|E|}{2\Delta n} \geq \frac{\gamma}{4\Delta}.$$

From Lemma 6.13 we use the bound E_{exc} on the sum of κ over stopping nodes with no assigned copy, and from the proof of Lemma 6.14 the weighted moment constant C_{128}:

$$E_{\mathrm{exc}} = 2e(\theta, \eta/2), \qquad C_{128} = \left(\frac{2^{14}}{(\lambda - 1/2)^2} \right)^{128}. \tag{8}$$

The definition of $e(v,\tau)$ before Lemma 6.10 gives the estimate

$$\begin{aligned} e(v,\tau) &\le 8 + 2^{23}\tau^{-2}v^{-1/512} + 2^{83}v^{-5/512} \\ &\le 2^{84}\tau^{-2}v^{-1/64} \qquad (0 < v,\tau \le 1). \end{aligned} \tag{9}$$

Lemma F.1 (Common numerical bounds)**.** *There is an effective dyadic constant s_0, depending only on the fixed graph promise, such that the recipe in Definition 6.1 with $n \le \rho^{-1}$, $\rho \le s^{-4}$, and $s \ge s_0$ satisfies all source and internal-interlace bounds listed in* (10) *below. The choice of s_0 is independent of the base matrix and the number of rounds.*

Proof. The middle column of the table below lists upper bounds obtained by substituting the recipe. Each tends to zero as s increases. The last column lists the bound that the proof requires for each quantity.

quantity	upper bound	required allowance
$s\eta$	s^{-1}	$2^{-16}\min\{1,\delta_v,\mu\zeta\}$
$4s\lvert E\rvert/q_2$	$2\Delta/s$	1
$s^2\rho$	s^{-2}	$1/4$
sq_1/q_2	s^{-4}	$\zeta/128$
$\max\{q_2x, sx, wx\}$	s^{-24}	10^{-4}
w^2x/η	s^{-14}	$1/(32\lambda)$
w^2q_2x	s^{-10}	1
wq_2x/η	s^{-16}	$1/(128\mu)$
$e(\theta/4)/(sq_2)$	$2^{85}s^{-53/8}$	1
$C_{128}s^{-128}/\theta$	$512C_{128}s^{-104}$	$1/4$
$2sE_{\mathrm{exc}}/(\eta w)$	$2^{89}s^{-5/8}$	$1/16$

(10)

For the two rows involving e, (9) with $\theta = 2^{-9}s^{-24}$ and $\eta = s^{-2}$ gives $e(\theta/4) \le 2^{85}s^{3/8}$ and $E_{\mathrm{exc}} \le 2^{88}s^{35/8}$, while $sq_2 = s^3/\rho \ge s^7$. The other rows follow by substituting the widths of Definition 6.1 and using $\lvert E\rvert \le \Delta n/2$, $n \le \rho^{-1}$, and $\rho \le s^{-4}$. We choose a dyadic $s_0 \ge 256$ so that every middle entry is below its allowance, using $\gamma/(4\Delta)$ in place of δ_v. Clearing the fixed rational exponents makes this a finite set of integer-power comparisons. Their left sides decrease with s, so the same choice works for every $s \ge s_0$. None of these comparisons involves the base matrix or the number of rounds.

The table's second row gives $q_2 \ge 4s\lvert E\rvert$. Since q_2 is a power of two, $q_2 \ge q_0 = \mathrm{pow}_2(4s\lvert E\rvert)$, as Definition 6.2 requires. In particular, $q_2 \ge 2sn \ge 2(n+s-3)$, so all item labels fit: $\lvert\mathcal{I}\rvert = 4s(n+s-3) \le 2sq_2$, as Lemma 6.18 requires. The table's last bound also implies the two bounds used to count assigned nodes:

$$E_{\mathrm{exc}} \le \frac{\eta w}{32s} < \frac{w}{4}, \qquad e(\theta) \le e(\theta,\eta/2) = \frac{E_{\mathrm{exc}}}{2} \le \frac{\eta w}{64s} < \frac{w}{128}.$$

Here $e(\theta) = e(\theta,1) \le e(\theta,\eta/2)$ because $e(v,\tau)$ is nonincreasing in the error bound τ.

The remaining fixed-fraction checks appear in the proofs that use them. For each, we list the table rows it uses and derive here only the bounds that the proof does not display.

- *Block counts* (proof of Lemma 6.6). Since $s \ge 256$ and $s\rho < 10^{-3}$, the three lower bounds for m/u satisfy

$$\frac{3}{5}(1-2/s) - q_1\rho > \frac{9}{16}, \qquad \frac{3}{5} - q_2x > \frac{9}{16}, \qquad \frac{44}{75} - sx > \frac{9}{16},$$

the last since $2^j(\beta_j - (1-\lambda))/\lambda \geq ((3/5) - 4(1-\lambda))/\lambda = 44/75$ for $j \in \{0,1\}$. The bound on m/κ in the proof of Theorem 6.16 uses $wx \leq 1/256$ and $4e(\theta)/w < 1/32$ from the display above.

- *Item deletion and collisions* (proof of Theorem 6.9). The row $s\eta$ gives $4096\eta sn \leq \delta_v n/16$ and $4096\eta s(s-3) \leq (s-3)/16 < s/2$. The rows $s\eta$ and sq_1/q_2 give $8sq_1 + 4s\eta q_2/\mu \leq (1/16 + 2^{-14})\zeta q_2 < \zeta q_2/4$.

- *Row and template thresholds.* The rows $s^2\rho$, $s\eta$ and $\max\{q_2x, sx, wx\}$ give $2s\rho \leq 1/8$, $2wx + \eta/(16\lambda) < 1$, $2sx \leq 10^{-3}$, and hence $999/1000 - 2(s-1)x > 998/1000$ in the proof of Theorem 6.9 and $(99/100 + \lambda^2 - 1)/\lambda^2 - 10^{-3} > 49/64$ in Claim B.2.

- *Conservation* (proof of Theorem 6.16). That proof uses the rows w^2x/η, wq_2x/η and $2sE_{\mathrm{exc}}/(\eta w)$ and displays the resulting bounds.

- *Moment bound* (proof of Lemma 6.14). Since $D_0 \geq 512s$, the probability bound $C_{128}D_0^{-128}$ from Lemma C.1 satisfies $C_{128}D_0^{-128} \leq C_{128}s^{-128} \leq \theta/4$ by the row $C_{128}s^{-128}/\theta$.

□

F.1 Full products

In the randomized reduction, the graph-encoding width $q_2 = s^2 2^L$ is the only width that contributes a term linear in L to the per-round overhead h. The widths s, $w = s^8$, and $Q = s^{24}$ are polynomial in $\log n$. A smaller overhead allows more amplification rounds before the final sample becomes too large.

Lemma F.2 (Parameters for full products)**.** *Put $L = \lceil \log n \rceil$, $\rho = 2^{-L}$, and $s = \mathrm{pow}_2(2^{16}(L+1))$, and use the recipe of Definition 6.1. For every sufficiently large promised source, these choices satisfy the hypotheses of Lemma F.1, the numerical conditions for the full-product deletion bounds for the source matrices, and the numerical hypotheses of Theorem 6.17 with $b = L$ and $\delta = \delta_U$. The per-round overhead is*

$$h = 2\log s + \log q_2 + \log w + \log Q = L + 36\log s = L + O(\log L).$$

There is an effective fixed cutoff beyond which also $L \geq 16$, $d_0 \leq L$, and $h \leq 7L$, where d_0 is the cost of the initial matrix in Lemma 5.4. Computing the widths has polynomial bit complexity and uses neither the base matrix nor the number of rounds.

Proof. Since $s = \mathrm{pow}_2(2^{16}(L+1))$, we have $2^{16}(L+1) \leq s < 2^{17}(L+1)$. Hence $s \geq s_0$ and $2^{-L} \leq s^{-4}$ beyond an effective fixed cutoff, so the common lemma applies. In each application of the deletion bounds for the source matrices, the block parameter satisfies $u \geq s/2$. Since $\log(1/x) = 8L$,

$$u \geq 2^{15}(L+1) > 4096\log(1/x),$$

which gives the full-product block threshold $x \geq 2^{-u/1024}$. The common numerical bounds give the required block counts and row-weight bounds. For the final interlace the remaining checks are

$$0 < \delta_U = \frac{1}{16s^{10}} < \frac{1}{64}, \qquad Q\theta = \frac{1}{512} < \frac{1}{20}, \qquad Q\delta_U = \frac{s^{14}}{16} \geq 80L.$$

Thus Theorem 6.17 applies with $b = L$.

Substituting the widths gives the displayed value of h. Writing $\ell = \lceil \log(L+1) \rceil$, the initial matrix of Lemma 5.4 has $d_0 = \log p_0 + 1 = \ell + 7$ and $\log s = 16 + \ell$, so $d_0 \le L$ and $h \le 7L$ eventually. Since $s = \Theta(L+1)$ with explicit constants, the ratios $s^4 2^{-L}$, d_0/L and $(h-L)/L$ are at most fixed multiples of $(L+1)^4 2^{-L}$ or $(1+\log(L+1))/L$. These bounds decrease to zero beyond a computable point. Hence the cutoff is effective and valid at every larger size. All widths except $q_1 = s-2$ are powers of two with exponents $O(L)$; forming those powers and subtracting two from s gives the stated bit complexity. □

F.2 Explicit lists

For the deterministic reduction, we choose s with $s^4 \ge n$ and set $\rho = s^{-4}$. Then every width is polynomial in s, including the final-interlace width $Q = s^{24}$ required by Theorem 7.2.

Lemma F.3 (Parameters for explicit lists)**.** *After increasing the effective dyadic constant s_0 if necessary, every dyadic $s \ge \max\{n^{1/4}, s_0\}$ satisfies the bounds below. Choose the least such s and put $\mathsf{N} = s^4$, $L_{\mathrm{g}} = \log \mathsf{N}$, and $\rho = \mathsf{N}^{-1}$. With these choices, the common recipe satisfies all numerical hypotheses of the explicit-list construction and of the final interlace in Theorem 7.2. Its overhead is*

$$h_{\mathrm{g}} = 2\log s + \log q_2 + \log w + \log Q = 40 \log s = 10 L_{\mathrm{g}}.$$

With $\mathsf{N}_0 = s_0^4$, the padding satisfies $n \le \mathsf{N} < 16 \max\{n, \mathsf{N}_0\}$. Every width is polynomial in N. One round has a fixed polynomial cost in N and the complete base encoding, with exponent independent of the round number.

Proof. Here $\rho = s^{-4}$, so the common lemma applies. Only the deletion bounds for the source matrices at tuple weight x and the final interlace need additional checks. By (9),

$$e(x) \le 2^{84} s^{1/2} \le s/128, \qquad e(\rho, \delta_U/2) \le 2^{94} s^{20+1/16}.$$

We increase s_0 so that $2^{84} s^{1/2} \le s/128$ and $128(2^{94} s^{20+1/16} + 1) \le s^{24} = Q$. Dividing each condition by its right side leaves fixed multiples of negative powers of s, so this enlargement is effective and valid for every larger s.

For the deletion bounds for the source matrices, $u \ge s/2$, and by the three block-count bounds in the proof of Lemma 6.6, more than $9u/16$ blocks meet the required row-weight threshold. Removing at most $e(x) \le s/128 \le u/32$ exceptional coordinates leaves more than $9u/16 - u/32 > u/2$ blocks. The row thresholds obey $u\rho \le s\rho \le 1$ for the first matrix and $ux \le \max\{q_2 x, sx\} \le 1$ in the other two. For the final interlace, $0 < \rho \le 1$, $0 < \delta_U < 1/64$, $Q\theta = 1/512 < 1/20$, and the bound $Q \ge 128(e(\rho, \delta_U/2) + 1)$ above are the numerical hypotheses of Theorem 7.2 with $\phi = \rho$ and $\delta = \delta_U$.

The least dyadic choice satisfies $s < 2\max\{n^{1/4}, s_0\}$, proving the factor-16 size bound. It can be found by comparing integer fourth powers, without approximating a root. The widths are $q_2 = s^6$, $w = s^8$, and $Q = s^{24}$, giving the stated overhead. The independence order stays fixed at 8192, so by Lemma 7.1 the fixed number of tuple-list constructions have cost polynomial in these widths and in the complete rational encoding of the base matrix. Neither the parameter choices nor the exponent of the per-round polynomial cost depends on the number of rounds. □

G Coloring promises from SAT

The two communication reductions use different coloring problems: bounded-degree graph coloring for interlacing (Theorem 5.2), and balanced hypergraph coloring for strict alternation (Proposition 10.2). This appendix derives both hardness results from the literature and checks the input conventions that the communication constructions need.

G.1 Bounded-degree graph coloring of almost-linear size

The graph source of Theorem 5.2 needs a constant coloring gap and must keep the almost-linear size of sparse SAT. Composing the reductions of Dinur and of Bogdanov, Obata and Trevisan gives both. We record this composition and our graph conventions because the size bound determines the final additive-error bound.

Proof of Theorem 5.2. For a SAT instance of encoding length L, Dinur's almost-linear PCP construction gives a constant-alphabet binary constraint graph of size $L(\log L)^{O(1)}$, with perfect completeness and a constant unsatisfied-constraint fraction in the NO case [5, Lemma 8.3 and proof of Theorem 8.1]. Her degree reduction makes the degree constant, preserves perfect completeness, and changes the size and the NO-case unsatisfied fraction by only constant factors [5, Lemma 1.7]. A constant-size Boolean encoding, followed by [3, Lemma 11], gives bounded-occurrence 3-SAT with perfect completeness, a constant NO-case gap and almost-linear size. The gap-preserving reduction of [3, Theorem 14 and its proof, pp. 98–99] then gives a bounded-degree graph of linear size.

It remains to match the conventions of Theorem 5.2. We remove isolated vertices and add a disjoint edge. This preserves completeness and, when the graph already has an edge, decreases the NO-case gap by at most a factor of two. For trivial inputs we use fixed promise graphs. For a sparse formula on N variables, $L = O(N \log(N+2))$, so the resulting graph H has $N(\log(N+2))^{O(1)}$ vertices and edges. Taking $\max\{1, \lceil N/|E(H)|\rceil\}$ disjoint copies gives $N \le |E(G)| \le N + |E(H)|$ without changing the degree or gap. The remaining edge bounds follow from the absence of isolated vertices and the constant maximum degree. Parallel edges are retained, and edge occurrences are counted with multiplicity. We choose a rational $\gamma > 0$ below the resulting gap and enlarge the fixed exponent p to cover the constant factors. The cited reductions and these adjustments are deterministic and polynomial-time. □

G.2 Balanced hypergraph coloring with distinct vertices

The hypergraph promise of Proposition 10.2 comes from the hardness result of Guruswami and Lee [8], whose edges may contain repeated vertices. We replace repeated occurrences by distinct labeled clones and check that the YES and NO conditions of Proposition 10.2 are preserved.

Proof of Proposition 10.2. Recall the fixed parameters of Proposition 10.2

$$s_{\mathrm{a}} = 2^t, \qquad K_{\mathrm{a}} = s_{\mathrm{a}}^2, \qquad S_{\mathrm{a}} = 2^{s_{\mathrm{a}}}.$$

By Theorem 1.1 of Guruswami and Lee [8], it is NP-hard to distinguish K_{a}-uniform occurrence hypergraphs admitting an s_{a}-coloring with at least $s_{\mathrm{a}} - 1$ occurrences of each color in every edge from those with no independent set containing at least a $1/(2S_{\mathrm{a}})$ fraction of the vertices. Here, as in their reduction, an edge of an occurrence hypergraph is a family of K_{a} vertex occurrences indexed by $[s_{\mathrm{a}}] \times [s_{\mathrm{a}}]$, listed in a fixed order. A vertex may occur more than once in an edge, and colors in an edge are counted with multiplicity, as in their completeness proof. Hypergraphs of the second kind admit no proper S_{a}-coloring, since some color class would contain at least a $1/S_{\mathrm{a}}$ fraction of the vertices and so would not be independent.

We replace each vertex u by $K_{\mathrm{a}}S_{\mathrm{a}}$ labeled clones. For each ordered occurrence edge $(u_1, \dots, u_{K_{\mathrm{a}}})$, we include every tuple

$$(u_1^{(j_1)}, \dots, u_{K_{\mathrm{a}}}^{(j_{K_{\mathrm{a}}})})$$

of pairwise distinct clones, in the displayed order. Retain repeated edge occurrences as separately labeled edges. Each original edge produces at least one such tuple, since each vertex has $K_{\mathrm{a}}S_{\mathrm{a}} \ge K_{\mathrm{a}}$

clones. Giving clones their source vertex's color preserves the color multiset of every edge, and hence the YES promise.

Conversely, suppose the cloned hypergraph has a proper S_{a}-coloring. We give each source vertex a color appearing on at least K_{a} of its clones. Suppose some occurrence edge were monochromatic in this induced coloring. Each source vertex occurs at most K_{a} times in that edge, so we can choose different clones of that color for its positions. This gives a monochromatic edge of distinct clones, contradicting properness. Hence the induced coloring is proper, contradicting the NO promise.

Each vertex produces $K_{\mathrm{a}}S_{\mathrm{a}}$ clones and each edge produces at most $(K_{\mathrm{a}}S_{\mathrm{a}})^{K_{\mathrm{a}}}$ cloned edges. These are constants for fixed t, so enumeration is deterministic and polynomial-time, although the degree may depend on t. If the vertex set is empty, we add one isolated labeled vertex, since Proposition 10.2 requires a nonempty vertex set. Because each ordered edge has distinct vertices, the position $\iota_\alpha(v)$ in Definition 10.3 is well defined. □

H Finite-field arrays

The alternating lower bound needs a short, explicitly generated list of arrays on which no prescribed bit pattern occurs much more often than it would for independent fair bits. We construct that list here, completing the algorithmic step promised in Corollary 10.6.

We use the polynomial-root construction of Alon, Goldreich, Håstad and Peralta [1, Construction 3 and Proposition 3]. The conversion from parity bounds to pattern frequencies is also recorded there, with credit to Vazirani [1, Lemma 1 and Corollary 1]. We use the following direct consequence and keep its seed labels explicit so that the reduction can print the full array list.

Lemma H.1. *For integers $n, r \geq 1$, a deterministic algorithm prints a nonempty labeled list $(w^\ell)_{\ell\in\mathcal{L}}$ of n-bit arrays satisfying*

$$\frac{|\{\ell \in \mathcal{L} : w^\ell|_I = z\}|}{|\mathcal{L}|} \leq 2^{-|I|} + 2^{-r} \qquad (I \subseteq \{0,\dots,n-1\},\ z \in \{0,1\}^I).$$

Its running time and complete output length are polynomial in $n2^r$.

Proof of Lemma H.1. In this proof only, let $R = 2^d$ be the least power of two with $R \geq n2^r$. Construction 3 and Proposition 3 of Alon et al. [1] enumerate R^2 labeled n-bit arrays whose nonempty parities have bias $\leq (n-1)/R \leq 2^{-r}$, where bias is the absolute expectation of the $\{\pm1\}$-valued parity. The parity-to-pattern conversion [1, Lemma 1] cited above then gives the displayed bound for every nonempty I and every z, and the case $I = \varnothing$ is immediate.

Explicitly, the construction fixes a binary basis of the field $\mathbb{F}_R$, writes $\mathrm{vec}(z) \in \mathbb{F}_2^d$ for the coordinate vector of $z \in \mathbb{F}_R$, and uses the array $w^{u,y}(j) = \langle u, \mathrm{vec}(y^j)\rangle$ (inner product over $\mathbb{F}_2$, $0 \leq j < n$) for each $(u, y) \in \mathbb{F}_2^d \times \mathbb{F}_R$, with $y^0 = 1$. We enumerate all seeds and keep repeated arrays as distinct labels. A field representation can be found by enumerating degree-d monic binary polynomials and checking their possible factors, in time polynomial in R. Field arithmetic and enumeration of the R^2 length-n arrays also take polynomial time in nR. Since $R < 2n2^r$, this proves the claimed printing and encoding bounds. □

Finally, we apply Lemma H.1 on the positions Ω_{a} of Definition 10.3 with error ϵ_{a}, and check that the list has size polynomial in H_{a} for each fixed t.

Proof of Corollary 10.6. We number Ω_{a} by $0,\dots,n-1$ and apply Lemma H.1 with

$$n = K_{\mathrm{a}}H_{\mathrm{a}}, \qquad r = 4(b+3t).$$

Here $2^{-r} = \epsilon_a$ and $n2^r = K_a 2^{12t} H_a^5$, which is polynomial in H_a for fixed t. The integers n and r have $O_t(b+1)$ bits, and the polynomial bound also covers $H_a = 1$. □

AI methodology

The authors' initial goal was to sparsify interlacing while retaining the hardness needed for the reductions. That research direction preceded the AI-assisted development of the detailed constructions and proofs.

The intermediate combined development already sparsified interlaces using balanced families with controlled projections. The present work distinguishes three constructions: implicit full products with one final random compression, explicit input lists that preserve the conditions for repeated amplification, and a separate small-bias construction for strict alternation.

ChatGPT contributed substantive proof proposals for full-product amplification followed by a single final compression in the randomized construction. ChatGPT later supplied proposals for the bounded-independence and conditioning arguments in the unrestricted deterministic construction. ChatGPT also proposed the separate small-bias argument for strictly alternating protocols.

Codex assisted with reconstructing and integrating the proposals, and with mathematical review of the arguments and their interfaces. Its work included checking implementation details and parameter accounting, running finite verification scripts, and drafting and revising the exposition. The affected material included the randomized amplification and final compression, the unrestricted deterministic construction, and the strict-alternation construction and its supporting appendices, together with exposition throughout the manuscript. Codex also assisted with the encoding-first proof overview and the terminology revision in the present version. Claude supplied editorial review.

The authors retain responsibility for the mathematical claims and their attribution. AI was used in proof development and review, not solely for copy-editing. AI-assisted checks, agreement between model reviews, and finite experiments are neither formal verification nor independent human review.

References

[1] Noga Alon, Oded Goldreich, Johan Håstad, and René Peralta. Simple constructions of almost k-wise independent random variables. *Random Structures & Algorithms*, 3(3):289–304, 1992. doi: 10.1002/rsa.3240030308. URL `https://web.math.princeton.edu/~nalon/PDFS/aghp4.pdf`. Our theorem numbering refers to the author-hosted version dated 22 February 2002: Lemma 1, Corollary 1, Construction 3 and Proposition 3.

[2] Beyond the Library. Formal proof inspector for Refuting the Direct Sum Conjecture. FormaliaXiv, 2026. URL `https://beyondthelibrary.github.io/formal_archive/p/Refuting/`. Accessed 25 October 2026.

[3] Andrej Bogdanov, Kenji Obata, and Luca Trevisan. A lower bound for testing 3-colorability in bounded-degree graphs. In *Proceedings of the 43rd Annual IEEE Symposium on Foundations of Computer Science*, pages 93–102. IEEE Computer Society, 2002. doi: 10.1109/SFCS.2002.1181886. URL `https://lucatrevisan.github.io/pubs/bot.pdf`.

[4] Binh-Minh Bui-Xuan, Jan Arne Telle, and Martin Vatshelle. H-join decomposable graphs and algorithms with runtime single exponential in rankwidth. *Discrete Applied Mathematics*, 158 (7):809–819, 2010. doi: 10.1016/j.dam.2009.09.009. URL `https://perso.lip6.fr/binh-minh.bui-xuan/files/BTV.pdf`. Our Proposition 3.6 locator follows the author-hosted manuscript.

[5] Irit Dinur. The PCP theorem by gap amplification. *Journal of the ACM*, 54(3), June 2007. doi: 10.1145/1236457.1236459. URL https://www.wisdom.weizmann.ac.il/~dinuri/mypapers/combpcp.pdf. Article 12. Our theorem numbering refers to the author manuscript dated 13 February 2007.

[6] Serge Gaspers, Tao Zixu He, and Simon Mackenzie. NP-completeness of deterministic communication complexity via relaxed interlacing, 2026. URL https://arxiv.org/abs/2508.05597v4. Earlier version of the present paper, version 4, 13 April 2026.

[7] Serge Gaspers, Tao Zixu He, and Simon Mackenzie. NP-completeness and ETH-based additive inapproximability of deterministic communication complexity via relaxed interlacing, 2026. URL https://arxiv.org/abs/2508.05597v5. Earlier ETH stage of the present paper, version 5, 22 October 2026.

[8] Venkatesan Guruswami and Euiwoong Lee. Strong inapproximability results on balanced rainbow-colorable hypergraphs. *Combinatorica*, 38(3):547–599, 2018. doi: 10.1007/s00493-016-3383-0. URL https://eccc.weizmann.ac.il/report/2014/043/. Theorem numbering in our source discussion refers to ECCC TR14-043.

[9] Shuichi Hirahara, Rahul Ilango, and Bruno Loff. Hardness of constant-round communication complexity. In *36th Computational Complexity Conference (CCC 2021)*, volume 200 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 31:1–31:30, 2021. doi: 10.4230/LIPIcs.CCC.2021.31. URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CCC.2021.31.

[10] Shuichi Hirahara, Rahul Ilango, and Bruno Loff. Communication complexity is NP-hard, 2025. URL https://arxiv.org/abs/2507.10426v1. Preprint, version 1, 14 July 2025.

[11] Pavel Hrubeš. Hard submatrices for non-negative rank and communication complexity. *Computational Complexity*, 34, 2025. doi: 10.1007/s00037-025-00269-4. URL https://doi.org/10.1007/s00037-025-00269-4. Article 9.

[12] Rahul Ilango, Bruno Loff, and Igor Carboni Oliveira. NP-hardness of circuit minimization for multi-output functions. In *35th Computational Complexity Conference (CCC 2020)*, volume 169 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 22:1–22:36, 2020. doi: 10.4230/LIPIcs.CCC.2020.22. URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CCC.2020.22.

[13] Interlacing additive Lean. Interlacing and additive inapproximability: Lean companion. GitHub repository, 2026. URL https://github.com/SimonWMackenzie/interlacing-additive-lean. Companion to version 42 of this manuscript.

[14] Interlacing additive proof inspector. Interlacing and additive inapproximability: Paper and Lean proof inspector. Interactive proof website, 2026. URL https://simonwmackenzie.github.io/interlacing-additive-lean/inspector/. Companion to version 42 of this manuscript.

[15] Daphne Koller and Nimrod Megiddo. Constructing small sample spaces satisfying given constraints. *SIAM Journal on Discrete Mathematics*, 7(2):260–274, 1994. doi: 10.1137/S0895480192228140. URL https://theory.stanford.edu/~megiddo/pdf/stoc93.pdf. First appeared in STOC 1993. Algorithm and theorem numbering here refers to the author-hosted conference version.

[16] Eyal Kushilevitz and Enav Weinreb. On the complexity of communication complexity. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing*, pages 465–474. Association for Computing Machinery, 2009. doi: 10.1145/1536414.1536479. URL https://doi.org/10.1145/1536414.1536479.

[17] Simon Mackenzie. Lossless hardness condensation in deterministic communication complexity, 2026. URL https://arxiv.org/abs/2609.28691v1. Preprint, version 1, 23 September 2026.

[18] Simon Mackenzie and Abdallah Saffidine. Refuting the direct sum conjecture for total functions in deterministic communication complexity, 2026. URL https://arxiv.org/abs/2411.19003v2. Corrected full version, 14 April 2026; conference version appeared in STOC 2025.

[19] Noam Nisan and Avi Wigderson. Rounds in communication complexity revisited. *SIAM Journal on Computing*, 22(1):211–219, 1993. doi: 10.1137/0222016. URL https://doi.org/10.1137/0222016.

[20] Sang-il Oum, Sigve Hortemo Sæther, and Martin Vatshelle. Faster algorithms for vertex partitioning problems parameterized by clique-width. *Theoretical Computer Science*, 535: 16–24, 2014. doi: 10.1016/j.tcs.2014.03.024. URL https://arxiv.org/abs/1311.0224v2. Our theorem numbering refers to arXiv version 2.

[21] Arshia Soltani Moakhar, Iman Gholami, Max Springer, Mahdi JafariRaviz, and Mohammad-Taghi Hajiaghayi. Beyond the Library: An agentic framework for autoformalizing research mathematics, 2026. URL https://arxiv.org/abs/2606.31134v3. Version 3, 5 August 2026; direct-sum formalization in Section 3.2.2 and Appendix C.3.

[22] Andrew Chi-Chih Yao. Some complexity questions related to distributive computing (preliminary report). In *Proceedings of the Eleventh Annual ACM Symposium on Theory of Computing*, pages 209–213. Association for Computing Machinery, 1979. doi: 10.1145/800135.804414. URL https://doi.org/10.1145/800135.804414.